

27-HOUR BUILD GUIDE

Order Management System

Full-Stack Microservices with Spring Boot & Angular

Build it step by step · understand every layer · explain it confidently in interviews.
Includes Spring annotations, MySQL interview queries, JWT, Kafka, Swagger, abstract classes, TypeScript, Angular, Material UI, CSS and a full interview Q&A bank.

Java 21

Spring Boot

Spring Cloud

MySQL

Kafka

JWT

Swagger

Angular

TypeScript

Material UI

Docker

Angular

Gateway

Services

MySQL

Kafka

Contents

01 Start Here	6
1.1 What you are building	6
1.2 Your environment check - what it told us	6
1.3 The Build - Break - Explain method	7
1.4 Your 27-hour plan: Friday, Saturday, Sunday	7
1.5 Where each interview topic is covered	9
02 Architecture	10
2.1 The big picture	10
2.2 Ports and responsibilities	10
2.3 What happens when a customer places an order	11
2.4 Design decisions (be ready to defend each one)	12
2.5 Inside every service: layered architecture	12
2.6 Final folder structure	13
03 Project Setup	14
3.1 Create the folders and Git repository	14
3.2 Generate the six Spring Boot projects	14
3.3 Build each project once	16
3.4 MySQL: databases and an application user	16
3.5 Environment variables (secrets)	17
3.6 Use application.yml instead of application.properties	17
04 Shared Building Blocks	18
4.1 Abstract class #1: BaseEntity	18
4.2 Abstract class #2: BaseException	19
4.3 The error response and the global handler	20
4.4 Swagger / OpenAPI configuration	21
4.5 Abstract class vs interface (frequent interview question)	21
05 Product Service	23
5.1 Configuration	23
5.2 Package structure	23
5.3 Entity (extends the abstract BaseEntity)	23
5.4 DTO, repository, service	24
5.5 Controller with Swagger annotations	26
5.6 Optional sample data	27
5.7 Run and test	27
06 Eureka and API Gateway	29

6.1 The problem Eureka solves	29
6.2 Service Discovery (Eureka Server)	29
6.3 API Gateway	30
6.4 What CORS is (and why Angular needs it)	31
07 User Service and JWT	33
7.1 How JWT login works	33
7.2 Configuration	34
7.3 Entity, role and repository	34
7.4 DTOs	35
7.5 Security configuration and JwtService	36
7.6 AuthService and controller	37
7.7 Gateway security filter (verifies the JWT)	39
7.8 Test the security flow	40
08 Order Service	42
8.1 Configuration	42
8.2 Entities	42
8.3 Repository (with a fetch join)	44
8.4 DTOs	44
8.5 OpenFeign: calling Product Service	45
8.6 Service: the order algorithm	46
8.7 Controller	47
8.8 Test through the Gateway	48
09 Kafka and Notification Service	49
9.1 Kafka in 5 minutes	49
9.2 Run Kafka with Docker Compose	49
9.3 Order Service: publish the event	50
9.4 Notification Service	51
9.5 Abstract class #3: Template Method	53
9.6 Test the event flow	54
10 Resilience4j	56
10.1 Configure timeouts and the breaker	56
10.2 Protect the call	57
10.3 Failure experiment (do this exact script)	57
10.4 The resilience family	58
11 TypeScript Primer	59
11.1 Java vs TypeScript at a glance	59
11.2 Types, interfaces and unions	59
11.3 Functions, arrays and the operators you will use daily	60

11.4	Classes and generics	60
11.5	Promises vs Observables (key for Angular)	60
11.6	Modules, decorators and strict mode	61
12	Angular Application	62
12.1	How Angular talks to Spring Boot	62
12.2	Create the project	62
12.3	Folder structure	63
12.4	Config, models and error helper	63
12.5	AuthService (login + localStorage)	64
12.6	Interceptor and guards	65
12.7	API service (products, orders, notifications)	66
12.8	App wiring: config, routes, root component	66
12.9	Login and register page	67
12.10	Product list (Material table + RxJS search)	69
12.11	Product form (create + edit)	70
12.12	Orders page (FormArray + order history)	71
12.13	Notifications and dashboard	73
12.14	RxJS operators you actually used	75
12.15	Component lifecycle and data binding cheat-sheet	75
13	Material UI and CSS	77
13.1	Angular Material components used in the project	77
13.2	Theming in one minute	77
13.3	Global stylesheet used by the app	78
13.4	CSS essentials (and the interview answers)	79
14	JWT, localStorage and CORS End to End	81
14.1	The complete journey	81
14.2	Token lifecycle	81
14.3	Where to store the token?	82
14.4	CORS: the exact sequence	82
14.5	Debug it like a professional	82
14.6	Common failures and what they mean	83
15	Testing and Integration	84
15.1	The testing pyramid for this project	84
15.2	Unit test: ProductService (Mockito)	84
15.3	Controller test: validation and error JSON	85
15.4	Unit test: the order total is computed on the server	85
15.5	The 14-point integration checklist	86
15.6	Postman tips	87

15.7 Start everything reliably	87
16 Spring Annotations Cheat-Sheet	88
16.1 Core / Spring Boot	88
16.2 Web / REST	88
16.3 Validation (Jakarta Bean Validation)	89
16.4 JPA / Hibernate / Spring Data	89
16.5 Spring Cloud, Kafka, Resilience4j, Swagger	90
16.6 Testing	90
16.7 Concepts behind the annotations	91
17 MySQL Interview Queries	92
17.1 Create the practice database	92
17.2 Basics	93
17.3 Aggregation: GROUP BY and HAVING	93
17.4 Joins	94
17.4b Subqueries	95
17.5 The classic interview questions	95
17.6 Window functions	96
17.7 CASE, UNION, CTE, views, pagination	97
17.8 DML and DDL	98
17.9 Indexes and EXPLAIN	99
17.10 Transactions, ACID and isolation	99
17.11 Theory table: what to say out loud	100
18 Interview Q&A; Bank	101
18.1 Spring Boot and Spring	101
18.2 Microservices and architecture	102
18.3 Java, OOP and JPA	103
18.4 Angular, TypeScript, CSS	104
18.5 Security, Kafka and project-specific	105
19 Finish Line	107
19.1 README template (copy to README.md)	107
19.2 Draw the architecture diagram	107
19.3 Git: commit after every checkpoint	108
19.4 Your 2-minute project pitch	108
19.5 Five-minute live demo script	108
19.6 Master troubleshooting table	109
19.7 After the weekend: what to build next	110
19.8 Final Sunday-night checklist	110

01 Start Here

How to use this guide, what you will learn, and your 27-hour plan

1.1 What you are building

You are building a small but **real** e-commerce backend with a browser UI. A customer logs in, sees products, places an order, and a notification is created in the background. It is deliberately small so that you can understand **every** line, yet it contains the building blocks that interviewers ask about for Java / Spring Boot roles.

Layer	Technology	Why it is in the project (interview angle)
Backend	Java 21, Spring Boot 3.5, Spring Web, Spring Data JPA	REST APIs, layered architecture, dependency injection, annotations
Microservices	Eureka, Spring Cloud Gateway, OpenFeign, Resilience4j	Service discovery, single entry point, sync calls, fault tolerance
Messaging	Apache Kafka	Async, event-driven communication; REST vs Kafka
Database	MySQL 8 (one DB per service)	JPA, joins, indexes, transactions, interview SQL
Security	JWT + BCrypt + Spring Security	Stateless auth, password hashing, role checks
Docs	Swagger / OpenAPI (springdoc)	API documentation and quick testing in the browser
OOP	Abstract classes + Template Method	BaseEntity, BaseException, NotificationHandler
Frontend	Angular, TypeScript, Angular Material, CSS	Components, services, RxJS, interceptor, guard, forms

1.2 Your environment check - what it told us

You ran the checks from the original plan. Here is the result and what to do about each line. Nothing here blocks you - one small decision and two small fixes.

Tool	Your result	Status	What to do
Java	21.0.10 LTS	OK	Use Java 21 for the project (Spring Boot 3.5 fully supports it). Java 17 is not required.
Maven	'mvn' not recognised	OK	You do not need Maven installed. Every Spring Initializr project contains mvnw.cmd (Maven Wrapper). Always use .\mvnw.cmd .
Node / npm	v24.15.0 / 12.1.0	OK	Works with your Angular CLI.
Angular CLI	22.2.0	OK	Use it as is. This guide uses standalone components, signals and @if/@for .
MySQL	'mysql' not recognised in PowerShell, but you said MySQL works	OK	The server + Workbench work; only the command-line client is not on PATH. Optional fix in Chapter 3.
Git	2.56.0	OK	Ready.

Tool	Your result	Status	What to do
Docker	29.7.2, Compose v5.5.0	OK	Ready - we will run Kafka with it.

NOTE Version note (read once)

This guide is written for **Spring Boot 3.5.x** with the matching **Spring Cloud 2025.0.x** release train, **springdoc-openapi 2.8.x** and **JWT 0.12.x**.

If Spring Initializr only offers a different Boot version, open spring.io/projects/spring-cloud and use the Cloud train that matches your Boot version. Gateway property names changed between trains - Chapter 6 shows both ways (explicit routes always work).

1.3 The Build - Break - Explain method

Copying code teaches typing. Interviews test understanding. For **every** step in this guide do three things:

Step	What you do	Example
1 BUILD	Type the code yourself (do not paste). Run it. Hit the checkpoint.	Create the Product CRUD and see rows in MySQL.
2 BREAK	Deliberately break it and read the error.	Stop MySQL, remove <code>@Entity</code> , use a wrong port, send an invalid JSON body.
3 EXPLAIN	Say the answer out loud in 30 seconds (the INTERVIEW boxes give you the wording).	"Eureka is a service registry; services register and send heartbeats..."

Look for these coloured boxes throughout the guide:

TIP Shortcuts

A shortcut or best practice that saves time.

WARNING Common traps

A mistake that commonly costs an hour. Read these.

CHECKPOINT Stop and verify

Do not continue until this works. Most 'mysterious' bugs come from skipping a checkpoint.

INTERVIEW Say it aloud

Questions you can now answer using your own project as the example.

LEARN Go deeper

Concepts to understand deeply, not just copy.

1.4 Your 27-hour plan: Friday, Saturday, Sunday

The order matters. Build a **working core** first (Product - Gateway - User - Order), then add the advanced features (Kafka, Resilience4j), and the UI last. If you fall behind, use the 'cut line' at the end of this section.

DAY 1 · Friday · 9 hours · Foundation

Time	Hrs	What you do	Ch.
0:00-0:40	0.7	Verify tools, create folders, <code>git init</code> , <code>.gitignore</code>	3
0:40-1:40	1.0	Generate 6 Spring Boot projects with Initializr; first build of each	3
1:40-2:10	0.5	MySQL: 4 databases, app user, environment variables	3
2:10-3:00	0.8	Shared building blocks: <code>BaseEntity</code> , <code>BaseException</code> , error handler, Swagger config	4
3:00-4:30	1.5	Product Service CRUD + Swagger + test in Postman/Swagger	5
4:30-5:10	0.7	Eureka server + register Product Service	6
5:10-5:50	0.7	API Gateway routes + CORS	6
5:50-7:30	1.7	User Service : register, login, BCrypt, JWT, admin seed, Gateway JWT filter	7
7:30-9:00	1.5	Practice: SQL set 1 (Chapter 17), annotations revision (Chapter 16), commit to Git	16, 17

DAY 2 · Saturday · 10 hours · Microservices depth

Time	Hrs	What you do	Ch.
0:00-2:30	2.5	Order Service : entities, OpenFeign call to Product, total calculation, order endpoints	8
2:30-4:30	2.0	Kafka (Docker) + producer in Order + Notification Service with abstract handler	9
4:30-5:30	1.0	Validation edge cases + JUnit/Mockito tests for Product and Order services	15
5:30-6:30	1.0	Resilience4j circuit breaker, timeouts, failure experiments	10
6:30-7:30	1.0	Integration test: register, login, product, order, notification (Swagger + Postman)	15
7:30-10:00	2.5	Debug, SQL set 2, explain each component aloud, interview Q&A (Microservices + Spring)	17, 18

DAY 3 · Sunday · 8 hours · Angular + integration

Time	Hrs	What you do	Ch.
0:00-1:00	1.0	Angular project, TypeScript primer, Material, routes, models	11, 12
1:00-3:00	2.0	AuthService, interceptor, guard, login page; product list + form (CRUD)	12, 14
3:00-4:00	1.0	Orders page, notifications page, dashboard	12
4:00-5:00	1.0	CORS fix, polish with CSS, full end-to-end test	13, 14
5:00-6:00	1.0	README, architecture diagram, push to GitHub	19
6:00-7:00	1.0	Mock interview: answer Chapter 18 questions out loud, using your project	18
7:00-8:00	1.0	Buffer / revision / optional Dockerisation	-

WARNING The cut line (if you run late)

Protect this core first: **Product Service - Eureka - Gateway - Order Service (Feign) - Angular product+order pages.**

Then add in this order: JWT - Kafka/Notification - Resilience4j - Swagger polish - Docker.

Never skip the Explain step: a smaller project you can explain beats a bigger one you cannot.

1.5 Where each interview topic is covered

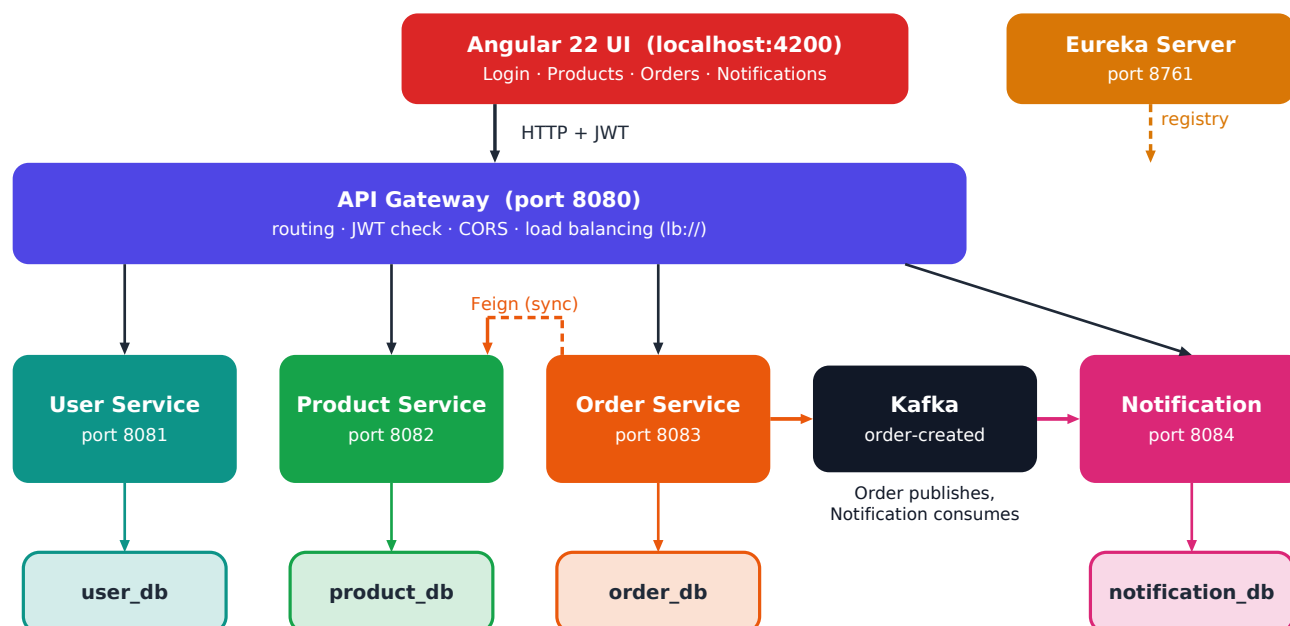
You asked to learn	Where
Spring Boot annotations	Used in every build chapter; full categorised cheat-sheet in Chapter 16
MySQL interview queries	Chapter 17: 40+ queries with a practice database, window functions, indexes, transactions
Microservice architecture	Chapter 2 (design), Ch 6 (Eureka/Gateway), 8 (Feign), 9 (Kafka), 10 (Resilience4j)
Abstract class + Swagger	Chapter 4 (BaseEntity, BaseException, Swagger), Ch 9 (Template Method)
TypeScript	Chapter 11 primer + used in all Angular code
Angular integration with backend	Chapter 12: services, HttpClient, interceptor, guard, forms
Material UI and CSS	Chapter 13 (Angular Material + CSS essentials)
JWT and localStorage, backend to UI	Chapter 14: full end-to-end flow with diagrams and security trade-offs
Interview answers	Chapter 18: Q&A bank (60+ questions) + Chapter 19: how to present the project

02 Architecture

The big picture, request flow and the design decisions behind it

2.1 The big picture

Read this chapter slowly. If you can redraw the diagram below from memory and narrate a request through it, you already have the heart of every microservices interview.



Every service registers with Eureka (dashed) — Gateway and Feign ask Eureka where "product-service" lives.

Figure 2.1 - System architecture: one entry point, four business services, one database per service, Kafka for events

2.2 Ports and responsibilities

Component	Port	Database	Responsibility
Angular UI	4200	-	Screens; sends HTTP requests with JWT to the Gateway only
API Gateway	8080	-	Single entry point; routes <code>/products/**</code> etc.; validates JWT; CORS; load balances with <code>lb://</code>
Eureka Server	8761	-	Service registry: who is running and where
User Service	8081	user_db	Register, login, BCrypt hashing, creates JWT
Product Service	8082	product_db	Product catalogue CRUD and stock
Order Service	8083	order_db	Creates orders; calls Product (Feign); publishes Kafka event
Notification Service	8084	notification_db	Consumes Kafka event; stores notification
Kafka	9092	-	Event log; decouples Order from Notification

LEARN Eureka does NOT route requests

Eureka is only a **phone book**. The Gateway asks it 'where is product-service?' and then **the Gateway** forwards the request. Interviewers love this distinction: **service discovery** (Eureka) vs **routing/edge** (Gateway).

2.3 What happens when a customer places an order

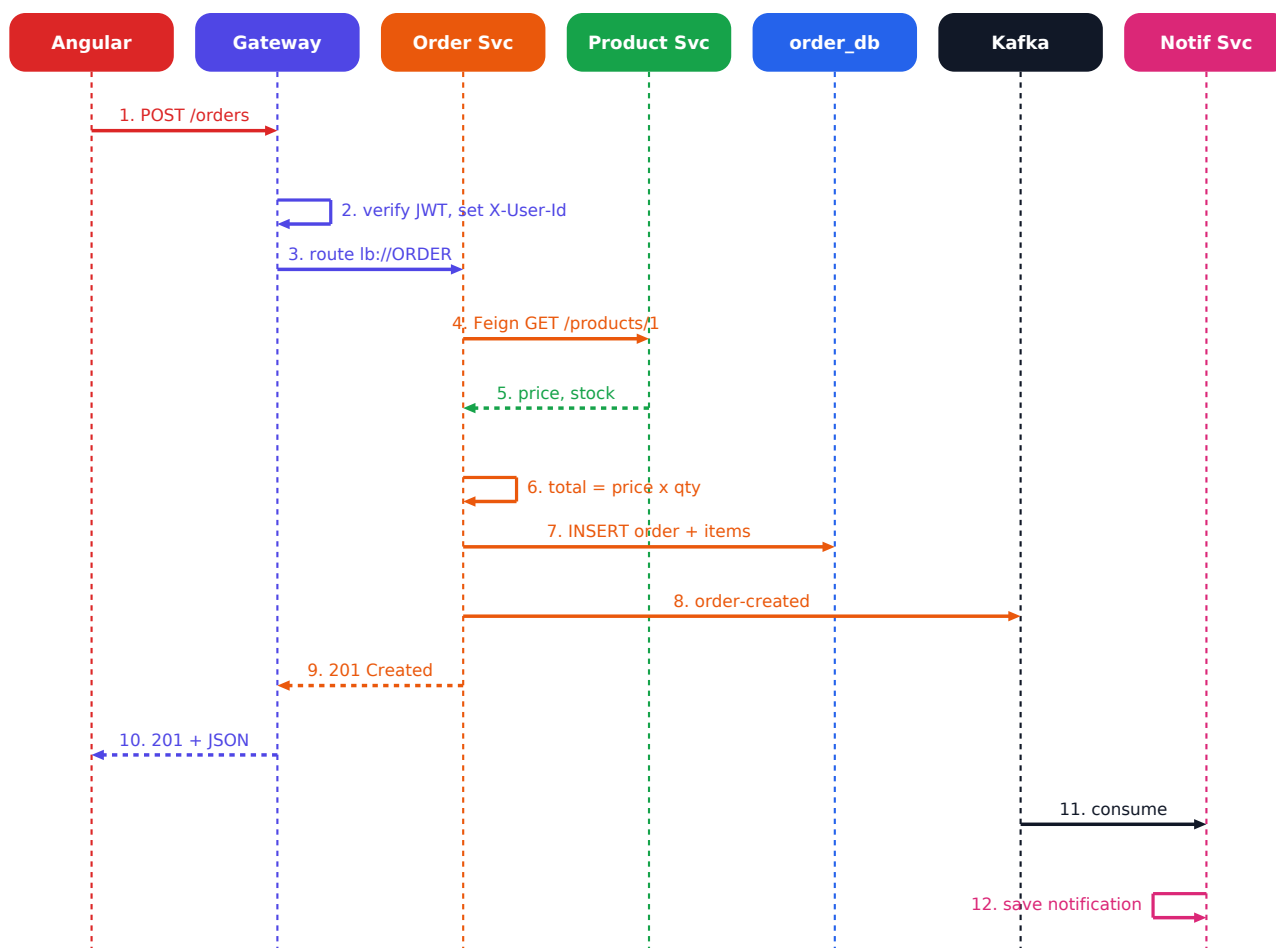


Figure 2.2 - Sequence of a create-order request (steps 1-10 are synchronous; 11-12 happen asynchronously)

1. Angular sends **POST /orders** with **Authorization: Bearer <jwt>** to the **Gateway** (never directly to Order Service).
2. The Gateway verifies the JWT signature, rejects bad tokens, and adds **X-User-Id / X-User-Role** headers for downstream services.
3. The Gateway looks up **ORDER-SERVICE** in Eureka and forwards the request (client-side load balancing).
4. Order Service calls Product Service through **OpenFeign** (synchronous HTTP) to read the **real** price and stock.
5. Order Service computes the total **on the server** - the browser is never trusted for money.
6. The order and its items are saved to **order_db** in one database transaction.
7. An **OrderCreatedEvent** is published to the Kafka topic **order-created** and the HTTP response returns immediately.
8. Later, Notification Service consumes the event and stores a notification. If Notification Service is down, Kafka keeps the event until it returns.

2.4 Design decisions (be ready to defend each one)

Decision	Why	Trade-off you should mention
Database per service	Each service owns its data; no hidden coupling; independent schema changes	No cross-service JOIN or single ACID transaction; use API calls/events
Product name + price snapshot in OrderItem	Historical orders stay correct even if the price changes tomorrow	Some duplicated data (intentional)
Total calculated in Order Service	Never trust the client for money	Needs a call to Product Service (so we protect it with a circuit breaker)
JWT validated at the Gateway	One place for authentication; services stay simple	Services must not be reachable from outside; add service-level checks for defence in depth
userId taken from JWT (header), not from the body	A client cannot place an order *as someone else*	Direct calls to Order Service need the header (Swagger shows it)
Kafka for notification	Order creation does not wait for notification; services are decoupled	Eventual consistency; real systems need outbox + idempotent consumers
DTOs, not entities, in APIs	API contract is separate from table design; avoids leaking fields	Extra mapping code
<code>ddl-auto=update</code>	Fast learning: tables are created for you	Production uses Flyway/Liquibase migrations

2.5 Inside every service: layered architecture

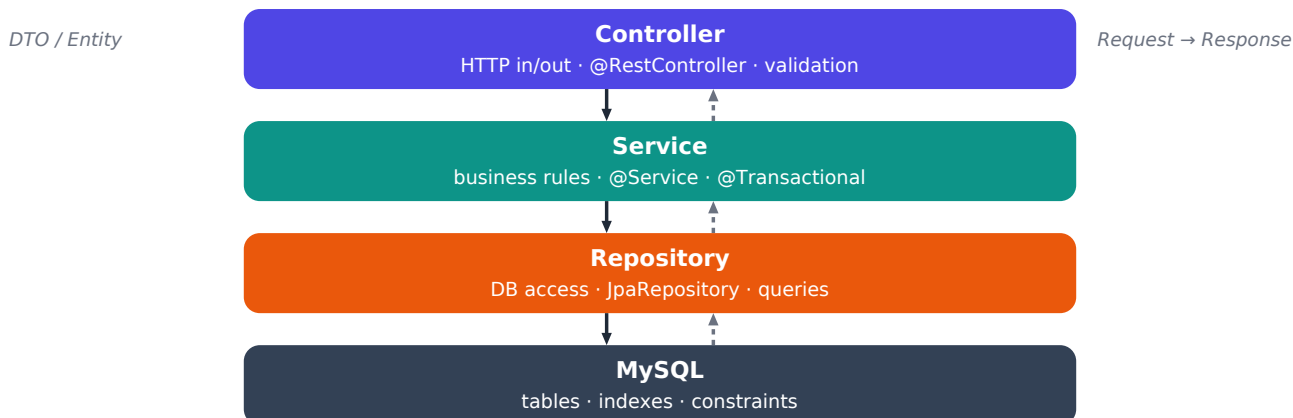


Figure 2.3 - Request goes down through the layers, data comes back up. Each layer has one job.

Layer	Annotation	Rule of thumb
Controller	<code>@RestController</code>	Only HTTP: parse input, call service, return status. No business logic.
Service	<code>@Service</code> , <code>@Transactional</code>	Business rules (stock check, total). Throws meaningful exceptions.
Repository	<code>@Repository</code> / <code>JpaRepository</code>	Database access only.
Entity	<code>@Entity</code>	Maps to a table. Never returned directly from controllers in a real project.

Layer	Annotation	Rule of thumb
DTO	Java <code>record</code>	Defines what the client may send / receive. Validated with <code>@Valid</code> .

2.6 Final folder structure

Repository layout	TEXT
<pre>order-management-system/ -- backend/ -- service-discovery/ (Eureka, 8761) -- api-gateway/ (Gateway, 8080) -- user-service/ (8081) -- product-service/ (8082) -- order-service/ (8083) -- notification-service/ (8084) -- frontend/ -- order-management-ui/ (Angular, 4200) -- infrastructure/ -- docker-compose.yml (Kafka) -- start-all.ps1 (start everything in order) -- docs/ -- architecture.png -- .gitignore -- README.md</pre>	

INTERVIEW Why microservices? (30-second answer)

"I split the system by business capability - users, products, orders, notifications - so each service can be developed, deployed and scaled independently and owns its own database. The price is distributed-system complexity: network failures, eventual consistency and more operations work. That is why I added Eureka, a Gateway, a circuit breaker and Kafka in my project."

03 Project Setup

Folders, Git, Spring Initializr, MySQL and environment variables

3.1 Create the folders and Git repository

Open **PowerShell** and run:

PowerShell

BASH

```
mkdir E:\order-management-system
cd E:\order-management-system
mkdir backend, frontend, infrastructure, docs
git init
```

Create **.gitignore** in the root (VS Code: File > New File):

.gitignore

TEXT

```
target/
node_modules/
dist/
.angular/
.env
.idea/
.vscode/
*.iml
*.log
```

WARNING Never commit secrets

Database passwords and the JWT secret live in **environment variables**, never in Git. A password pushed to GitHub is public forever.

3.2 Generate the six Spring Boot projects

Open **start.spring.io** and use the same base settings for every project:

Setting	Value
Project / Language	Maven / Java
Spring Boot	3.5.x (latest 3.5 offered; see the version note in Chapter 1)
Group	com.af rudeen
Packaging / Java	Jar / 21
Package name	Click *Configure / Options* and set it explicitly (table below). This avoids odd generated names.

Artifact	Package name	Dependencies to tick in Initializr
service-discovery	com.af rudeen.discovery	Eureka Server
api-gateway	com.af rudeen.gateway	Reactive Gateway, Eureka Discovery Client, Spring Boot Actuator

Artifact	Package name	Dependencies to tick in Initializr
user-service	com.afrudeen.user	Spring Web, Spring Data JPA, MySQL Driver, Validation, Spring Security, Eureka Discovery Client, Actuator
product-service	com.afrudeen.product	Spring Web, Spring Data JPA, MySQL Driver, Validation, Eureka Discovery Client, Actuator
order-service	com.afrudeen.order	Spring Web, Spring Data JPA, MySQL Driver, Validation, OpenFeign, Eureka Discovery Client, Actuator, Spring for Apache Kafka
notification-service	com.afrudeen.notification	Spring Web, Spring Data JPA, MySQL Driver, Validation, Spring for Apache Kafka, Eureka Discovery Client, Actuator

Click **Generate**, unzip each project and move it so the folder is exactly

`E:\order-management-system\backend\<artifact>`. Then add the extra dependencies below (Initializr does not list them).

Extra dependencies (add inside <dependencies> of each pom.xml)

pom.xml - Swagger / OpenAPI	XML
<pre> <!-- Swagger UI: user-service, product-service, order-service, notification-service --> <dependency> <groupId>org.springdoc</groupId> <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId> <version>2.8.9</version> <!-- use the newest 2.8.x on Maven Central --> </dependency> <!-- Swagger UI: api-gateway only (reactive stack) --> <dependency> <groupId>org.springdoc</groupId> <artifactId>springdoc-openapi-starter-webflux-ui</artifactId> <version>2.8.9</version> </dependency> </pre>	
pom.xml - JWT and Resilience4j	XML
<pre> <!-- JWT: user-service AND api-gateway --> <dependency> <groupId>io.jsonwebtoken</groupId><artifactId>jjwt-api</artifactId><version>0.12.6</version> </dependency> <dependency> <groupId>io.jsonwebtoken</groupId><artifactId>jjwt-impl</artifactId> <version>0.12.6</version><scope>runtime</scope> </dependency> <dependency> <groupId>io.jsonwebtoken</groupId><artifactId>jjwt-jackson</artifactId> <version>0.12.6</version><scope>runtime</scope> </dependency> <!-- Resilience4j: order-service only --> <dependency> <groupId>io.github.resilience4j</groupId> <artifactId>resilience4j-spring-boot3</artifactId> <version>2.3.0</version> </dependency> <dependency> <groupId>org.springframework.boot</groupId> <artifactId>spring-boot-starter-aop</artifactId> </dependency> </pre>	

LEARN What is the Spring Cloud BOM?

When you tick a Cloud dependency, Initializr adds a `<dependencyManagement>` block importing **spring-cloud-dependencies**. A BOM (Bill of Materials) fixes compatible versions, which is why Gateway, Eureka and Feign dependencies have **no version number**. Never type random versions for those.

3.3 Build each project once

Before writing any code, prove each project compiles. In PowerShell, inside each service folder:

PowerShell (repeat for all six)

BASH

```
cd E:\order-management-system\backend\product-service
.\mvnw.cmd clean package -DskipTests
```

The first run downloads dependencies and takes a few minutes. You want **BUILD SUCCESS**. If you see a dependency error, fix it now.

TIP Opening everything in IntelliJ

Open the **backend** folder, then in the **Maven** tool window click **+** and select each **pom.xml**. All six services appear in one window and the **Services** tool window lets you start and stop them with buttons.

3.4 MySQL: databases and an application user

In **MySQL Workbench**, open a query tab and run (this is your first interview-relevant SQL: DDL and privileges):

MySQL Workbench

SQL

```
CREATE DATABASE user_db;
CREATE DATABASE product_db;
CREATE DATABASE order_db;
CREATE DATABASE notification_db;

CREATE USER 'order_app'@'localhost' IDENTIFIED BY 'ChangeThis_123';

GRANT ALL PRIVILEGES ON user_db.* TO 'order_app'@'localhost';
GRANT ALL PRIVILEGES ON product_db.* TO 'order_app'@'localhost';
GRANT ALL PRIVILEGES ON order_db.* TO 'order_app'@'localhost';
GRANT ALL PRIVILEGES ON notification_db.* TO 'order_app'@'localhost';
FLUSH PRIVILEGES;

SHOW DATABASES;
```

CHECKPOINT Checkpoint

SHOW DATABASES; lists all four databases. Replace **ChangeThis_123** with your own password and remember it.

NOTE Optional: make mysql work in PowerShell

Your MySQL works through Workbench, so this is optional. To use the command line, add the MySQL **bin** folder to PATH, e.g. **C:\Program Files\MySQL\MySQL Server 8.x\bin** (search 'Edit environment variables'), then open a **new** PowerShell and run **mysql --version**.

3.5 Environment variables (secrets)

Every service reads `${DB_PASSWORD}`; User Service and Gateway also read `${JWT_SECRET}`. Choose **one** method:

Method	How	Notes
A IntelliJ (recommended)	Run > Edit Configurations > select the service > Environment variables: <code>DB_PASSWORD=ChangeThis_123;JWT_SECRET=...</code>	Per run-configuration; easiest while learning
B Windows user variables	PowerShell: <code>setx DB_PASSWORD "ChangeThis_123"</code> then restart IntelliJ/terminals	Applies to every program you start
C Per terminal	<code>\$env:DB_PASSWORD="ChangeThis_123"</code> before <code>.\mvnw.cmd</code> <code>spring-boot:run</code>	Lost when the window closes

Generate a strong JWT secret (at least 32 characters; HS256 needs 256 bits). In PowerShell:

```
PowerShell - random 48-character secret BASH

-join ((48..57)+(65..90)+(97..122) | Get-Random -Count 48 | % {[char]$_})
```

WARNING Same secret in two places

User Service **signs** tokens and the Gateway **verifies** them, so both must have the **identical** `JWT_SECRET`. A mismatch gives `401 Unauthorized` on every request.

3.6 Use application.yml instead of application.properties

Initializr creates `application.properties`. Delete it and create `application.yml` in the same folder - YAML is shorter for nested config and is required by the Gateway chapter. Indentation uses **2 spaces, never tabs**.

04 Shared Building Blocks

Abstract classes, one exception model, a global error handler, and Swagger

Before the first service, we create four small pieces that **every** service reuses. This is where we use **abstract classes** and **Swagger**. Create them in package `com.af rudeen.product.common` first; for other services copy the files and change only the `package` line.

LEARN Why copy instead of a shared library?

In real teams a shared `common-lib` is sometimes used, but it couples every service to one release cycle. For a first project, copying tiny base classes keeps services independent. Mention this trade-off in interviews.

4.1 Abstract class #1: BaseEntity

Every table needs an id and audit timestamps. Instead of repeating them in four entities, put them in one **abstract** class. `@MappedSuperclass` tells JPA: 'my fields become columns in each child's table, but I am not a table myself'.

`common/BaseEntity.java`

JAVA

```
package com.af rudeen.product.common;

import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;
import java.time.LocalDateTime;

@MappedSuperclass
public abstract class BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, updatable = false)
    private LocalDateTime createdAt;

    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
        this.updatedAt = this.createdAt;
    }

    @PreUpdate
    protected void onUpdate() {
        this.updatedAt = LocalDateTime.now();
    }

    /** Every child must say how it should appear in logs. */
    @JsonIgnore // keep it out of the JSON sent to clients
    public abstract String getDisplayName();

    public Long getId() { return id; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public LocalDateTime getUpdatedAt() { return updatedAt; }
}
```

- `@PrePersist` / `@PreUpdate` are JPA **lifecycle callbacks** - they run automatically right before INSERT / UPDATE.
- `abstract String getDisplayName()` has **no body**; the compiler forces `Product`, `Order`, `User` and `Notification` to implement it.
- You cannot write `new BaseEntity()` - an abstract class can never be instantiated.

4.2 Abstract class #2: BaseException

All business errors share the same shape: a message and an HTTP status. An abstract exception lets **one** handler deal with them all.

`common/BaseException.java`

JAVA

```
package com.af rudeen.product.common;

import org.springframework.http.HttpStatus;

public abstract class BaseException extends RuntimeException {

    protected BaseException(String message) {
        super(message);
    }

    /** Each concrete exception decides its own HTTP status. */
    public abstract HttpStatus getStatus();
}
```

`common/ResourceNotFoundException.java`

JAVA

```
package com.af rudeen.product.common;

import org.springframework.http.HttpStatus;

public class ResourceNotFoundException extends BaseException {
    public ResourceNotFoundException(String message) { super(message); }

    @Override
    public HttpStatus getStatus() { return HttpStatus.NOT_FOUND; } // 404
}
```

`BusinessException.java and ServiceUnavailableException.java (separate files)`

JAVA

```
public class BusinessException extends BaseException {
    public BusinessException(String message) { super(message); }

    @Override
    public HttpStatus getStatus() { return HttpStatus.BAD_REQUEST; } // 400
}

public class ServiceUnavailableException extends BaseException {
    public ServiceUnavailableException(String message) { super(message); }

    @Override
    public HttpStatus getStatus() { return HttpStatus.SERVICE_UNAVAILABLE; } // 503
}
```

Put each public class in its own file with the same name. Create `BusinessException` and `ServiceUnavailableException` only in services that use them (Order and User need them; Product needs only the not-found one).

4.3 The error response and the global handler

common/ApiError.java

JAVA

```
package com.afrudeen.product.common;

import java.time.Instant;
import java.util.Map;

public record ApiError(
    Instant timestamp,
    int status,
    String error,
    String message,
    String path,
    Map<String, String> fieldErrors) {
}
```

common/GlobalExceptionHandler.java

JAVA

```
package com.afrudeen.product.common;

import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.*;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;
import java.time.Instant;
import java.util.*;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(BaseException.class)
    public ResponseEntity<ApiError> handleBase(BaseException ex, HttpServletRequest req) {
        HttpStatus status = ex.getStatus(); // polymorphism in action
        return build(status, ex.getMessage(), req, null);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError> handleValidation(MethodArgumentNotValidException ex,
        HttpServletRequest req) {
        Map<String, String> fields = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors()
            .forEach(fe -> fields.put(fe.getField(), fe.getDefaultMessage()));
        return build(HttpStatus.BAD_REQUEST, "Validation failed", req, fields);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiError> handleOther(Exception ex, HttpServletRequest req) {
        return build(HttpStatus.INTERNAL_SERVER_ERROR, "Unexpected error", req, null);
    }

    private ResponseEntity<ApiError> build(HttpStatus status, String msg,
        HttpServletRequest req, Map<String, String> fields) {
        ApiError body = new ApiError(Instant.now(), status.value(), status.getReasonPhrase(),
            msg, req.getRequestURI(), fields);
        return ResponseEntity.status(status).body(body);
    }
}
```

LEARN How does @RestControllerAdvice work?

It is a global interceptor for controller exceptions. When any controller throws, Spring finds the **most specific** `@ExceptionHandler` method (a `ResourceNotFoundException` matches `BaseException` before `Exception`). Clients get clean JSON with the right status instead of a stack trace.

4.4 Swagger / OpenAPI configuration

The `springdoc` dependency (Chapter 3) already generates API docs automatically from your controllers. This small class only adds a title and a **Authorize** button for JWT.

common/OpenApiConfig.java

JAVA

```
package com.af rudeen.product.common;

import io.swagger.v3.oas.annotations.OpenAPIDefinition;
import io.swagger.v3.oas.annotations.enums.SecuritySchemeType;
import io.swagger.v3.oas.annotations.info.Info;
import io.swagger.v3.oas.annotations.security.SecurityScheme;
import org.springframework.context.annotation.Configuration;

@Configuration
@OpenAPIDefinition(info = @Info(
    title = "Product Service API",
    version = "1.0",
    description = "Product catalogue and stock"))
@SecurityScheme(
    name = "bearerAuth",
    type = SecuritySchemeType.HTTP,
    scheme = "bearer",
    bearerFormat = "JWT")
public class OpenApiConfig {
}
```

URL	What you get
http://localhost:8082/swagger-ui/index.html	Interactive page: expand an endpoint, Try it out , Execute
http://localhost:8082/v3/api-docs	The raw OpenAPI JSON (what Swagger UI reads)

Swagger annotation	Put it on	Purpose
@Tag(name=...)	Controller class	Groups endpoints under a heading
@Operation(summary=...)	Controller method	Short description of the endpoint
@ApiResponse(responseCode=...)	Controller method	Documents status codes
@Schema(description, example)	DTO field / record component	Describes fields and example values
@SecurityRequirement(name="bearerAuth")	Controller or method	Shows the padlock and sends the token
@Parameter(description=...)	Method parameter	Describes path/query/header params

4.5 Abstract class vs interface (frequent interview question)

	Abstract class	Interface
Purpose	Shared state + behaviour for closely related classes (IS-A)	A capability/contract that unrelated classes can promise
Fields	Instance fields allowed (<code>id</code> , <code>createdAt</code>)	Only <code>public static final</code> constants
Methods	Abstract and concrete methods, constructors	Abstract, <code>default</code> , <code>static</code> , <code>private</code> methods; no constructors
Inheritance	A class extends one abstract class	A class implements many interfaces

	Abstract class	Interface
In our project	<code>BaseEntity</code> , <code>BaseException</code> , <code>NotificationHandler</code>	<code>JpaRepository</code> , <code>ProductClient</code> (Feign)
Choose when	You have shared fields or a fixed algorithm skeleton	You only need to define a contract

INTERVIEW Where did you use an abstract class in your project, and why?

"Three places. `BaseEntity` is a `@MappedSuperclass` holding id and audit timestamps so the four entities do not repeat them. `BaseException` forces every business exception to declare its HTTP status, so one `@RestControllerAdvice` handles them all through polymorphism. And `NotificationHandler` is the Template Method pattern: the base class fixes the algorithm - validate, build message, save - and subclasses supply only the message."

05 Product Service

Your first complete service: entity, repository, service, controller, Swagger

This service is the **template** for all others. Learn this pattern once and the rest is repetition.

5.1 Configuration

product-service/src/main/resources/application.yml

YAML

```
spring:
  application:
    name: product-service
  datasource:
    url: jdbc:mysql://localhost:3306/product_db
    username: order_app
    password: ${DB_PASSWORD}
  jpa:
    hibernate:
      ddl-auto: update          # learning only; production uses Flyway/Liquibase
      show-sql: true
      properties:
        hibernate:
          format_sql: true

server:
  port: 8082

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/

management:
  endpoints:
    web:
      exposure:
        include: health,info
```

5.2 Package structure

Package layout

TEXT

```
com.af rudeen.product
|-- ProductServiceApplication.java
|-- common/      BaseEntity, BaseException, ResourceNotFoundException,
|                ApiError, GlobalExceptionHandler, OpenApiConfig
|-- controller/  ProductController
|-- service/     ProductService
|-- repository/  ProductRepository
|-- entity/      Product
`-- dto/         ProductRequest
```

5.3 Entity (extends the abstract BaseEntity)

entity/Product.java

JAVA

```
package com.af rudeen.product.entity;

import com.af rudeen.product.common.BaseEntity;
import jakarta.persistence.*;
import java.math.BigDecimal;

@Entity
@Table(name = "products")
public class Product extends BaseEntity {

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal price;

    @Column(nullable = false)
    private Integer stock;

    protected Product() { } // required by JPA

    public Product(String name, BigDecimal price, Integer stock) {
        this.name = name;
        this.price = price;
        this.stock = stock;
    }

    @Override
    public String getDisplayName() { return name; } // implements the abstract method

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public BigDecimal getPrice() { return price; }
    public void setPrice(BigDecimal price) { this.price = price; }
    public Integer getStock() { return stock; }
    public void setStock(Integer stock) { this.stock = stock; }
}
```

LEARN Why BigDecimal for money?

`double` cannot represent 0.1 exactly ($0.1 + 0.2 = 0.30000000000000004$). `BigDecimal` is exact, and `precision=10, scale=2` creates `DECIMAL(10,2)` in MySQL.

5.4 DTO, repository, service

dto/ProductRequest.java

JAVA

```
package com.af rudeen.product.dto;

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.*;
import java.math.BigDecimal;

public record ProductRequest(
    @Schema(example = "Fuel Injector") @NotBlank String name,
    @Schema(example = "1299.00") @NotNull @DecimalMin("0.01") BigDecimal price,
    @Schema(example = "20") @NotNull @Min(0) Integer stock) {
}
```

repository/ProductRepository.java

JAVA

```
package com.afrudeen.product.repository;

import com.afrudeen.product.entity.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface ProductRepository extends JpaRepository<Product, Long> {

    // Spring Data builds the SQL from the method name:
    // SELECT * FROM products WHERE LOWER(name) LIKE LOWER('%text%')
    List<Product> findByNameContainingIgnoreCase(String text);
}
```

service/ProductService.java

JAVA

```
package com.afrudeen.product.service;

import com.afrudeen.product.common.ResourceNotFoundException;
import com.afrudeen.product.dto.ProductRequest;
import com.afrudeen.product.entity.Product;
import com.afrudeen.product.repository.ProductRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;

@Service
public class ProductService {

    private static final Logger log = LoggerFactory.getLogger(ProductService.class);
    private final ProductRepository repository;

    public ProductService(ProductRepository repository) { // constructor injection
        this.repository = repository;
    }

    @Transactional
    public Product create(ProductRequest r) {
        Product saved = repository.save(new Product(r.name(), r.price(), r.stock()));
        log.info("Created product {}", saved.getDisplayName());
        return saved;
    }

    @Transactional(readOnly = true)
    public List<Product> findAll(String search) {
        if (search == null || search.isBlank()) return repository.findAll();
        return repository.findByNameContainingIgnoreCase(search);
    }

    @Transactional(readOnly = true)
    public Product findById(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Product not found: " + id));
    }

    @Transactional
    public Product update(Long id, ProductRequest r) {
        Product p = findById(id);
        p.setName(r.name());
        p.setPrice(r.price());
        p.setStock(r.stock());
        return p; // dirty checking: JPA issues UPDATE on commit
    }

    @Transactional
    public void delete(Long id) {
        repository.delete(findById(id));
    }
}
```

```
}
}
```

LEARN Dirty checking

Inside a `@Transactional` method, entities loaded from the database are **managed**. If you change a field, Hibernate notices at commit and runs the `UPDATE` itself - no `save()` call needed. This is a favourite interview question.

5.5 Controller with Swagger annotations

controller/ProductController.java

JAVA

```
package com.afrudeen.product.controller;

import com.afrudeen.product.dto.ProductRequest;
import com.afrudeen.product.entity.Product;
import com.afrudeen.product.service.ProductService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/products")
@Tag(name = "Products", description = "Product catalogue")
public class ProductController {

    private final ProductService service;

    public ProductController(ProductService service) { this.service = service; }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    @Operation(summary = "Create a product")
    public Product create(@Valid @RequestBody ProductRequest request) {
        return service.create(request);
    }

    @GetMapping
    @Operation(summary = "List products, optionally filtered by name")
    public List<Product> findAll(@RequestParam(required = false) String search) {
        return service.findAll(search);
    }

    @GetMapping("/{id}")
    @Operation(summary = "Get one product")
    public Product findById(@PathVariable Long id) { return service.findById(id); }

    @PutMapping("/{id}")
    @Operation(summary = "Update a product")
    public Product update(@PathVariable Long id, @Valid @RequestBody ProductRequest request) {
        return service.update(id, request);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    @Operation(summary = "Delete a product")
    public void delete(@PathVariable Long id) { service.delete(id); }
}
```

Returning the entity keeps the first version short. A stricter design returns a `ProductResponse` record; Order Service already uses its own `ProductResponse` DTO.

5.6 Optional sample data

SampleData.java

JAVA

```
package com.afrudeen.product;

import com.afrudeen.product.entity.Product;
import com.afrudeen.product.repository.ProductRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import java.math.BigDecimal;

@Configuration
public class SampleData {

    @Bean
    CommandLineRunner seedProducts(ProductRepository repo) {
        return args -> {
            if (repo.count() == 0) {
                repo.save(new Product("Fuel Injector", new BigDecimal("1299.00"), 20));
                repo.save(new Product("Oil Filter", new BigDecimal("349.50"), 100));
                repo.save(new Product("Piston Ring Set", new BigDecimal("2150.00"), 15));
            }
        };
    }
}
```

5.7 Run and test

PowerShell (or press Run in IntelliJ)

BASH

```
cd E:\order-management-system\backend\product-service
$env:DB_PASSWORD="ChangeThis_123"
.\mvnw.cmd spring-boot:run
```

Open Swagger at <http://localhost:8082/swagger-ui/index.html> and try each endpoint (or use Postman):

Method	URL	Body / result
POST	/products	{"name": "Keyboard", "price": 1299.00, "stock": 20} -> 201 with id
GET	/products	200 list
GET	/products/1	200 one product; unknown id -> 404 JSON error
PUT	/products/1	200 updated; invalid body -> 400 with fieldErrors
DELETE	/products/1	204 no content

CHECKPOINT Checkpoint

- Create, read, update and delete a product from Swagger.
- Open Workbench: `SELECT * FROM product_db.products;` shows your rows, including `created_at`.
- Send `price: -5` and read the 400 response. Send `/products/999` and read the 404.

INTERVIEW Common questions from this chapter

- **What does `@SpringBootApplication` do?** It combines `@Configuration`, `@EnableAutoConfiguration` and `@ComponentScan`.
- **Why constructor injection?** Immutable fields, easy to unit test without Spring, clear required dependencies.
- **PUT vs PATCH?** PUT replaces the full resource; PATCH changes part of it.
- **What does `ddl-auto=update` do?** Hibernate adds missing tables/columns at startup. Not for production.

06 Eureka and API Gateway

Service discovery, one front door, and CORS

6.1 The problem Eureka solves

Today Product Service runs on 8082. Tomorrow you run **three copies** on random ports in the cloud. How does Order Service find one? Hard-coding `http://localhost:8082` breaks immediately. With Eureka, services **register** their name and address, send **heartbeats**, and others look up by name.



Figure 6.1 - Register, heartbeat every 30 s, lookup by name. If heartbeats stop, Eureka removes the instance.

6.2 Service Discovery (Eureka Server)

ServiceDiscoveryApplication.java

JAVA

```

package com.afrudeen.discovery;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class ServiceDiscoveryApplication {
    public static void main(String[] args) {
        SpringApplication.run(ServiceDiscoveryApplication.class, args);
    }
}
  
```

service-discovery/src/main/resources/application.yml

YAML

```

spring:
  application:
    name: service-discovery
server:
  port: 8761
eureka:
  client:
    register-with-eureka: false    # the server does not register with itself
    fetch-registry: false
  
```

Start it and open **`http://localhost:8761`**. You will see an empty 'Instances currently registered' table. Then start Product Service - within about 30 seconds **PRODUCT-SERVICE** appears. (Product Service already has the Eureka client config from Chapter 5.)

CHECKPOINT Checkpoint

PRODUCT-SERVICE shows **UP** in the Eureka dashboard. Stop Product Service and watch it eventually disappear.

WARNING Eureka is slow to notice changes

Registration and cache refresh take up to ~30 seconds. If a call fails right after startup, wait and retry before changing code.

6.3 API Gateway

The Gateway is the **only** URL the browser knows (<http://localhost:8080>). It routes by path, validates JWTs (Chapter 7), and handles CORS in one place.

api-gateway/src/main/resources/application.yml

YAML

```
spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      server:
        webflux: # Spring Cloud 2025.0.x property prefix
      discovery:
        locator:
          enabled: true # auto routes /<service-id>/** (used for Swagger below)
          lower-case-service-id: true
      routes:
        - id: user-service
          uri: lb://USER-SERVICE
          predicates:
            - Path=/users/**
        - id: product-service
          uri: lb://PRODUCT-SERVICE
          predicates:
            - Path=/products/**
        - id: order-service
          uri: lb://ORDER-SERVICE
          predicates:
            - Path=/orders/**
        - id: notification-service
          uri: lb://NOTIFICATION-SERVICE
          predicates:
            - Path=/notifications/**
      globalcors:
        cors-configurations:
          '[/**]':
            allowedOrigins: "http://localhost:4200"
            allowedMethods: [GET, POST, PUT, DELETE, OPTIONS]
            allowedHeaders: [Authorization, Content-Type]

server:
  port: 8080

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/

jwt:
  secret: ${JWT_SECRET}

springdoc:
  swagger-ui:
    urls:
      - name: user-service
```

```
url: /user-service/v3/api-docs
- name: product-service
  url: /product-service/v3/api-docs
- name: order-service
  url: /order-service/v3/api-docs
- name: notification-service
  url: /notification-service/v3/api-docs
```

WARNING Gateway property names depend on the Spring Cloud version

The block above is for **Spring Cloud 2025.0.x** (`spring.cloud.gateway.server.webflux...`) with the starter `spring-cloud-starter-gateway-server-webflux`.

On **older** trains (2024.x and before) the prefix is `spring.cloud.gateway`, directly (for example `spring.cloud.gateway.routes`) and the starter is `spring-cloud-starter-gateway`.

If the Gateway ignores your routes, this prefix mismatch is the first thing to check.

How a route reads

Part	Meaning
<code>id</code>	A label for the route
<code>uri: lb://PRODUCT-SERVICE</code>	<code>lb://</code> = load balanced : look up PRODUCT-SERVICE in Eureka, pick an instance
<code>predicates: Path=/products/**</code>	Match condition: use this route when the URL path starts with /products
<code>globalcors</code>	Answers the browser's CORS preflight (OPTIONS) for <code>http://localhost:4200</code>

Start order and test

1. Start Eureka, then Product Service, then the Gateway (wait ~30 s).
2. Test `GET http://localhost:8080/products` - you get the same list as `http://localhost:8082/products`.
3. Open `http://localhost:8080/swagger-ui.html` - a dropdown lets you switch between every service's API docs.

CHECKPOINT Checkpoint

`GET /products` through port **8080** returns your products, and Eureka lists both PRODUCT-SERVICE and API-GATEWAY.

6.4 What CORS is (and why Angular needs it)

Your Angular page is served from **origin** `http://localhost:4200`. It calls `http://localhost:8080`. Different port = different origin, so the browser blocks the response unless the server says it is allowed. For requests with an `Authorization` header the browser first sends a **preflight** `OPTIONS` request asking 'may origin 4200 send these methods and headers?'. The Gateway's `globalcors` answers yes.

WARNING Configure CORS in only ONE place

If both the Gateway and a service add `Access-Control-Allow-Origin`, the browser sees it twice and rejects the response. Keep CORS only in the Gateway.

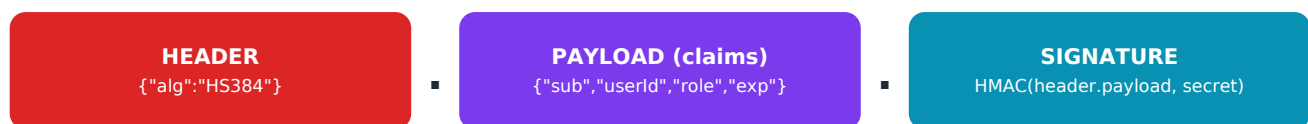
INTERVIEW Eureka vs API Gateway; and what is client-side load balancing?

- **Eureka** answers *where is service X right now?* **Gateway** answers *where should this incoming request go, and is it allowed?*
- In `lb://PRODUCT-SERVICE`, the caller itself picks one of the registered instances (round-robin) - *client-side* load balancing via Spring Cloud LoadBalancer.
- **Heartbeat**: every ~30 s each service tells Eureka it is alive; missing heartbeats lead to eviction.
- **Self-preservation mode**: if many heartbeats are lost at once, Eureka assumes a network problem and stops evicting.

07 User Service and JWT

Registration, BCrypt, login, signed tokens and the Gateway security filter

7.1 How JWT login works



Base64URL-encoded and joined with dots: xxxxx.yyyyy.zzzzz (readable by anyone - NOT encrypted; only tamper-proof).
Anyone can read the payload; nobody can change it without the secret, because the signature would no longer match.

Figure 7.1 - Anatomy of a JSON Web Token

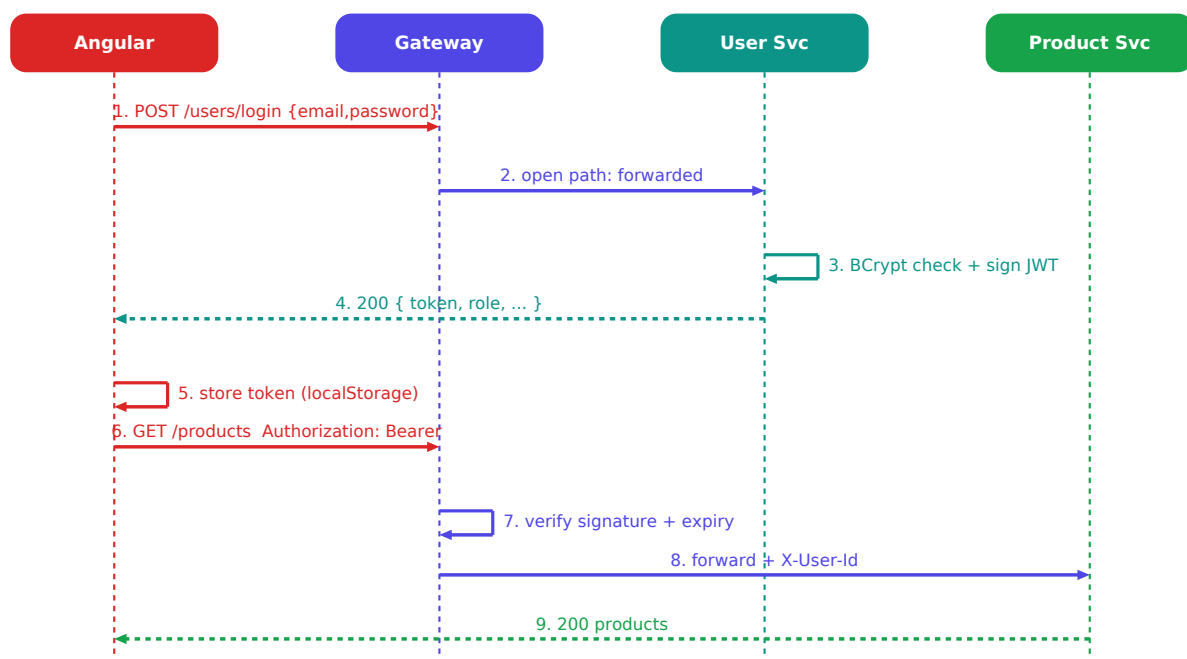


Figure 7.2 - Login once, then every request carries the token. The server keeps NO session (stateless).

Claim	Example	Used for
sub	afrudeen@example.com	Who the token belongs to
userId	3	Gateway forwards it as the X-User-Id header
role	USER or ADMIN	Authorisation (admin-only product changes)
iat / exp	issued / expiry time	Token stops working after expiry (here 2 hours)

LEARN Authentication vs authorisation

Authentication = who are you? (login, token valid). **Authorisation** = what may you do? (role ADMIN may delete products). 401 means *not authenticated*; 403 means *authenticated but not allowed*.

7.2 Configuration

user-service/src/main/resources/application.yml

YAML

```
spring:
  application:
    name: user-service
  datasource:
    url: jdbc:mysql://localhost:3306/user_db
    username: order_app
    password: ${DB_PASSWORD}
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true

server:
  port: 8081

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/

jwt:
  secret: ${JWT_SECRET}
  expiration-ms: 7200000 # 2 hours

app:
  admin:
    email: admin@example.com
    password: Admin@12345 # seed admin for learning only
```

Copy the common package from Chapter 4 (change `package` to `com.af rudeen.user.common`) and add one more exception for wrong credentials:

common/UnauthorizedException.java (also create BusinessException from Ch 4)

JAVA

```
public class UnauthorizedException extends BaseException {
    public UnauthorizedException(String message) { super(message); }

    @Override
    public HttpStatus getStatus() { return HttpStatus.UNAUTHORIZED; } // 401
}
```

7.3 Entity, role and repository

entity/Role.java

JAVA

```
package com.af rudeen.user.entity;

public enum Role { USER, ADMIN }
```

entity/User.java

JAVA

```
package com.af rudeen.user.entity;

import com.af rudeen.user.common.BaseEntity;
import jakarta.persistence.*;

@Entity
@Table(name = "users", uniqueConstraints = @UniqueConstraint(columnNames = "email"))
public class User extends BaseEntity {

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, length = 150)
    private String email;

    @Column(nullable = false)
    private String password; // BCrypt hash, never plain text

    @Enumerated(EnumType.STRING) // store "ADMIN", not 1
    @Column(nullable = false, length = 20)
    private Role role;

    protected User() { }

    public User(String name, String email, String password, Role role) {
        this.name = name;
        this.email = email;
        this.password = password;
        this.role = role;
    }

    @Override
    public String getDisplayName() { return email; }

    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getPassword() { return password; }
    public Role getRole() { return role; }
}
```

repository/UserRepository.java

JAVA

```
package com.af rudeen.user.repository;

import com.af rudeen.user.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
    boolean existsByEmail(String email);
}
```

WARNING @Enumerated(EnumType.STRING), always

The default **ORDINAL** stores 0, 1, 2. If you later insert a new enum value in the middle, every old row silently changes meaning. Use **STRING**.

7.4 DTOs

dto/ (three separate files)

JAVA

```

public record RegisterRequest(
    @NotBlank String name,
    @NotBlank @Email String email,
    @NotBlank @Size(min = 8, message = "Password must be at least 8 characters") String password) {
    }

public record LoginRequest(
    @NotBlank @Email String email,
    @NotBlank String password) { }

public record AuthResponse(String token, Long userId, String name, String email, String role) { }

```

7.5 Security configuration and JwtService

The Security starter protects every endpoint by default. Because the **Gateway** authenticates requests, User Service only needs the **PasswordEncoder** bean and a config that does not block anything. Keep this service unreachable from the internet except through the Gateway.

config/SecurityConfig.java

JAVA

```

package com.afrudeen.user.config;

import org.springframework.context.annotation.*;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable()) // stateless API, no cookies
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(a -> a.anyRequest().permitAll());
        return http.build();
    }
}

```

security/JwtService.java

JAVA

```

package com.afrudeen.user.security;

import com.afrudeen.user.entity.User;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;

@Service
public class JwtService {

    private final SecretKey key;

```

```

private final long expirationMs;

public JwtService(@Value("${jwt.secret}") String secret,
                  @Value("${jwt.expiration-ms}") long expirationMs) {
    this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
    this.expirationMs = expirationMs;
}

public String generate(User user) {
    Date now = new Date();
    return Jwts.builder()
        .subject(user.getEmail())
        .claim("userId", user.getId())
        .claim("name", user.getName())
        .claim("role", user.getRole().name())
        .issuedAt(now)
        .expiration(new Date(now.getTime() + expirationMs))
        .signWith(key) // algorithm chosen from key length
        .compact();
}
}

```

WARNING WeakKeyException?

HS256 needs a secret of at least 32 bytes. A short `JWT_SECRET` throws `WeakKeyException` at the first login. Use the 48-character secret from Chapter 3.

7.6 AuthService and controller

service/AuthService.java

JAVA

```

package com.afrudeen.user.service;

import com.afrudeen.user.common.*;
import com.afrudeen.user.dto.*;
import com.afrudeen.user.entity.*;
import com.afrudeen.user.repository.UserRepository;
import com.afrudeen.user.security.JwtService;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class AuthService {

    private final UserRepository users;
    private final PasswordEncoder encoder;
    private final JwtService jwt;

    public AuthService(UserRepository users, PasswordEncoder encoder, JwtService jwt) {
        this.users = users;
        this.encoder = encoder;
        this.jwt = jwt;
    }

    @Transactional
    public AuthResponse register(RegisterRequest r) {
        if (users.existsByEmail(r.email())) {
            throw new BusinessException("Email already registered");
        }
        User user = users.save(new User(r.name(), r.email(),
            encoder.encode(r.password()), Role.USER)); // default role
        return toResponse(user);
    }

    @Transactional(readOnly = true)
    public AuthResponse login(LoginRequest r) {
        User user = users.findByEmail(r.email())
    }

```

```

        .orElseThrow(() -> new UnauthorizedException("Invalid email or password"));
    if (!encoder.matches(r.password(), user.getPassword())) {
        throw new UnauthorizedException("Invalid email or password"); // same message!
    }
    return toResponse(user);
}

private AuthResponse toResponse(User u) {
    return new AuthResponse(jwt.generate(u), u.getId(), u.getName(), u.getEmail(),
        u.getRole().name());
}
}

```

controller/AuthController.java

JAVA

```

package com.af rudeen.user.controller;

import com.af rudeen.user.dto.*;
import com.af rudeen.user.service.AuthService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/users")
@Tag(name = "Auth", description = "Register and login")
public class AuthController {

    private final AuthService service;

    public AuthController(AuthService service) { this.service = service; }

    @PostMapping("/register")
    @ResponseStatus(HttpStatus.CREATED)
    @Operation(summary = "Register a new customer (role USER)")
    public AuthResponse register(@Valid @RequestBody RegisterRequest request) {
        return service.register(request);
    }

    @PostMapping("/login")
    @Operation(summary = "Login and receive a JWT")
    public AuthResponse login(@Valid @RequestBody LoginRequest request) {
        return service.login(request);
    }
}

```

Seed an admin account

config/AdminSeeder.java

JAVA

```

package com.af rudeen.user.config;

import com.af rudeen.user.entity.*;
import com.af rudeen.user.repository.UserRepository;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.*;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class AdminSeeder {

    @Bean
    CommandLineRunner seedAdmin(UserRepository users, PasswordEncoder encoder,
        @Value("${app.admin.email}") String email,
        @Value("${app.admin.password}") String password) {

        return args -> {

```

```

        if (!users.existsByEmail(email)) {
            users.save(new User("Admin", email, encoder.encode(password), Role.ADMIN));
        }
    };
}
}

```

7.7 Gateway security filter (verifies the JWT)

Add this class to the **api-gateway** project (add the JWT dependencies from Chapter 3 first). It runs on **every** request, lets login/register/Swagger through, rejects bad tokens, enforces *admin-only product changes*, and forwards trusted identity headers.

api-gateway/.../JwtAuthFilter.java

JAVA

```

package com.afrudeen.gateway;

import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.cloud.gateway.filter.*;
import org.springframework.core.Ordered;
import org.springframework.http.*;
import org.springframework.server.reactive.ServerHttpRequest;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;
import reactor.core.publisher.Mono;
import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;

@Component
public class JwtAuthFilter implements GlobalFilter, Ordered {

    private final SecretKey key;

    public JwtAuthFilter(@Value("${jwt.secret}") String secret) {
        this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
    }

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        ServerHttpRequest req = exchange.getRequest();
        String path = req.getURI().getPath();
        HttpMethod method = req.getMethod();

        // 1) never trust identity headers sent by a client
        ServerHttpRequest clean = req.mutate()
            .headers(h -> { h.remove("X-User-Id"); h.remove("X-User-Role"); }).build();
        ServerWebExchange cleanExchange = exchange.mutate().request(clean).build();

        // 2) open endpoints (CORS preflight, login, register, docs, health)
        if (HttpMethod.OPTIONS.equals(method) || isOpen(path)) {
            return chain.filter(cleanExchange);
        }

        // 3) require "Authorization: Bearer <token>"
        String auth = req.getHeaders().getFirst(HttpHeaders.AUTHORIZATION);
        if (auth == null || !auth.startsWith("Bearer ")) {
            return reject(exchange, HttpStatus.UNAUTHORIZED);
        }
        try {
            Claims claims = Jwts.parser().verifyWith(key).build()
                .parseSignedClaims(auth.substring(7)).getPayload(); // checks signature + exp
            String role = claims.get("role", String.class);
            Number userId = claims.get("userId", Number.class);

            // 4) role rule: only ADMIN may change products
            if (path.startsWith("/products") && !HttpMethod.GET.equals(method))

```

```

        && !"ADMIN".equals(role)) {
            return reject(exchange, HttpStatus.FORBIDDEN);
        }
        ServerHttpRequest withUser = clean.mutate()
            .header("X-User-Id", String.valueOf(userId.longValue()))
            .header("X-User-Role", role).build();
        return chain.filter(cleanExchange.mutate().request(withUser).build());
    } catch (JwtException | IllegalArgumentException e) {
        return reject(exchange, HttpStatus.UNAUTHORIZED); // bad or expired token
    }
}

private boolean isOpen(String path) {
    return path.equals("/users/login") || path.equals("/users/register")
        || path.startsWith("/swagger-ui") || path.contains("/v3/api-docs")
        || path.startsWith("/webjars") || path.startsWith("/actuator/health");
}

private Mono<Void> reject(ServerWebExchange exchange, HttpStatus status) {
    exchange.getResponse().setStatusCode(status);
    return exchange.getResponse().setComplete();
}

@Override
public int getOrder() { return -1; } // run before routing filters
}

```

LEARN Defence in depth

This design validates tokens in **one** place. A beginner-friendly start, but not the whole story:

- Services must **not** be reachable except through the Gateway (private network / firewall).
- Production systems also validate the token inside each service (Spring Security resource server) so a bypassed Gateway is not fatal.
- Stretch goal: add `spring-boot-starter-oauth2-resource-server` or the same JJWT check to Order Service.

7.8 Test the security flow

#	Request (through Gateway :8080)	Expected
1	POST <code>/users/register</code> with name, email, password	201 + token
2	POST <code>/users/login</code> with a wrong password	401 Invalid email or password
3	POST <code>/users/login</code> with correct credentials	200 { token, role: USER ... } - copy the token
4	GET <code>/products</code> without a token	401
5	GET <code>/products</code> with <code>Authorization: Bearer <token></code>	200
6	POST <code>/products</code> with the USER token	403 Forbidden
7	Login as <code>admin@example.com</code> / <code>Admin@12345</code> , POST <code>/products</code> with the admin token	201 Created

Paste the token into jwt.io (use only fake data) to see the three parts and the claims. In Swagger UI, click **Authorize** and paste the token without the word 'Bearer'.

CHECKPOINT Checkpoint

All seven rows behave exactly as shown. Open `user_db.users` in Workbench: the password column holds a `$2a$10$... hash`, not the password.

INTERVIEW JWT and password questions

- **Why BCrypt?** It is a slow, salted, one-way hash. The same password produces different hashes, which defeats rainbow tables and makes brute force expensive.
- **Is a JWT encrypted?** No - signed. Anyone can decode the payload, so never put secrets in it.
- **Why stateless?** No server session, so any instance can verify a token - scales horizontally.
- **How do you log out with JWT?** Delete the token on the client; for server-side revocation use short expiry + refresh tokens or a deny-list.
- **401 vs 403?** Not authenticated vs authenticated but not permitted.
- **Why the same error message for unknown email and wrong password?** To avoid revealing which emails are registered.

08 Order Service

Entities, OpenFeign, server-side totals and ownership rules

Now the project starts behaving like a real microservice system. Order Service owns `order_db` and **calls** Product Service over HTTP to price an order.

8.1 Configuration

`order-service/src/main/resources/application.yml`

YAML

```
spring:
  application:
    name: order-service
  datasource:
    url: jdbc:mysql://localhost:3306/order_db
    username: order_app
    password: ${DB_PASSWORD}
  jpa:
    hibernate:
      ddl-auto: update
      show-sql: true
  cloud:
    openfeign:
      client:
        config:
          default:
            connect-timeout: 2000      # ms
            read-timeout: 3000

server:
  port: 8083

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
```

Copy the `common` package (Chapter 4) into `com.afrudeen.order.common`, and also create `BusinessException`, `ServiceUnavailableException` and:

`common/ForbiddenException.java`

JAVA

```
public class ForbiddenException extends BaseException {
    public ForbiddenException(String message) { super(message); }

    @Override
    public HttpStatus getStatus() { return HttpStatus.FORBIDDEN; }    // 403
}
```

8.2 Entities

WARNING Do not name the table 'order'

`ORDER` is a reserved word in SQL (`ORDER BY`). `@Table(name = "orders")` avoids a confusing syntax error.

entity/OrderStatus.java

JAVA

```
package com.af rudeen.order.entity;

public enum OrderStatus { CREATED, CONFIRMED, CANCELLED }
```

entity/Order.java

JAVA

```
package com.af rudeen.order.entity;

import com.af rudeen.order.common.BaseEntity;
import jakarta.persistence.*;
import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "orders")
public class Order extends BaseEntity {

    @Column(nullable = false)
    private Long userId; // plain id, NOT a join to another service's table

    @Column(nullable = false, precision = 12, scale = 2)
    private BigDecimal totalAmount = BigDecimal.ZERO;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private OrderStatus status = OrderStatus.CREATED;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<OrderItem> items = new ArrayList<>();

    protected Order() { }

    public Order(Long userId) { this.userId = userId; }

    public void addItem(OrderItem item) { // keeps both sides of the relation in sync
        items.add(item);
        item.setOrder(this);
    }

    @Override
    public String getDisplayName() { return "Order #" + getId(); }

    public Long getUserId() { return userId; }
    public BigDecimal getTotalAmount() { return totalAmount; }
    public void setTotalAmount(BigDecimal totalAmount) { this.totalAmount = totalAmount; }
    public OrderStatus getStatus() { return status; }
    public List<OrderItem> getItems() { return items; }
}
```

entity/OrderItem.java

JAVA

```
package com.af rudeen.order.entity;

import com.af rudeen.order.common.BaseEntity;
import jakarta.persistence.*;
import java.math.BigDecimal;

@Entity
@Table(name = "order_items")
public class OrderItem extends BaseEntity {

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "order_id")
    private Order order;

    @Column(nullable = false) private Long productId;
```

```

@Column(nullable = false) private String productName;           // snapshot
@Column(nullable = false) private Integer quantity;
@Column(nullable = false, precision = 10, scale = 2)
private BigDecimal unitPrice;                                   // snapshot

protected OrderItem() { }

public OrderItem(Long productId, String productName, Integer quantity, BigDecimal unitPrice) {
    this.productId = productId;
    this.productName = productName;
    this.quantity = quantity;
    this.unitPrice = unitPrice;
}

@Override
public String getDisplayName() { return productName + " x" + quantity; }

void setOrder(Order order) { this.order = order; }
public Long getProductId() { return productId; }
public String getProductName() { return productName; }
public Integer getQuantity() { return quantity; }
public BigDecimal getUnitPrice() { return unitPrice; }
}

```

LEARN Why store productName and unitPrice again?

If the product price changes next month, last month's order must still show what the customer actually paid. The order item keeps a **snapshot**. This duplication is intentional - it is the cost of owning your own data.

8.3 Repository (with a fetch join)

repository/OrderRepository.java

JAVA

```

package com.af rudeen.order.repository;

import com.af rudeen.order.entity.Order;
import org.springframework.data.jpa.repository.*;
import org.springframework.data.repository.query.Param;
import java.util.List;

public interface OrderRepository extends JpaRepository<Order, Long> {

    // JOIN FETCH loads orders AND items in ONE query (avoids the N+1 problem)
    @Query("""
        select distinct o from Order o
        left join fetch o.items
        where o.userId = :userId
        order by o.createdAt desc
        """)
    List<Order> findByUserWithItems(@Param("userId") Long userId);
}

```

INTERVIEW What is the N+1 problem?

"If I load 20 orders and then touch `order.getItems()` for each, Hibernate runs 1 query for orders plus 20 queries for items - 21 in total. I fix it with `JOIN FETCH`, an `@EntityGraph`, or batch fetching."

8.4 DTOs

dto/ (separate files)

JAVA

```
public record OrderItemRequest(
    @NotNull Long productId,
    @NotNull @Min(1) Integer quantity) { }

public record CreateOrderRequest(
    @NotEmpty @Valid List<OrderItemRequest> items) { }

// What Product Service returns (our OWN copy of the contract, not its entity)
public record ProductResponse(Long id, String name, BigDecimal price, Integer stock) { }
```

dto/OrderResponse.java (+ OrderItemResponse.java)

JAVA

```
public record OrderItemResponse(Long productId, String productName,
    Integer quantity, BigDecimal unitPrice, BigDecimal lineTotal) { }

public record OrderResponse(Long id, Long userId, BigDecimal totalAmount, String status,
    LocalDateTime createdAt, List<OrderItemResponse> items) {

    public static OrderResponse from(Order o) {
        List<OrderItemResponse> items = o.getItems().stream()
            .map(i -> new OrderItemResponse(i.getProductId(), i.getProductName(),
                i.getQuantity(), i.getUnitPrice(),
                i.getUnitPrice().multiply(BigDecimal.valueOf(i.getQuantity()))))
            .toList();
        return new OrderResponse(o.getId(), o.getUserId(), o.getTotalAmount(),
            o.getStatus().name(), o.getCreatedAt(), items);
    }
}
```

8.5 OpenFeign: calling Product Service

A Feign client is just an **interface**. Spring generates the HTTP code. `name = "product-service"` is resolved through Eureka - no host or port written anywhere.

client/ProductClient.java

JAVA

```
package com.afrudeen.order.client;

import com.afrudeen.order.dto.ProductResponse;
import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.*;

@FeignClient(name = "product-service")
public interface ProductClient {

    @GetMapping("/products/{id}")
    ProductResponse findById(@PathVariable("id") Long id);
}
```

OrderServiceApplication.java

JAVA

```
@SpringBootApplication
@EnableFeignClients // scan for @FeignClient interfaces
public class OrderServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}
```

Wrap the client in a small component. In Chapter 10 this is where the circuit breaker goes:

client/ProductGateway.java

JAVA

```
package com.afrudeen.order.client;

import com.afrudeen.order.common.ResourceNotFoundException;
import com.afrudeen.order.dto.ProductResponse;
import feign.FeignException;
import org.springframework.stereotype.Component;

@Component
public class ProductGateway {

    private final ProductClient client;

    public ProductGateway(ProductClient client) { this.client = client; }

    public ProductResponse getProduct(Long id) {
        try {
            return client.findById(id);
        } catch (FeignException.NotFound e) {
            // Product Service answered 404
            throw new ResourceNotFoundException("Product not found: " + id);
        }
    }
}
```

8.6 Service: the order algorithm

service/OrderService.java

JAVA

```
package com.afrudeen.order.service;

import com.afrudeen.order.client.ProductGateway;
import com.afrudeen.order.common.*;
import com.afrudeen.order.dto.*;
import com.afrudeen.order.entity.*;
import com.afrudeen.order.repository.OrderRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.math.BigDecimal;
import java.util.List;

@Service
public class OrderService {

    private final OrderRepository repository;
    private final ProductGateway products;

    public OrderService(OrderRepository repository, ProductGateway products) {
        this.repository = repository;
        this.products = products;
    }

    @Transactional
    public OrderResponse create(Long userId, CreateOrderRequest request) {
        Order order = new Order(userId);
        BigDecimal total = BigDecimal.ZERO;

        for (OrderItemRequest line : request.items()) {
            ProductResponse p = products.getProduct(line.productId()); // sync call
            if (p.stock() < line.quantity()) {
                throw new BusinessException("Insufficient stock for " + p.name());
            }
            order.addItem(new OrderItem(p.id(), p.name(), line.quantity(), p.price()));
            total = total.add(p.price().multiply(BigDecimal.valueOf(line.quantity())));
        }
        order.setTotalAmount(total); // total computed on the SERVER
        Order saved = repository.save(order);
        return OrderResponse.from(saved);
    }
}
```

```

    }

    @Transactional(readOnly = true)
    public OrderResponse findById(Long id, Long userId, boolean admin) {
        Order o = repository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Order not found: " + id));
        if (!admin && !o.getUserId().equals(userId)) {
            throw new ForbiddenException("This order belongs to another user");
        }
        return OrderResponse.from(o);
    }

    @Transactional(readOnly = true)
    public List<OrderResponse> findMine(Long userId) {
        return repository.findByUserWithItems(userId).stream().map(OrderResponse::from).toList();
    }

    @Transactional(readOnly = true)
    public List<OrderResponse> findAll() {
        return repository.findAll().stream().map(OrderResponse::from).toList();
    }
}

```

8.7 Controller

controller/OrderController.java

JAVA

```

package com.af rudeen.order.controller;

import com.af rudeen.order.common.ForbiddenException;
import com.af rudeen.order.dto.*;
import com.af rudeen.order.service.OrderService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/orders")
@Tag(name = "Orders")
public class OrderController {

    private final OrderService service;

    public OrderController(OrderService service) { this.service = service; }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    @Operation(summary = "Place an order (user id comes from the JWT via the Gateway)")
    public OrderResponse create(@RequestHeader("X-User-Id") Long userId,
                               @Valid @RequestBody CreateOrderRequest request) {
        return service.create(userId, request);
    }

    @GetMapping("/my")
    @Operation(summary = "My order history")
    public List<OrderResponse> mine(@RequestHeader("X-User-Id") Long userId) {
        return service.findMine(userId);
    }

    @GetMapping("/{id}")
    @Operation(summary = "One order (owner or admin)")
    public OrderResponse one(@PathVariable Long id,
                            @RequestHeader("X-User-Id") Long userId,
                            @RequestHeader("X-User-Role") String role) {
        return service.findById(id, userId, "ADMIN".equals(role));
    }
}

```

```

@GetMapping
@Operation(summary = "All orders (admin only)")
public List<OrderResponse> all(@RequestHeader("X-User-Role") String role) {
    if (!"ADMIN".equals(role)) throw new ForbiddenException("Admin only");
    return service.findAll();
}

```

NOTE Why /orders/my instead of /orders/user/{userId}?

The original plan had `GET /orders/user/{userId}`. With JWT we already **know** the user, so a client-supplied id would only let a user ask for someone else's orders. The same reason applies to taking `userId` from the header instead of the request body.

8.8 Test through the Gateway

1. Start: Eureka, Product, User, Order, Gateway. Log in and copy a token (Chapter 7).
2. `POST http://localhost:8080/orders` with `{"items":[{"productId":1,"quantity":2}]}` and the Bearer token -> **201** with `totalAmount`.
3. Use `productId: 999` -> **404** Product not found. Use `quantity: 0` -> **400** with `fieldErrors`.
4. Use a quantity larger than the stock -> **400** Insufficient stock.
5. `GET /orders/my` shows your history. Check `order_db.orders` and `order_db.order_items` in Workbench.

CHECKPOINT Checkpoint

Total is calculated by the backend, an order for a missing product is rejected, and `order_items` contains the snapshot name and price.

WARNING Stock is only checked, not reduced (on purpose)

This MVP checks stock but does not decrement it. Real systems need **reservation + compensation** (Saga pattern), because a normal DB transaction cannot span two services. Stretch goal: add `PUT /products/{id}/stock/decrease` and call it after saving the order.

Say this in interviews - it shows you understand the limits of the design.

INTERVIEW Order Service questions

- **Feign vs RestTemplate/WebClient?** Feign is declarative: an interface + annotations. Less code, integrates with Eureka load balancing and circuit breakers.
- **Why not share the Product entity between services?** It couples services to one schema and one deploy. We share a **contract** (DTO), not tables.
- **LAZY vs EAGER?** LAZY loads related data only when accessed (default for collections). EAGER loads immediately and can cause huge queries.
- **What is @Transactional doing here?** The order and its items are saved atomically: either all rows are saved or none.
- **How do you handle a distributed transaction?** Avoid 2PC; use Saga (choreography with events or orchestration) with compensating actions.

09 Kafka and Notification Service

Event-driven communication, template method and consumer groups

9.1 Kafka in 5 minutes

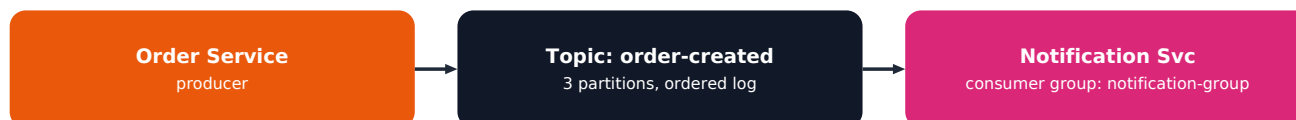


Figure 9.1 - The producer appends events; consumers read at their own pace. Kafka stores them even if a consumer is down.

Term	Meaning	In this project
Broker	A Kafka server	One container on <code>localhost:9092</code>
Topic	A named stream of events	<code>order-created</code>
Partition	Ordered slice of a topic; unit of parallelism	3 partitions; messages with the same key stay in order
Offset	Position of a message in a partition	Consumers remember where they stopped
Producer / Consumer	Writes / reads messages	Order Service / Notification Service
Consumer group	Consumers that share the work; each message goes to one member	<code>notification-group</code>

LEARN REST vs Kafka - the interview answer

Use **REST (sync)** when the caller needs the answer **now** (Order needs the product price). Use **Kafka (async)** when the caller does not need to wait and other services may react later (notifications). Kafka gives decoupling and buffering at the cost of eventual consistency and harder debugging.

9.2 Run Kafka with Docker Compose

infrastructure/docker-compose.yml

YAML

```

services:
  kafka:
    image: apache/kafka:3.9.0
    container_name: kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@localhost:9093
  
```

```
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
```

PowerShell

BASH

```
cd E:\order-management-system\infrastructure
docker compose up -d
docker ps
docker exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
```

NOTE If this image or tag is unavailable

Use the `docker-compose` example from the official Apache Kafka Docker documentation for the image version you pull. The important parts are: one node acting as broker + controller (KRaft mode, no ZooKeeper) and an advertised listener of `localhost:9092`.

CHECKPOINT Checkpoint

`docker ps` shows the kafka container **Up** and the topic list command runs without errors (empty list is fine).

9.3 Order Service: publish the event

order-service application.yml (add under the existing spring: block)

YAML

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
    properties:
      spring.json.add.type.headers: false
```

event/OrderCreatedEvent.java

JAVA

```
package com.afrudeen.order.event;

import java.math.BigDecimal;
import java.time.LocalDateTime;

public record OrderCreatedEvent(Long orderId, Long userId,
                               BigDecimal totalAmount, LocalDateTime createdAt) { }
```

event/OrderEventPublisher.java

JAVA

```
package com.afrudeen.order.event;

import org.apache.kafka.clients.admin.NewTopic;
import org.slf4j.*;
import org.springframework.context.annotation.*;
import org.springframework.kafka.config.TopicBuilder;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Component;

@Configuration
class KafkaTopicConfig {
    @Bean
    NewTopic orderCreatedTopic() {
        return TopicBuilder.name(OrderEventPublisher.TOPIC).partitions(3).replicas(1).build();
    }
}
```

```

    }
}

@Component
public class OrderEventPublisher {

    public static final String TOPIC = "order-created";
    private static final Logger log = LoggerFactory.getLogger(OrderEventPublisher.class);
    private final KafkaTemplate<String, OrderCreatedEvent> kafka;

    public OrderEventPublisher(KafkaTemplate<String, OrderCreatedEvent> kafka) {
        this.kafka = kafka;
    }

    public void publish(OrderCreatedEvent event) {
        kafka.send(TOPIC, String.valueOf(event.orderId()), event) // key = orderId
            .whenComplete((result, ex) -> {
                if (ex != null) log.error("Kafka publish failed for order {}", event.orderId(), ex);
                else log.info("Published order {} to partition {}", event.orderId(),
                    result.getRecordMetadata().partition());
            });
    }
}

```

Update `OrderService`: add the publisher to the constructor and publish after saving.

OrderService.java (changes)

JAVA

```

private final OrderEventPublisher events;

public OrderService(OrderRepository repository, ProductGateway products,
    OrderEventPublisher events) {
    this.repository = repository;
    this.products = products;
    this.events = events;
}

// ... inside create(), after: Order saved = repository.save(order);
events.publish(new OrderCreatedEvent(saved.getId(), saved.getUserId(),
    saved.getTotalAmount(), saved.getCreatedAt()));
return OrderResponse.from(saved);

```

WARNING Known weakness - say it before they ask

The event is published **inside** the transaction, before commit. If the commit fails after publishing, a notification exists for an order that does not. If the app crashes between commit and publish, the event is lost.

Production fix: the **Transactional Outbox** pattern (save the event in `order_db` in the same transaction, and a relay publishes it) plus **idempotent consumers**.

9.4 Notification Service

Copy `common` (package `com.af rudeen.notification.common`). Config:

notification-service/src/main/resources/application.yml

YAML

```

spring:
  application:
    name: notification-service
  datasource:
    url: jdbc:mysql://localhost:3306/notification_db
    username: order_app
    password: ${DB_PASSWORD}
  jpa:
    hibernate:
      ddl-auto: update

```

```

kafka:
  bootstrap-servers: localhost:9092
  consumer:
    group-id: notification-group
    auto-offset-reset: earliest
    key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
    value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
    properties:
      spring.json.value.default.type: com.afrudeen.notification.dto.OrderCreatedEvent
      spring.json.use.type.headers: false
      spring.json.trusted.packages: "*"

server:
  port: 8084

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/

```

`spring.json.value.default.type` tells the consumer which class to build from the JSON, because Order Service's class lives in a different package. Create the same record in `com.afrudeen.notification.dto`.

dto/OrderCreatedEvent.java

JAVA

```

package com.afrudeen.notification.dto;

public record OrderCreatedEvent(Long orderId, Long userId,
                               java.math.BigDecimal totalAmount, java.time.LocalDateTime createdAt) { }

```

entity/Notification.java

JAVA

```

package com.afrudeen.notification.entity;

import com.afrudeen.notification.common.BaseEntity;
import jakarta.persistence.*;

@Entity
@Table(name = "notifications")
public class Notification extends BaseEntity {

    @Column(nullable = false) private Long userId;
    @Column(nullable = false) private Long orderId;
    @Column(nullable = false, length = 500) private String message;

    protected Notification() { }

    public Notification(Long userId, Long orderId, String message) {
        this.userId = userId;
        this.orderId = orderId;
        this.message = message;
    }

    @Override
    public String getDisplayName() { return "Notification for order #" + orderId; }

    public Long getUserId() { return userId; }
    public Long getOrderId() { return orderId; }
    public String getMessage() { return message; }
}

```

repository/NotificationRepository.java

JAVA

```
public interface NotificationRepository extends JpaRepository<Notification, Long> {
    List<Notification> findByUserIdOrderByCreatedAtDesc(Long userId);
}
```

9.5 Abstract class #3: Template Method

The **algorithm** is always the same: validate - build message - save - after-save hook. Only the message differs per event type. The abstract class owns the algorithm (`handle` is `final`) and subclasses fill the gaps. Tomorrow you could add `OrderCancelledNotificationHandler` without touching the algorithm.

handler/NotificationHandler.java

JAVA

```
package com.af rudeen.notification.handler;

import com.af rudeen.notification.entity.Notification;
import com.af rudeen.notification.repository.NotificationRepository;
import org.slf4j.*;

public abstract class NotificationHandler<E> {

    private static final Logger log = LoggerFactory.getLogger(NotificationHandler.class);
    private final NotificationRepository repository;

    protected NotificationHandler(NotificationRepository repository) {
        this.repository = repository;
    }

    /** TEMPLATE METHOD: fixed steps, cannot be overridden. */
    public final void handle(E event) {
        validate(event);
        Notification n = new Notification(userIdOf(event), orderIdOf(event), buildMessage(event));
        afterSave(repository.save(n));
    }

    // steps that subclasses MUST provide
    protected abstract Long userIdOf(E event);
    protected abstract Long orderIdOf(E event);
    protected abstract String buildMessage(E event);

    // steps with a default that subclasses MAY override (hooks)
    protected void validate(E event) {
        if (event == null) throw new IllegalArgumentException("event is null");
    }

    protected void afterSave(Notification saved) {
        log.info("Saved: {}", saved.getDisplayName());
    }
}
```

handler/OrderCreatedNotificationHandler.java

JAVA

```
package com.af rudeen.notification.handler;

import com.af rudeen.notification.dto.OrderCreatedEvent;
import com.af rudeen.notification.repository.NotificationRepository;
import org.springframework.stereotype.Component;

@Component
public class OrderCreatedNotificationHandler extends NotificationHandler<OrderCreatedEvent> {

    public OrderCreatedNotificationHandler(NotificationRepository repository) {
        super(repository);
    }
}
```

```

@Override protected Long getUserId(OrderCreatedEvent e) { return e.userId(); }
@Override protected Long orderIdOf(OrderCreatedEvent e) { return e.orderId(); }

@Override
protected String buildMessage(OrderCreatedEvent e) {
    return "Order #%d created successfully. Total: INR %s".formatted(e.orderId(), e.totalAmount());
}
}

```

listener/OrderEventListener.java

JAVA

```

package com.afrudeen.notification.listener;

import com.afrudeen.notification.dto.OrderCreatedEvent;
import com.afrudeen.notification.handler.OrderCreatedNotificationHandler;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;

@Component
public class OrderEventListener {

    private final OrderCreatedNotificationHandler handler;

    public OrderEventListener(OrderCreatedNotificationHandler handler) { this.handler = handler; }

    @KafkaListener(topics = "order-created", groupId = "notification-group")
    public void onOrderCreated(OrderCreatedEvent event) {
        handler.handle(event);
    }
}

```

controller/NotificationController.java (also add OpenApiConfig from Ch 4)

JAVA

```

@RestController
@RequestMapping("/notifications")
@Tag(name = "Notifications")
public class NotificationController {

    private final NotificationRepository repository;

    public NotificationController(NotificationRepository repository) { this.repository = repository; }

    @GetMapping("/my")
    @Operation(summary = "My notifications, newest first")
    public List<Notification> mine(@RequestHeader("X-User-Id") Long userId) {
        return repository.findByIdOrderByCreatedAtDesc(userId);
    }
}

```

9.6 Test the event flow

1. Start Kafka, Eureka, Product, User, Order, Notification, Gateway.
2. Create an order through the Gateway (Chapter 8).
3. Order Service log: **Published order 1 to partition N**. Notification Service log: **Saved: Notification for order #1**.
4. Workbench: **SELECT * FROM notification_db.notifications**; shows the message.
5. Watch the raw events:

PowerShell - press Ctrl+C to stop

BASH

```
docker exec kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic  
order-created --from-beginning
```

Experiment: stop Notification Service, create two orders, then start it again. Both notifications appear - Kafka kept the events. That one experiment explains **why** asynchronous messaging is valuable.

CHECKPOINT Checkpoint

A notification appears in the database after you create an order, and the order response did not wait for it.

INTERVIEW Kafka questions

- **What if a consumer is down?** Messages stay in the topic; the consumer resumes from its last committed offset.
- **Ordering guarantee?** Only within a partition. Using `orderId` as the key sends all events of one order to the same partition.
- **Duplicates?** Delivery is at-least-once by default, so consumers should be idempotent (e.g., a unique key on `orderId` + type).
- **Kafka vs RabbitMQ?** Kafka is a durable replayable log with high throughput; RabbitMQ is a traditional broker with flexible routing and per-message acknowledgement.
- **Template Method pattern?** A base class defines the skeleton of an algorithm; subclasses override specific steps without changing the structure.

10 Resilience4j

Timeouts, circuit breaker states and graceful failure

What happens if Product Service is down or slow while 100 users place orders? Without protection, Order Service threads pile up waiting, then Order Service itself stops responding - a **cascading failure**. A circuit breaker stops calling a failing service for a while.



Figure 10.1 - Circuit breaker states for the Order -> Product call

10.1 Configure timeouts and the breaker

Timeouts are already in the Feign config (2 s connect, 3 s read). Add the circuit breaker settings:

order-service application.yml (add)

YAML

```

resilience4j:
  circuitbreaker:
    instances:
      productService:
        sliding-window-type: COUNT_BASED
        sliding-window-size: 6           # look at the last 6 calls
        minimum-number-of-calls: 4      # need 4 calls before judging
        failure-rate-threshold: 50      # >= 50% failures -> OPEN
        wait-duration-in-open-state: 10s # stay OPEN 10 s
        permitted-number-of-calls-in-half-open-state: 2
        automatic-transition-from-open-to-half-open-enabled: true
        register-health-indicator: true
        ignore-exceptions:               # a 404 is NOT a service failure
          - com.afrudeen.order.common.ResourceNotFoundException

management:
  endpoints:
    web:
      exposure:
        include: health,info,circuitbreakers,circuitbreakerevents
  health:
    circuitbreakers:
      enabled: true
  
```

10.2 Protect the call

client/ProductGateway.java (replace the Chapter 8 version)

JAVA

```
package com.afrudeen.order.client;

import com.afrudeen.order.common.*;
import com.afrudeen.order.dto.ProductResponse;
import feign.FeignException;
import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
import org.slf4j.*;
import org.springframework.stereotype.Component;

@Component
public class ProductGateway {

    private static final Logger log = LoggerFactory.getLogger(ProductGateway.class);
    private final ProductClient client;

    public ProductGateway(ProductClient client) { this.client = client; }

    @CircuitBreaker(name = "productService", fallbackMethod = "fallback")
    public ProductResponse getProduct(Long id) {
        try {
            return client.findById(id);
        } catch (FeignException.NotFound e) {
            throw new ResourceNotFoundException("Product not found: " + id);
        }
    }

    // same parameters + the exception. Never create an order silently!
    private ProductResponse fallback(Long id, Throwable ex) {
        if (ex instanceof ResourceNotFoundException notFound) throw notFound;
        log.warn("Product service unavailable for product {}: {}", id, ex.toString());
        throw new ServiceUnavailableException(
            "Product service is unavailable right now. Please try again shortly.");
    }
}
```

LEARN Why a separate ProductGateway class?

`@CircuitBreaker` works through a **Spring proxy**. A method calling another method of the **same class** bypasses the proxy, so the annotation is silently ignored. Putting the protected call in its own bean guarantees the proxy is used. (Classic interview trap - the same applies to `@Transactional` and `@Async`.)

10.3 Failure experiment (do this exact script)

Step	Action	What you should observe
1	All services running; create an order	201 . Breaker is CLOSED
2	Stop Product Service	-
3	Create orders 4-5 times quickly	First calls wait up to the timeout then return 503 ; after the failure rate passes 50% the breaker goes OPEN
4	Create another order	Instant 503 - the breaker rejects without calling (<code>CallNotPermittedException</code>)
5	Open http://localhost:8083/actuator/circuitbreakers	State shows OPEN , then later HALF_OPEN
6	Start Product Service again; wait ~10 s; create an order	Trial calls succeed -> breaker returns to CLOSED ; 201

CHECKPOINT Checkpoint

You can narrate CLOSED -> OPEN -> HALF-OPEN -> CLOSED using **your** order flow, and you saw the 503 JSON from `ServiceUnavailableException`.

10.4 The resilience family

Pattern	What it does	Example use
Timeout / TimeLimiter	Stop waiting after N ms	Feign read-timeout 3 s
Circuit Breaker	Stop calling a failing dependency; fail fast; recover automatically	Order -> Product
Retry	Try again a few times for transient errors (use with backoff, only for idempotent calls)	GET product details
Bulkhead	Limit concurrent calls so one slow dependency cannot use all threads	Separate pool per dependency
Rate Limiter	Limit calls per time window	Public API protection at the Gateway
Fallback	Alternative response when the call fails	Return 503 (here) or cached data

INTERVIEW Resilience questions

- **Explain the three states using your project.** CLOSED: calls to Product flow. After $\geq 50\%$ of the last 6 calls fail, OPEN: Order Service rejects instantly for 10 s. Then HALF-OPEN: 2 trial calls; success closes it, failure re-opens it.
- **Why not just retry?** Retrying a dead service multiplies load. A breaker gives it time to recover.
- **Why ignore 404?** A missing product is a normal business answer, not a sign the service is unhealthy.
- **Why not create the order when Product is down?** We could not verify price or stock; creating it would break data integrity.

11 TypeScript Primer

Everything in TypeScript you need before touching Angular

TypeScript is **JavaScript + types**. The browser runs JavaScript; the Angular build compiles your TypeScript to it and **catches mistakes before you run the app**. If you know Java, most of it will feel familiar.

11.1 Java vs TypeScript at a glance

Concept	Java	TypeScript
Variable	<code>int age = 20;</code>	<code>let age: number = 20; / const name = 'Afru';</code>
Types	<code>int, long, double, String, boolean</code>	<code>number, string, boolean, null, undefined, any, unknown</code>
List	<code>List<String></code>	<code>string[]</code> or <code>Array<string></code>
Class	<code>class A { private int x; }</code>	<code>class A { private x: number; }</code>
Interface	Contract implemented by classes	Describes the shape of an object (used for JSON data)
Null safety	<code>Optional<T></code>	<code>T null, ?. (optional chaining), ?? (nullish coalescing)</code>
Lambda	<code>x -> x * 2</code>	<code>x => x * 2</code> (arrow function)
Generics	<code>List<Product></code>	<code>Observable<Product[]></code>
Access	<code>private final</code>	<code>private readonly</code>
Async	<code>CompletableFuture</code>	<code>Promise</code> / <code>async-await</code> , and RxJS <code>Observable</code> in Angular

11.2 Types, interfaces and unions

types.ts

TS

```
// primitives and inference
let title: string = 'Keyboard';
let price = 1299.5;           // inferred as number
const inStock: boolean = true;
let ids: number[] = [1, 2, 3];

// interface = shape of JSON from the backend
interface Product {
  id: number;
  name: string;
  price: number;
  stock: number;
  description?: string;      // optional property
}

const p: Product = { id: 1, name: 'Mouse', price: 599, stock: 50 };

// union and literal types
type Role = 'USER' | 'ADMIN';
let role: Role = 'ADMIN';    // role = 'GUEST' would be a compile error
let value: string | null = null;
```

```
// enum
enum OrderStatus { CREATED = 'CREATED', CONFIRMED = 'CONFIRMED', CANCELLED = 'CANCELLED' }
```

11.3 Functions, arrays and the operators you will use daily

functions.ts

TS

```
const total = (price: number, qty: number): number => price * qty;

const products: Product[] = [ /* ... */ ];

const names      = products.map(p => p.name);           // transform
const cheap      = products.filter(p => p.price < 1000); // keep some
const sum        = products.reduce((acc, p) => acc + p.price, 0); // fold to one value
const mouse      = products.find(p => p.name === 'Mouse'); // first match or undefined
const hasStock   = products.some(p => p.stock > 0);

// destructuring and spread
const { name, price } = products[0];
const updated = { ...products[0], price: 999 };           // copy + change (immutability)
const more    = [...products, newProduct];

// optional chaining and nullish coalescing
const city = user?.address?.city ?? 'Unknown';

// template literal
const label = `${name} costs ${price}`;
```

11.4 Classes and generics

classes.ts

TS

```
class Cart {
  private items: { product: Product; qty: number }[] = [];

  constructor(private readonly owner: string) {} // parameter property: declares + assigns

  add(product: Product, qty = 1): void {
    this.items.push({ product, qty });
  }

  get total(): number {
    return this.items.reduce((s, i) => s + i.product.price * i.qty, 0); // getter: cart.total
  }
}

function firstOf<T>(list: T[]): T | undefined { // generic function
  return list[0];
}
```

11.5 Promises vs Observables (key for Angular)

	Promise	Observable (RxJS)
Values	Exactly one future value	Zero or many values over time
Starts	Immediately when created	Only when you subscribe
Cancel	No	Yes (unsubscribe , switchMap)
Operators	then , catch	map , filter , tap , switchMap , debounceTime , catchError , forkJoin

	Promise	Observable (RxJS)
Angular	Rare	<code>HttpClient</code> returns Observables

async.ts

TS

```
// async/await with a Promise
async function load(): Promise<void> {
  const res = await fetch('http://localhost:8080/products');
  const data: Product[] = await res.json();
}

// Observable with HttpClient - nothing happens until subscribe()
this.http.get<Product[]>(url).subscribe({
  next: list => this.products = list,           // success
  error: err => console.error(err),             // HTTP error
  complete: () => console.log('done')
});
```

11.6 Modules, decorators and strict mode

- `export / import { X } from './x'` - every file is a module (like a Java package + public class).
- **Decorators** such as `@Component(...)` and `@Injectable(...)` attach metadata to a class - similar to Java annotations. Angular reads them.
- `tsconfig.json` has `"strict": true`: no implicit `any`, strict null checks. Leave it on; it prevents whole classes of bugs.
- `any` turns checking off. Prefer `unknown` or a real interface.

INTERVIEW TypeScript questions

- **TypeScript vs JavaScript?** TS adds static typing, interfaces, generics and better tooling; it compiles to JS and types disappear at runtime.
- **interface vs type?** Both describe shapes; interfaces can be extended/merged; `type` can also express unions and primitives.
- **any vs unknown?** `unknown` forces you to check the type before using it; `any` disables checks.
- **== vs ===?** `===` compares without type coercion - always use it.
- **let, const, var?** `var` is function-scoped and hoisted (avoid). `let/const` are block-scoped; use `const` by default.

12 Angular Application

Components, services, HttpClient, interceptor, guard, forms and RxJS

12.1 How Angular talks to Spring Boot

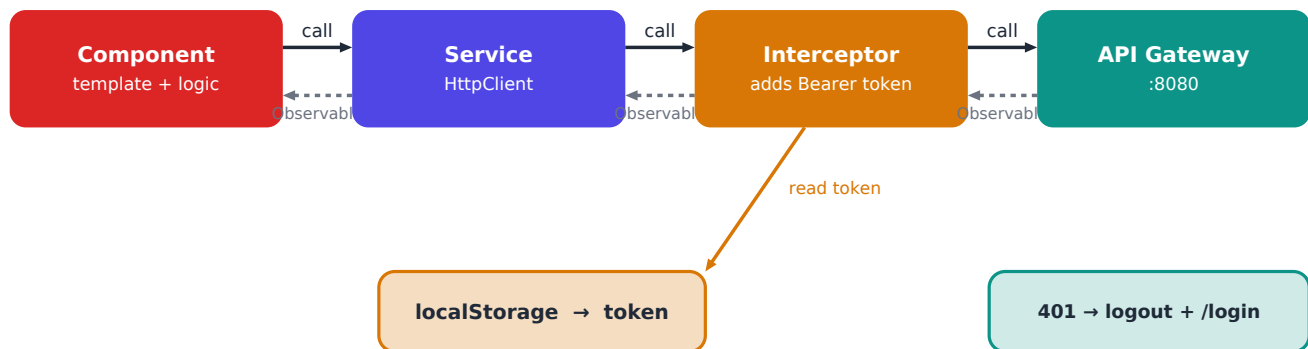


Figure 12.1 - A component never calls HTTP directly. It uses a service; an interceptor adds the JWT to every request.

Angular building block	What it is	Java / Spring analogy
Component	A piece of UI: TypeScript class + HTML template + CSS	Controller + view
Service (@Injectable)	Reusable logic, HTTP calls; shared via dependency injection	@Service
inject() / DI	Angular supplies the object you ask for	Constructor injection / @Autowired
Router	Maps URLs to components	@RequestMapping
Guard	Allows or blocks a route	Spring Security filter / @PreAuthorize
Interceptor	Runs on every HTTP request/response	Servlet filter / OncePerRequestFilter
Reactive form	Form model defined in TypeScript with validators	@Valid DTO + Bean Validation
Signal	A reactive value: <code>count()</code> / <code>count.set(1)</code>	-
Observable	Stream of async values (HTTP, input events)	Reactor Mono/Flux

12.2 Create the project

PowerShell

BASH

```

cd E:\order-management-system\frontend
ng new order-management-ui --style=css
cd order-management-ui
ng add @angular/material
ng serve
  
```

- For prompts: stylesheet **CSS**; SSR/SSG **No**; accept routing if asked.

- `ng add @angular/material` asks for a theme (pick any, e.g. *Azure/Blue*), global typography (**Yes**). Keep whatever it generates.
- Open **`http://localhost:4200`**. You should see the Angular starter page.

WARNING PowerShell says 'running scripts is disabled'?

Run `ng.cmd serve` instead, or allow scripts for your user once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

NOTE File names in new Angular versions

Recent Angular CLIs generate files without suffixes (`login.ts`, class `Login`). This guide uses explicit names such as `login.component.ts` and `LoginComponent` so each file's role is obvious. **Create files by hand in VS Code** (right-click folder > New File) - both styles work. The generated root component is `src/app/app.ts` (class `App`); if yours is `AppComponent`, keep your own class name.

12.3 Folder structure

Angular structure

TEXT

```
src/app/
|-- app.ts                root component (toolbar + router outlet)
|-- app.config.ts         providers: router, HttpClient, interceptor
|-- app.routes.ts         URL -> component map
|-- core/
|   |-- config.ts         API_URL
|   |-- models.ts         TypeScript interfaces
|   |-- auth.service.ts   login, token storage
|   |-- auth.guard.ts     authGuard + adminGuard
|   |-- auth.interceptor.ts adds Authorization header
|   |-- api.service.ts    product, order, notification calls
|   |-- http-error.ts     readable error messages
|-- features/
|   |-- login/login.component.ts
|   |-- dashboard/dashboard.component.ts
|   |-- products/product-list.component.ts
|   |-- products/product-form.component.ts
|   |-- orders/orders.component.ts
|   |-- notifications/notifications.component.ts
```

12.4 Config, models and error helper

core/config.ts

TS

```
export const API_URL = 'http://localhost:8080'; // always the GATEWAY
```

core/models.ts

TS

```
export interface AuthResponse {
  token: string; userId: number; name: string; email: string; role: 'USER' | 'ADMIN';
}
export interface Product { id: number; name: string; price: number; stock: number; }
export interface ProductRequest { name: string; price: number; stock: number; }
export interface OrderItemRequest { productId: number; quantity: number; }
export interface OrderItem {
  productId: number; productName: string; quantity: number; unitPrice: number; lineTotal: number;
}
export interface Order {
  id: number; userId: number; totalAmount: number; status: string;
  createdAt: string; items: OrderItem[];
}
```

```

}
export interface AppNotification {
  id: number; userId: number; orderId: number; message: string; createdAt: string;
}

```

core/http-error.ts

TS

```

export function errorMessage(err: any): string {
  if (err?.status === 0) return 'Cannot reach the server. Is the Gateway running?';
  const fields = err?.error?.fieldErrors;
  if (fields) return Object.values(fields).join(', ');
  return err?.error?.message ?? 'Unexpected error';
}

```

Our Spring `ApiError` returns `message` and `fieldErrors`, so the UI can show the **real** reason (for example 'Insufficient stock for Oil Filter').

12.5 AuthService (login + localStorage)

core/auth.service.ts

TS

```

import { Injectable, computed, inject, signal } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Router } from '@angular/router';
import { tap } from 'rxjs';
import { API_URL } from './config';
import { AuthResponse } from './models';

const TOKEN_KEY = 'oms_token';
const USER_KEY = 'oms_user';

@Injectable({ providedIn: 'root' })
export class AuthService {
  private http = inject(HttpClient);
  private router = inject(Router);

  private _user = signal<AuthResponse | null>(this.loadUser());
  readonly user = this._user.asReadonly();
  readonly isLoggedIn = computed(() => this._user() !== null);
  readonly isAdmin = computed(() => this._user()?.role === 'ADMIN');

  login(body: { email: string; password: string }) {
    return this.http.post<AuthResponse>(`${API_URL}/users/login`, body)
      .pipe(tap(res => this.save(res)));
  }

  register(body: { name: string; email: string; password: string }) {
    return this.http.post<AuthResponse>(`${API_URL}/users/register`, body)
      .pipe(tap(res => this.save(res)));
  }

  logout(): void {
    localStorage.removeItem(TOKEN_KEY);
    localStorage.removeItem(USER_KEY);
    this._user.set(null);
    this.router.navigate(['/login']);
  }

  get token(): string | null {
    return localStorage.getItem(TOKEN_KEY);
  }

  hasValidToken(): boolean {
    const t = this.token;
    if (!t) return false;
    try {
      const b64 = t.split('.')[1].replace(/-/g, '+').replace(/_/g, '/');
      const payload = JSON.parse(atob(b64));
    }
  }

```

```

        return payload.exp * 1000 > Date.now(); // exp is in seconds
    } catch {
        return false;
    }
}

private save(res: AuthResponse): void {
    localStorage.setItem(TOKEN_KEY, res.token);
    localStorage.setItem(USER_KEY, JSON.stringify({ ...res, token: '' }));
    this._user.set(res);
}

private loadUser(): AuthResponse | null {
    const raw = localStorage.getItem(USER_KEY);
    return raw ? (JSON.parse(raw) as AuthResponse) : null;
}
}

```

12.6 Interceptor and guards

core/auth.interceptor.ts

TS

```

import { HttpResponse, HttpErrorResponse, HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { catchError, throwError } from 'rxjs';
import { AuthService } from '../auth.service';
import { API_URL } from '../config';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
    const auth = inject(AuthService);
    const token = auth.token;

    // only attach the token to OUR API, never to third-party URLs
    const request = token && req.url.startsWith(API_URL)
        ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
        : req;

    return next(request).pipe(
        catchError((err: HttpErrorResponse) => {
            if (err.status === 401 && !req.url.includes('/users/login')) {
                auth.logout(); // expired/invalid token
            }
            return throwError(() => err);
        })
    );
};

```

core/auth.guard.ts

TS

```

import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../auth.service';

export const authGuard: CanActivateFn = () => {
    const auth = inject(AuthService);
    return auth.isValidToken() ? true : inject(Router).createUrlTree(['/login']);
};

export const adminGuard: CanActivateFn = () => {
    const auth = inject(AuthService);
    return auth.isAdmin() ? true : inject(Router).createUrlTree(['/products']);
};

```

12.7 API service (products, orders, notifications)

core/api.service.ts

TS

```
import { Injectable, inject } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
import { API_URL } from './config';
import { AppNotification, Order, OrderItemRequest, Product, ProductRequest } from './models';

@Injectable({ providedIn: 'root' })
export class ProductApi {
  private http = inject(HttpClient);
  private url = `${API_URL}/products`;

  getAll(search = '') {
    const params = search ? new HttpParams().set('search', search) : undefined;
    return this.http.get<Product[]>(this.url, { params });
  }
  get(id: number) { return this.http.get<Product>(`${this.url}/${id}`); }
  create(p: ProductRequest) { return this.http.post<Product>(this.url, p); }
  update(id: number, p: ProductRequest) { return this.http.put<Product>(`${this.url}/${id}`, p); }
  delete(id: number) { return this.http.delete<void>(`${this.url}/${id}`); }
}

@Injectable({ providedIn: 'root' })
export class OrderApi {
  private http = inject(HttpClient);
  private url = `${API_URL}/orders`;

  create(items: OrderItemRequest[]) { return this.http.post<Order>(this.url, { items }); }
  mine() { return this.http.get<Order[]>(`${this.url}/my`); }
}

@Injectable({ providedIn: 'root' })
export class NotificationApi {
  private http = inject(HttpClient);
  mine() { return this.http.get<AppNotification[]>(`${API_URL}/notifications/my`); }
}
```

LEARN providedIn: 'root'

Makes the service a **singleton** shared by the whole app - Angular's equivalent of a Spring `@Service` bean. `inject(HttpClient)` is dependency injection: you declare what you need and Angular supplies it.

12.8 App wiring: config, routes, root component

app.config.ts

TS

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { routes } from './app.routes';
import { authInterceptor } from './core/auth.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor])),
    // keep any provider that `ng add @angular/material` already added
  ],
};
```

app.routes.ts

TS

```
import { Routes } from '@angular/router';
import { authGuard, adminGuard } from '../core/auth.guard';
import { LoginComponent } from '../features/login/login.component';
import { DashboardComponent } from '../features/dashboard/dashboard.component';
import { ProductListComponent } from '../features/products/product-list.component';
import { ProductFormComponent } from '../features/products/product-form.component';
import { OrdersComponent } from '../features/orders/orders.component';
import { NotificationsComponent } from '../features/notifications/notifications.component';

export const routes: Routes = [
  { path: 'login', component: LoginComponent },
  { path: 'dashboard', component: DashboardComponent, canActivate: [authGuard] },
  { path: 'products', component: ProductListComponent, canActivate: [authGuard] },
  { path: 'products/new', component: ProductFormComponent, canActivate: [authGuard, adminGuard] },
  { path: 'products/:id/edit', component: ProductFormComponent, canActivate: [authGuard, adminGuard] },
  { path: 'orders', component: OrdersComponent, canActivate: [authGuard] },
  { path: 'notifications', component: NotificationsComponent, canActivate: [authGuard] },
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' },
  { path: '**', redirectTo: 'dashboard' },
];
```

app.ts

TS

```
import { Component, inject } from '@angular/core';
import { RouterLink, RouterLinkActive, RouterOutlet } from '@angular/router';
import { MatToolbarModule } from '@angular/material/toolbar';
import { MatButtonModule } from '@angular/material/button';
import { MatIconModule } from '@angular/material/icon';
import { AuthService } from '../core/auth.service';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink, RouterLinkActive, MatToolbarModule, MatButtonModule,
    MatIconModule],
  template: `
    @if (auth.isLoggedIn()) {
      <mat-toolbar color="primary" class="nav">
        <mat-icon>inventory_2</mat-icon>
        <span class="brand">Order Management</span>
        <a mat-button routerLink="/dashboard" routerLinkActive="active">Dashboard</a>
        <a mat-button routerLink="/products" routerLinkActive="active">Products</a>
        <a mat-button routerLink="/orders" routerLinkActive="active">Orders</a>
        <a mat-button routerLink="/notifications" routerLinkActive="active">Notifications</a>
        <span class="spacer"></span>
        <span class="who">{{ auth.user()?.name }} ({{ auth.user()?.role }})</span>
        <button mat-stroked-button (click)="auth.logout()">Logout</button>
      </mat-toolbar>
    }
    <main class="page"><router-outlet /></main>
  `,
})
export class App {
  auth = inject(AuthService);
}
```

12.9 Login and register page

features/login/login.component.ts

TS

```
import { Component, inject, signal } from '@angular/core';
import { FormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { MatCardModule } from '@angular/material/card';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatInputModule } from '@angular/material/input';
```

```

import { MatButtonModule } from '@angular/material/button';
import { AuthService } from '../../core/auth.service';
import { errorMessage } from '../../core/http-error';

@Component({
  selector: 'app-login',
  imports: [ReactiveFormsModule, MatCardModule, MatFormFieldModule, MatInputModule, MatButtonModule],
  template: `
    <mat-card class="auth-card">
      <mat-card-header>
        <mat-card-title>{{ registerMode() ? 'Create account' : 'Sign in' }}</mat-card-title>
      </mat-card-header>
      <mat-card-content>
        <form [formGroup]="form" (ngSubmit)="submit()" class="stack">
          @if (registerMode()) {
            <mat-form-field appearance="outline">
              <mat-label>Name</mat-label>
              <input matInput formControlName="name" />
              <mat-error>Name is required</mat-error>
            </mat-form-field>
          }
          <mat-form-field appearance="outline">
            <mat-label>Email</mat-label>
            <input matInput type="email" formControlName="email" />
            <mat-error>Enter a valid email</mat-error>
          </mat-form-field>
          <mat-form-field appearance="outline">
            <mat-label>Password</mat-label>
            <input matInput type="password" formControlName="password" />
            <mat-error>Minimum 8 characters</mat-error>
          </mat-form-field>
          @if (error()) { <p class="error">{{ error() }}</p> }
          <button mat-flat-button color="primary" [disabled]="loading()">
            {{ registerMode() ? 'Register' : 'Login' }}
          </button>
        </form>
      </mat-card-content>
      <mat-card-actions>
        <button mat-button (click)="toggle()">
          {{ registerMode() ? 'Have an account? Sign in' : 'New here? Register' }}
        </button>
      </mat-card-actions>
    </mat-card>
  `,
})
export class LoginComponent {
  private fb = inject(FormBuilder).nonNullable;
  private auth = inject(AuthService);
  private router = inject(Router);

  registerMode = signal(false);
  loading = signal(false);
  error = signal('');

  form = this.fb.group({
    name: ['',],
    email: ['', [Validators.required, Validators.email]],
    password: ['', [Validators.required, Validators.minLength(8)]],
  });

  toggle() {
    this.registerMode.update(v => !v);
    this.error.set('');
    const name = this.form.controls.name;
    this.registerMode() ? name.addValidators(Validators.required) : name.clearValidators();
    name.updateValueAndValidity();
  }

  submit() {
    if (this.form.invalid) { this.form.markAllAsTouched(); return; }
    this.loading.set(true);
    const { name, email, password } = this.form.getRawValue();

```

```

const call = this.registerMode()
  ? this.auth.register({ name, email, password })
  : this.auth.login({ email, password });
call.subscribe({
  next: () => this.router.navigate(['/dashboard']),
  error: err => { this.error.set(errorMessage(err)); this.loading.set(false); },
});
}
}

```

`FormBuilder.nonNullable` makes every control type `string` instead of `string | null`. `mat-error` is shown automatically when the control is invalid **and** touched.

12.10 Product list (Material table + RxJS search)

features/products/product-list.component.ts

TS

```

import { Component, inject, signal } from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { FormControl, ReactiveFormsModule } from '@angular/forms';
import { CurrencyPipe } from '@angular/common';
import { RouterLink } from '@angular/router';
import { MatTableModule } from '@angular/material/table';
import { MatButtonModule } from '@angular/material/button';
import { MatIconModule } from '@angular/material/icon';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatInputModule } from '@angular/material/input';
import { MatProgressSpinnerModule } from '@angular/material/progress-spinner';
import { MatSnackBar } from '@angular/material/snack-bar';
import { catchError, debounceTime, distinctUntilChanged, of, startWith, switchMap, tap } from 'rxjs';
import { AuthService } from '../../../core/auth.service';
import { ProductApi } from '../../../core/api.service';
import { Product } from '../../../core/models';
import { errorMessage } from '../../../core/http-error';

```

```

@Component({
  selector: 'app-product-list',
  imports: [ReactiveFormsModule, CurrencyPipe, RouterLink, MatTableModule, MatButtonModule,
    MatIconModule, MatFormFieldModule, MatInputModule, MatProgressSpinnerModule],
  template: `
    <div class="row between">
      <h2>Products</h2>
      @if (auth.isAdmin()) {
        <a mat-flat-button color="primary" routerLink="/products/new">
          <mat-icon>add</mat-icon> Add product
        </a>
      }
    </div>

    <mat-form-field appearance="outline" class="search">
      <mat-label>Search by name</mat-label>
      <input matInput [formControl]="search" />
    </mat-form-field>

    @if (loading()) { <mat-spinner diameter="36" /> }
    @if (error()) { <p class="error">{{ error() }}</p> }

    <table mat-table [dataSource]="products()" class="mat-elevation-z2">
      <ng-container matColumnDef="name">
        <th mat-header-cell *matHeaderCellDef>Product</th>
        <td mat-cell *matCellDef="let p">{{ p.name }}</td>
      </ng-container>
      <ng-container matColumnDef="price">
        <th mat-header-cell *matHeaderCellDef>Price</th>
        <td mat-cell *matCellDef="let p">{{ p.price | currency:'INR' }}</td>
      </ng-container>
      <ng-container matColumnDef="stock">
        <th mat-header-cell *matHeaderCellDef>Stock</th>
        <td mat-cell *matCellDef="let p">{{ p.stock }}</td>
      </ng-container>
    </table>
  `
})
export class ProductListComponent {
  auth = inject(AuthService);
  productApi = inject(ProductApi);
  products = signal<Product[]>([]);
  loading = signal(false);
  error = signal('');
  search = new FormControl('');

  ngOnInit() {
    this.auth.isAdmin().pipe(takeUntilDestroyed()).subscribe(isAdmin => {
      if (isAdmin) {
        this.addProductButtonVisible.set(true);
      }
    });
    this.search.valueChanges.pipe(
      startWith(''),
      debounceTime(500),
      distinctUntilChanged(),
    ).pipe(takeUntilDestroyed()).subscribe(searchTerm => {
      this.loading.set(true);
      this.error.set('');
      this.productApi.getProducts(searchTerm).pipe(takeUntilDestroyed()).subscribe(
        products => this.products.set(products),
        error => this.error.set(errorMessage(error)),
      );
      this.loading.set(false);
    });
  }
}

```

```

</ng-container>
<ng-container matColumnDef="actions">
  <th mat-header-cell *matHeaderCellDef></th>
  <td mat-cell *matCellDef="let p">
    @if (auth.isAdmin()) {
      <a mat-icon-button [routerLink]="['/products', p.id, 'edit']"><mat-icon>edit</mat-icon></a>
      <button mat-icon-button color="warn"
        (click)="remove(p)"><mat-icon>delete</mat-icon></button>
    }
  </td>
</ng-container>
<tr mat-header-row *matHeaderRowDef="cols"></tr>
<tr mat-row *matRowDef="let row; columns: cols"></tr>
</table>
@if (!loading() && products().length === 0) { <p class="muted">No products found.</p> }
,
})
export class ProductListComponent {
  auth = inject(AuthService);
  private api = inject(ProductApi);
  private snack = inject(MatSnackBar);

  search = new FormControl('', { nullable: true });
  products = signal<Product[]>([]);
  loading = signal(true);
  error = signal('');
  cols = ['name', 'price', 'stock', 'actions'];

  constructor() {
    this.search.valueChanges.pipe(
      startWith(''),
      debounceTime(300), // wait until the user stops typing
      distinctUntilChanged(),
      tap(() => { this.loading.set(true); this.error.set(''); }),
      switchMap(q => this.api.getAll(q).pipe( // cancels the previous request
        catchError(err => { this.error.set(errorMessage(err)); return of([] as Product[]); })
      )),
      takeUntilDestroyed(), // auto-unsubscribe
    ).subscribe(list => { this.products.set(list); this.loading.set(false); });
  }

  remove(p: Product) {
    if (!confirm(`Delete ${p.name}?`)) return;
    this.api.delete(p.id).subscribe({
      next: () => {
        this.products.update(list => list.filter(x => x.id !== p.id));
        this.snack.open('Product deleted', 'OK', { duration: 2500 });
      },
      error: err => this.snack.open(errorMessage(err), 'OK', { duration: 4000 }),
    });
  }
}

```

12.11 Product form (create + edit)

features/products/product-form.component.ts

TS

```

import { Component, inject, signal } from '@angular/core';
import { FormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import { ActivatedRoute, Router, RouterLink } from '@angular/router';
import { MatCardModule } from '@angular/material/card';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatInputModule } from '@angular/material/input';
import { MatButtonModule } from '@angular/material/button';
import { MatSnackBar } from '@angular/material/snack-bar';
import { ProductApi } from '../../../core/api.service';
import { errorMessage } from '../../../core/http-error';

@Component({

```

```

selector: 'app-product-form',
imports: [ReactiveFormsModule, RouterLink, MatCardModule, MatFormFieldModule, MatInputModule,
  MatButtonModule],
template: `
  <mat-card class="form-card">
    <mat-card-title>{{ id ? 'Edit product' : 'New product' }}</mat-card-title>
    <mat-card-content>
      <form [formGroup]="form" (ngSubmit)="save()" class="stack">
        <mat-form-field appearance="outline">
          <mat-label>Name</mat-label>
          <input matInput formControlName="name" />
          <mat-error>Name is required</mat-error>
        </mat-form-field>
        <mat-form-field appearance="outline">
          <mat-label>Price (INR)</mat-label>
          <input matInput type="number" formControlName="price" />
          <mat-error>Price must be at least 0.01</mat-error>
        </mat-form-field>
        <mat-form-field appearance="outline">
          <mat-label>Stock</mat-label>
          <input matInput type="number" formControlName="stock" />
          <mat-error>Stock cannot be negative</mat-error>
        </mat-form-field>
        @if (error()) { <p class="error">{{ error() }}</p> }
        <div class="row">
          <button mat-flat-button color="primary" [disabled]="form.invalid">Save</button>
          <a mat-button routerLink="/products">Cancel</a>
        </div>
      </form>
    </mat-card-content>
  </mat-card>
`,
})
export class ProductFormComponent {
  private fb = inject(FormBuilder).nonNullable;
  private api = inject(ProductApi);
  private router = inject(Router);
  private snack = inject(MatSnackBar);

  id = Number(inject(ActivatedRoute).snapshot.paramMap.get('id')) || null;
  error = signal('');

  form = this.fb.group({
    name: ['', Validators.required],
    price: [0, [Validators.required, Validators.min(0.01)]],
    stock: [0, [Validators.required, Validators.min(0)]],
  });

  constructor() {
    if (this.id) this.api.get(this.id).subscribe(p => this.form.patchValue(p));
  }

  save() {
    const body = this.form.getRawValue();
    const call = this.id ? this.api.update(this.id, body) : this.api.create(body);
    call.subscribe({
      next: () => { this.snack.open('Saved', 'OK', { duration: 2000 });
        this.router.navigate(['/products']); },
      error: err => this.error.set(errorMessage(err)),
    });
  }
}

```

12.12 Orders page (FormArray + order history)

features/orders/orders.component.ts

TS

```

import { Component, inject, signal } from '@angular/core';
import { FormArray, FormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import { CurrencyPipe, DatePipe } from '@angular/common';
import { MatCardModule } from '@angular/material/card';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatInputModule } from '@angular/material/input';
import { MatSelectModule } from '@angular/material/select';
import { MatButtonModule } from '@angular/material/button';
import { MatIconModule } from '@angular/material/icon';
import { MatSnackBar } from '@angular/material/snack-bar';
import { OrderApi, ProductApi } from '../../core/api.service';
import { Order, Product } from '../../core/models';
import { ErrorMessage } from '../../core/http-error';

@Component({
  selector: 'app-orders',
  imports: [ReactiveFormsModule, CurrencyPipe, DatePipe, MatCardModule, MatFormFieldModule,
    MatInputModule, MatSelectModule, MatButtonModule, MatIconModule],
  template: `
    <mat-card class="form-card">
      <mat-card-title>Place an order</mat-card-title>
      <mat-card-content>
        <form [formGroup]="form" (ngSubmit)="place()" class="stack">
          <div formArrayName="lines" class="stack">
            @for (line of lines.controls; track $index) {
              <div [formGroupName]="$index" class="row">
                <mat-form-field appearance="outline" class="grow">
                  <mat-label>Product</mat-label>
                  <mat-select formControlName="productId">
                    @for (p of products(); track p.id) {
                      <mat-option [value]="p.id">{{ p.name }} - {{ p.price | currency:'INR' }} (stock {{
                        p.stock }})</mat-option>
                    }
                  </mat-select>
                </mat-form-field>
                <mat-form-field appearance="outline" class="qty">
                  <mat-label>Qty</mat-label>
                  <input matInput type="number" formControlName="quantity" />
                </mat-form-field>
                <button type="button" mat-icon-button (click)="removeLine($index)"
                  [disabled]="lines.length === 1">
                  <mat-icon>close</mat-icon>
                </button>
              </div>
            }
          </div>
          <div class="row">
            <button type="button" mat-stroked-button (click)="addLine()"><mat-icon>add</mat-icon> Add
              item</button>
            <button mat-flat-button color="primary" [disabled]="form.invalid || placing()">Place
              order</button>
          </div>
          @if (error()) { <p class="error">{{ error() }}</p> }
        </form>
      </mat-card-content>
    </mat-card>

    <h3>My orders</h3>
    @for (o of orders(); track o.id) {
      <mat-card class="order">
        <div class="row between">
          <strong>Order #{{ o.id }}</strong>
          <span class="chip">{{ o.status }}</span>
        </div>
        <div class="muted">{{ o.createdAt | date:'medium' }}</div>
        @for (i of o.items; track i.productId) {
          <div class="row between">
            <span>{{ i.productName }} x {{ i.quantity }}</span>
            <span>{{ i.lineTotal | currency:'INR' }}</span>
          </div>
        }
      </mat-card>
    }
  `
})

```

```

        </div>
      }
      <div class="row between total"><span>Total (calculated by server)</span>
        <strong>{{ o.totalAmount | currency:'INR' }}</strong></div>
    </mat-card>
  } @empty {
    <p class="muted">You have not placed any orders yet.</p>
  }
},
})
export class OrdersComponent {
  private fb = inject(FormBuilder).nonNullable;
  private productApi = inject(ProductApi);
  private orderApi = inject(OrderApi);
  private snack = inject(MatSnackBar);

  products = signal<Product[]>([]);
  orders = signal<Order[]>([]);
  placing = signal(false);
  error = signal('');

  form = this.fb.group({ lines: this.fb.array([this.newLine()]) });
  get lines() { return this.form.controls.lines as FormArray; }

  constructor() {
    this.productApi.getAll().subscribe(p => this.products.set(p));
    this.loadOrders();
  }

  newLine() {
    return this.fb.group({
      productId: [null as number | null, Validators.required],
      quantity: [1, [Validators.required, Validators.min(1)]],
    });
  }
  addLine() { this.lines.push(this.newLine()); }
  removeLine(i: number) { this.lines.removeAt(i); }

  loadOrders() { this.orderApi.mine().subscribe(o => this.orders.set(o)); }

  place() {
    this.placing.set(true);
    this.error.set('');
    const items = this.lines.getRawValue().map(l => ({ productId: l.productId!, quantity: l.quantity }));
    this.orderApi.create(items).subscribe({
      next: o => {
        this.snack.open(`Order #${o.id} placed. Total ${o.totalAmount}`, 'OK', { duration: 4000 });
        this.form.reset({ lines: [{ productId: null, quantity: 1 }] });
        this.placing.set(false);
        this.loadOrders();
      },
      error: err => { this.error.set(errorMessage(err)); this.placing.set(false); },
    });
  }
}

```

WARNING The browser never calculates the real total

The page only **displays** `o.totalAmount` returned by the backend. If Angular computed it, anyone could edit the request in DevTools and pay 1 rupee.

12.13 Notifications and dashboard

features/notifications/notifications.component.ts

TS

```
import { Component, inject, signal } from '@angular/core';
import { DatePipe } from '@angular/common';
import { MatListModule } from '@angular/material/list';
import { MatIconModule } from '@angular/material/icon';
import { NotificationApi } from '../../../core/api.service';
import { AppNotification } from '../../../core/models';

@Component({
  selector: 'app-notifications',
  imports: [MatListModule, MatIconModule, DatePipe],
  template: `
    <h2>Notifications</h2>
    <mat-list>
      @for (n of items(); track n.id) {
        <mat-list-item>
          <mat-icon matListItemIcon>notifications</mat-icon>
          <div matListItemTitle>{{ n.message }}</div>
          <div matListItemLine>{{ n.createdAt | date:'medium' }}</div>
        </mat-list-item>
      } @empty { <p class="muted">No notifications yet. Place an order!</p> }
    </mat-list>
  `,
})
export class NotificationsComponent {
  items = signal<AppNotification[]>([]);
  constructor() { inject(NotificationApi).mine().subscribe(n => this.items.set(n)); }
}
```

features/dashboard/dashboard.component.ts

TS

```
import { Component, OnInit, inject, signal } from '@angular/core';
import { RouterLink } from '@angular/router';
import { MatCardModule } from '@angular/material/card';
import { forkJoin } from 'rxjs';
import { NotificationApi, OrderApi, ProductApi } from '../../../core/api.service';
import { AuthService } from '../../../core/auth.service';

@Component({
  selector: 'app-dashboard',
  imports: [MatCardModule, RouterLink],
  template: `
    <h2>Welcome, {{ auth.user()?.name }}</h2>
    <div class="grid3">
      <a routerLink="/products"><mat-card class="stat blue"><h1>{{ counts().products }}</h1>Products</mat-card></a>
      <a routerLink="/orders"><mat-card class="stat green"><h1>{{ counts().orders }}</h1>My orders</mat-card></a>
      <a routerLink="/notifications"><mat-card class="stat orange"><h1>{{ counts().notifications }}</h1>Notifications</mat-card></a>
    </div>
  `,
})
export class DashboardComponent implements OnInit {
  auth = inject(AuthService);
  private products = inject(ProductApi);
  private orders = inject(OrderApi);
  private notifications = inject(NotificationApi);
  counts = signal({ products: 0, orders: 0, notifications: 0 });

  ngOnInit() {
    forkJoin({
      // 3 requests in parallel, one result
      p: this.products.getAll(), o: this.orders.mine(), n: this.notifications.mine(),
    }).subscribe(r => this.counts.set({ products: r.p.length, orders: r.o.length, notifications: r.n.length }));
  }
}
```

12.14 RxJS operators you actually used

Operator	What it does	Where we used it
<code>tap</code>	Side effect without changing the value	Save token after login
<code>map</code>	Transform each value	Mapping form lines to request items (array map)
<code>switchMap</code>	On a new value, cancel the previous inner request and start a new one	Product search
<code>debounceTime</code>	Wait for a pause in events	Search box (300 ms)
<code>distinctUntilChanged</code>	Ignore repeated identical values	Search box
<code>catchError</code>	Recover from an error with a fallback	Interceptor, product list
<code>forkJoin</code>	Run requests in parallel, emit when all complete	Dashboard counters
<code>takeUntilDestroyed</code>	Auto-unsubscribe when the component is destroyed	Avoid memory leaks

12.15 Component lifecycle and data binding cheat-sheet

Syntax	Name	Example
<code>{{ value }}</code>	Interpolation	<code>{{ p.name }}</code>
<code>[property]="expr"</code>	Property binding (TS -> DOM)	<code>[disabled]="form.invalid"</code>
<code>(event)="handler()"</code>	Event binding (DOM -> TS)	<code>(click)="remove(p)"</code>
<code>[(ngModel)]</code>	Two-way binding (template forms)	Not used; we use reactive forms
<code>@if / @for / @switch</code>	Built-in control flow	<code>@for (p of products(); track p.id)</code>
<code> pipe</code>	Format a value in the template	<code>{{ price currency:'INR' }}</code>
<code>ngOnInit</code>	Runs once after inputs are set	Load data (dashboard)
<code>ngOnDestroy</code>	Runs before removal	Clean up subscriptions

CHECKPOINT Checkpoint

- Register in the UI -> you land on the dashboard with a name in the toolbar.
- Log out, log in as `admin@example.com`; create, edit, delete a product in the table.
- Log in as a normal user: no Add/Edit/Delete buttons; place an order; see it in 'My orders' and a notification a few seconds later.
- Open DevTools > Network: every API request to :8080 carries `Authorization: Bearer ....`

INTERVIEW Angular questions

- **Component vs service?** A component owns a piece of UI; a service holds reusable logic/data and is injected into components.
- **How does Angular call Spring Boot?** `HttpClient` returns Observables; JSON maps to interfaces; calls go to the Gateway; an interceptor adds the JWT.
- **What does an interceptor do?** Runs on every HTTP request/response - here, adds the Bearer token and handles 401 by logging out.
- **What is a guard?** A function that decides if a route can be activated - here `authGuard` and `adminGuard`.
- **Template-driven vs reactive forms?** Reactive forms define the model in TypeScript - more testable and scalable.
- **Signals vs Observables?** Signals hold the **current value** synchronously and drive change detection; Observables model async streams.
- **What is change detection?** Angular re-checks bindings when something changes; signals let it update only the affected parts.

13 Material UI and CSS

Angular Material components, theming, and the CSS you need for the app

NOTE Material UI vs Angular Material

Material UI (MUI) is the React library. **Angular Material** is the Angular library. Both implement Google's *Material Design* system (cards, toolbars, form fields, elevation, ripple effects), so the ideas transfer directly between them. In this project you use Angular Material.

13.1 Angular Material components used in the project

Component	Import	Used for
<code>mat-toolbar</code>	<code>MatToolbarModule</code>	Top navigation bar
<code>mat-card</code>	<code>MatCardModule</code>	Login box, form container, order cards
<code>mat-form-field</code> + <code>matInput</code>	<code>MatFormFieldModule</code> , <code>MatInputModule</code>	Text inputs with floating label and <code>mat-error</code>
<code>mat-select</code> / <code>mat-option</code>	<code>MatSelectModule</code>	Product dropdown in the order form
<code>mat-table</code>	<code>MatTableModule</code>	Product table (header/row/cell definitions)
<code>mat-button</code> , <code>mat-flat-button</code> , <code>mat-icon-button</code>	<code>MatButtonModule</code>	Buttons: text, filled, icon
<code>mat-icon</code>	<code>MatIconModule</code>	Icons by name (<code>add</code> , <code>edit</code> , <code>delete</code>)
<code>mat-list</code>	<code>MatListModule</code>	Notification list
<code>mat-spinner</code>	<code>MatProgressSpinnerModule</code>	Loading indicator
<code>MatSnackBar</code> (service)	<code>inject(MatSnackBar)</code>	Small toast messages

LEARN Standalone imports

Each component lists the Material modules it uses in its own `imports: [...]`. If a tag like `<mat-card>` shows up as plain text or an 'unknown element' error, you forgot to import its module in that component.

13.2 Theming in one minute

`ng add @angular/material` added a theme to `src/styles.css` (or `styles.scss`). The theme defines the **primary** colour (toolbar, filled buttons), an accent/secondary colour, typography and light/dark mode. To change the look, re-run `ng add @angular/material` or edit the theme call the schematic generated; do not hard-code colours inside every component.

13.3 Global stylesheet used by the app

Material handles components; **your own CSS** handles layout. Put this in `src/styles.css` (below the Material theme):

src/styles.css

CSS

```
:root {
  --space: 16px;
  --radius: 12px;
  --muted: #6b7280;
  --danger: #dc2626;
  --blue: #2563eb;
  --green: #16a34a;
  --orange: #ea580c;
}

* { box-sizing: border-box; } /* width includes padding + border */

body {
  margin: 0;
  font-family: Roboto, "Segoe UI", sans-serif;
  background: #f4f6fb;
  color: #1f2937;
}

.nav { gap: 8px; position: sticky; top: 0; z-index: 10; }
.nav .brand { font-weight: 600; margin: 0 16px 0 8px; }
.nav a.active { background: rgba(255, 255, 255, .18); }
.spacer { flex: 1 1 auto; } /* pushes items to the right */
.who { font-size: 14px; margin-right: 8px; }

.page { max-width: 1000px; margin: 0 auto; padding: var(--space); }

/* layout helpers */
.row { display: flex; align-items: center; gap: 12px; flex-wrap: wrap; }
.between { justify-content: space-between; }
.stack { display: flex; flex-direction: column; gap: 4px; }
.grow { flex: 1; }
.qty { width: 110px; }
.grid3 { display: grid; grid-template-columns: repeat(3, 1fr); gap: var(--space); }

/* cards */
.auth-card { max-width: 400px; margin: 10vh auto 0; padding: 12px; }
.form-card { max-width: 640px; margin-bottom: 24px; padding: 8px; }
.order { margin-bottom: 12px; padding: 16px; display: flex; flex-direction: column; gap: 6px; }
.total { border-top: 1px dashed #d1d5db; padding-top: 8px; }
.chip { background: #e0e7ff; color: #3730a3; padding: 2px 10px; border-radius: 999px; font-size: 12px; }

.stat { padding: 20px; text-align: center; color: white; border-radius: var(--radius); }
.stat h1 { margin: 0 0 4px; font-size: 40px; }
.stat.blue { background: var(--blue); }
.stat.green { background: var(--green); }
.stat.orange { background: var(--orange); }
a { text-decoration: none; }

table { width: 100%; }
.search { width: 320px; max-width: 100%; }
.error { color: var(--danger); }
.muted { color: var(--muted); }

/* responsive */
@media (max-width: 700px) {
  .grid3 { grid-template-columns: 1fr; } /* stack the 3 cards on phones */
  .who { display: none; }
}
```

13.4 CSS essentials (and the interview answers)

The box model

Every element is a box: **content + padding + border + margin** (inside to outside). With `box-sizing: border-box`, a `width: 200px` box stays 200 px even after you add padding and border - far easier to reason about. That is why the global stylesheet starts with `* { box-sizing: border-box; }`.

Selectors and specificity

Selector	Example	Specificity weight
Element	<code>p { }</code>	Lowest (0,0,1)
Class	<code>.error { }</code>	(0,1,0)
ID	<code>#login { }</code>	(1,0,0)
Inline style	<code>style="..."</code>	Beats all of the above
!important	<code>color: red !important</code>	Beats inline - avoid
Combinators	<code>.nav a, .nav > a, h2 + p</code>	Add up the weights of the parts
Pseudo	<code>a:hover, li:nth-child(2), ::before</code>	Pseudo-class = class weight

When two rules conflict, the one with **higher specificity** wins; if equal, the **later** rule wins. The cascade is also **inherited** (for example `color` and `font-family` flow to children).

Flexbox (one dimension)

Flexbox	CSS
<pre>.row { display: flex; flex-direction: row; /* main axis: row (default) or column */ justify-content: space-between; /* distribute along the MAIN axis */ align-items: center; /* align along the CROSS axis */ gap: 12px; flex-wrap: wrap; } .grow { flex: 1; } /* take the remaining space */</pre>	

Grid (two dimensions)

Grid	CSS
<pre>.cards { display: grid; grid-template-columns: repeat(3, 1fr); /* three equal columns */ gap: 16px; } /* responsive without media queries */ .auto { grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); }</pre>	
Choose	When
Flexbox	A row or column of items: navbars, button groups, form rows, centring
Grid	A page or card layout with rows and columns: dashboards, galleries

Units, positioning and responsive design

Topic	Quick facts
Units	px fixed; rem relative to root font size (scales with user settings); em relative to the parent font size; % of the parent; vw/vh of the viewport
position	static default; relative offset from normal place; absolute relative to nearest positioned ancestor; fixed relative to the viewport; sticky scrolls then sticks (our navbar)
display	block full width; inline flows in text; inline-block ; flex ; grid ; none removes the element
Responsive	Use a <code><meta name="viewport"></code> tag (Angular adds it), relative units, flex/grid wrap and <code>@media (max-width: 700px) { ... }</code>
CSS variables	Declare <code>--blue: #2563eb;</code> on <code>:root</code> , use <code>var(--blue)</code> anywhere; change the theme in one place
Angular scoping	Component CSS is encapsulated : Angular adds unique attributes so one component's styles do not leak into another

INTERVIEW CSS and Material questions

- **Box model?** content, padding, border, margin; **border-box** includes padding and border in the width.
- **Flexbox vs Grid?** Flexbox lays out in one dimension; Grid in two.
- **display:none vs visibility:hidden?** **none** removes the element from layout; **hidden** keeps its space.
- **em vs rem?** **em** depends on the parent font size and can compound; **rem** depends only on the root.
- **How does Angular keep CSS from leaking?** View encapsulation (emulated shadow DOM) - attributes like `_ngcontent-xyz` scope each component's styles.
- **Why Angular Material?** Accessible, consistent, tested components that follow Material Design; saves building tables, dialogs and form fields yourself.

14 JWT, localStorage and CORS End to End

Following one token from MySQL to the browser and back

14.1 The complete journey

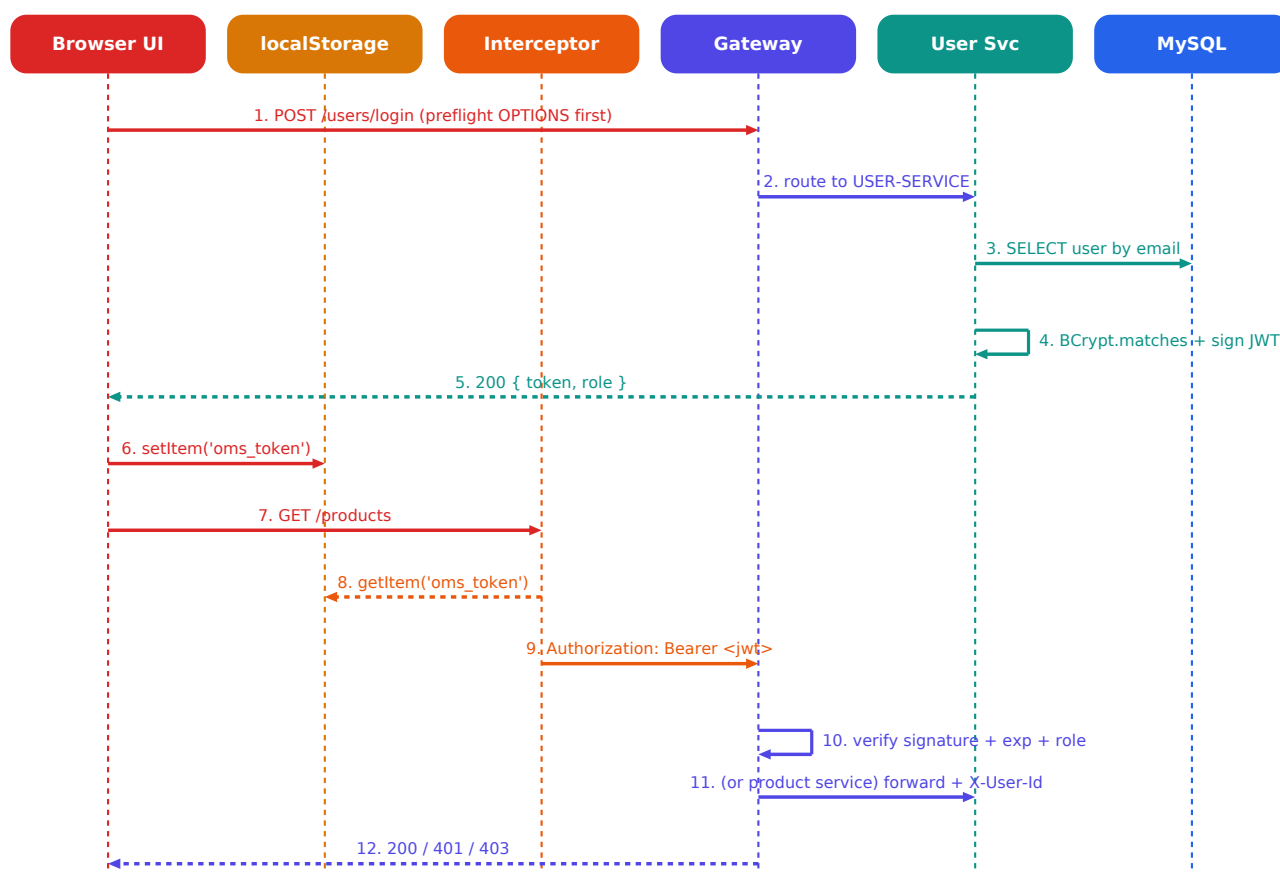


Figure 14.1 - Token journey: created once at login, stored in the browser, attached by the interceptor, verified by the Gateway

14.2 Token lifecycle

Phase	Who	What happens
1 Create	User Service	After BCrypt password check, <code>Jwts.builder()</code> creates a signed token with <code>sub</code> , <code>userId</code> , <code>role</code> , <code>exp</code>
2 Return	User Service -> Angular	<code>AuthResponse</code> JSON includes <code>token</code>
3 Store	<code>AuthService.save()</code>	<code>localStorage.setItem('oms_token', token)</code> and a <code>signal</code> holds the user
4 Attach	<code>authInterceptor</code>	Adds <code>Authorization: Bearer <token></code> to every request to the Gateway
5 Verify	Gateway <code>JwtAuthFilter</code>	Checks signature and expiry; enforces ADMIN rule; adds <code>X-User-Id</code> / <code>X-User-Role</code>
6 Use	Order Service	Reads <code>X-User-Id</code> from the header - the client never says who it is
7 Expire	Gateway -> Angular	After 2 hours the Gateway returns <code>401</code> ; the interceptor calls <code>logout()</code> and redirects to <code>/login</code>

Phase	Who	What happens
8 Logout	<code>AuthService.logout()</code>	Removes the token from <code>localStorage</code> , clears the signal, navigates to <code>/login</code>

14.3 Where to store the token?

Option	Pros	Cons / risk
localStorage (this project)	Simple; survives refresh and new tabs; easy to attach in JS	Readable by any JavaScript on the page -> XSS can steal the token
sessionStorage	Cleared when the tab closes	Same XSS exposure; no sharing between tabs
httpOnly, Secure, SameSite cookie	JavaScript cannot read it -> much better against XSS theft	Sent automatically -> needs CSRF protection and cookie-aware CORS config
In memory only	Not persisted, hard to steal after the fact	Lost on refresh; usually paired with a refresh cookie

WARNING Be honest about localStorage in interviews

"For this learning project I used `localStorage` because it makes the flow easy to see. In production I would prefer an `httpOnly` cookie with `SameSite`, short-lived access tokens, and refresh-token rotation, and I would harden against XSS with Angular's built-in sanitisation, a Content-Security-Policy and no `innerHTML` with untrusted data."

That answer shows you understand the **trade-off**, which is what interviewers want.

14.4 CORS: the exact sequence

1. The page at `http://localhost:4200` calls `http://localhost:8080/products` with an `Authorization` header.
2. This is a *non-simple* request, so the browser first sends `OPTIONS /products` with `Origin` and `Access-Control-Request-Headers: authorization`.
3. The Gateway (`globalcors`) replies `Access-Control-Allow-Origin: http://localhost:4200` and the allowed methods/headers. The Gateway filter lets `OPTIONS` through without a token.
4. The browser then sends the real `GET` with the token and allows Angular to read the response.
5. If any step is missing you see '**blocked by CORS policy**' in the console even though the server works fine in Postman.

14.5 Debug it like a professional

Where	What to look at	Learn
DevTools > Application > Local Storage	<code>oms_token</code> , <code>oms_user</code>	Proof the token is stored; delete it to simulate logout
DevTools > Network > a request > Headers	<code>Authorization: Bearer eyJ...</code>	Proof the interceptor works
DevTools > Network > the OPTIONS call	Status 200 + <code>Access-Control-*</code> headers	How CORS preflight works
jwt.io (fake data only)	Paste the token	Header, payload (claims), expiry; the signature check needs the secret

Where	What to look at	Learn
Gateway console	Route and filter logs	Which route matched
Eureka dashboard	Instance status	Is the target service registered?

14.6 Common failures and what they mean

Symptom	Likely cause	Fix
401 on every request after login	Different <code>JWT_SECRET</code> in User Service and Gateway, or the header is missing	Set the same secret in both; check the Network tab
<code>WeakKeyException</code> at login	Secret shorter than 32 characters	Use a 48-character secret
CORS error but Postman works	Origin not allowed, or CORS configured twice	Configure only in the Gateway: origin <code>http://localhost:4200</code> , headers <code>Authorization</code> , <code>Content-Type</code>
403 on POST /products	Logged in as USER, not ADMIN	Log in as <code>admin@example.com</code>
Works, then suddenly 401	Token expired (2 h)	Log in again; later add a refresh token
X-User-Id header missing in Order Service	Calling Order Service directly instead of via the Gateway	Call via <code>:8080</code> , or add the header manually in Swagger
Angular <code>Unknown element mat-card</code>	Module not in the component's <code>imports</code>	Import <code>MatCardModule</code>

INTERVIEW Full-stack security questions

- **Walk me through a login in your app.** Angular posts credentials to the Gateway; User Service verifies the BCrypt hash and returns a signed JWT; Angular stores it; the interceptor attaches it to every request; the Gateway validates it and passes the identity downstream.
- **What is CORS and who enforces it?** A browser security rule on cross-origin requests. The **browser** enforces it; the **server** declares what is allowed.
- **Is localStorage safe for tokens?** It is exposed to XSS. httpOnly cookies avoid JS access but need CSRF protection.
- **What is CSRF?** A malicious site makes the browser send a request with your cookies. Header-based tokens (Bearer) are not sent automatically, so they are not vulnerable to classic CSRF.
- **How do you secure Angular routes?** A `CanActivate` guard - but the **real** protection is the backend; a guard only improves UX.

15 Testing and Integration

Unit tests, API tests, the 14-point checklist and a reliable start-up script

15.1 The testing pyramid for this project

Level	Tool	What it proves	Speed
Unit	JUnit 5 + Mockito	One class (e.g. <code>OrderService</code>) works with fake dependencies	milliseconds
Slice / API	<code>@WebMvcTest</code> + MockMvc	Controller mapping, validation, JSON, error handler	fast
Integration	Swagger / Postman / browser	All services + DB + Kafka cooperate	slow

Maven's starter-test is already in every project (JUnit 5, Mockito, AssertJ, Spring Test). Run tests with `.\mvnw.cmd test`.

15.2 Unit test: ProductService (Mockito)

src/test/java/.../ProductServiceTest.java

JAVA

```
package com.af rudeen.product.service;

import com.af rudeen.product.common.ResourceNotFoundException;
import com.af rudeen.product.dto.ProductRequest;
import com.af rudeen.product.entity.Product;
import com.af rudeen.product.repository.ProductRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import java.math.BigDecimal;
import java.util.Optional;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class ProductServiceTest {

    @Mock ProductRepository repository;
    @InjectMocks ProductService service;

    @Test
    void findById_whenMissing_throwsNotFound() {
        when(repository.findById(99L)).thenReturn(Optional.empty());

        assertThrows(ResourceNotFoundException.class, () -> service.findById(99L));
    }

    @Test
    void create_savesProduct() {
        when(repository.save(any(Product.class))).thenAnswer(inv -> inv.getArgument(0));

        Product saved = service.create(new ProductRequest("Mouse", new BigDecimal("599.00"), 5));

        assertEquals("Mouse", saved.getName());
        verify(repository).save(any(Product.class));
    }
}
```

```
}
}
```

Pattern: **Arrange** (stub with **when**), **Act** (call the method), **Assert** (check result and **verify** interactions).

15.3 Controller test: validation and error JSON

src/test/java/.../ProductControllerTest.java

JAVA

```
package com.af rudeen.product.controller;

import com.af rudeen.product.common.ResourceNotFoundException;
import com.af rudeen.product.service.ProductService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(ProductController.class)
class ProductControllerTest {

    @Autowired MockMvc mvc;
    @MockitoBean ProductService service; // replaces the real bean with a mock

    @Test
    void getId_missing_returns404Json() throws Exception {
        when(service.findById(9L)).thenReturn(new ResourceNotFoundException("Product not found: 9"));

        mvc.perform(get("/products/9"))
            .andExpect(status().isNotFound())
            .andExpect(jsonPath("$.message").value("Product not found: 9"));
    }

    @Test
    void create_invalidBody_returns400() throws Exception {
        String bad = "{\"name\":\"\", \"price\":-1, \"stock\":1}";

        mvc.perform(post("/products").contentType(MediaType.APPLICATION_JSON).content(bad))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.fieldErrors.name").exists());
    }
}
```

@WebMvcTest starts only the web layer (controllers + **@RestControllerAdvice**), not the database. **@MockBean** was replaced by **@MockitoBean** in recent Spring Boot versions.

15.4 Unit test: the order total is computed on the server

src/test/java/.../OrderServiceTest.java

JAVA

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock OrderRepository repository;
    @Mock ProductGateway products;
    @Mock OrderEventPublisher events;
    @InjectMocks OrderService service;

    @Test
    void create_calculatesTotalFromProductServicePrice() {
```

```

        when(products.getProduct(1L))
            .thenReturn(new ProductResponse(1L, "Oil Filter", new BigDecimal("349.50"), 10));
        when(repository.save(any(Order.class))).thenAnswer(inv -> inv.getArgument(0));

        OrderResponse r = service.create(7L,
            new CreateOrderRequest(List.of(new OrderItemRequest(1L, 2))));

        assertEquals(new BigDecimal("699.00"), r.totalAmount());
        verify(events).publish(any());
    }

    @Test
    void create_whenStockTooLow_throwsAndSavesNothing() {
        when(products.getProduct(1L))
            .thenReturn(new ProductResponse(1L, "Oil Filter", new BigDecimal("349.50"), 1));

        assertThrows(BusinessException.class, () -> service.create(7L,
            new CreateOrderRequest(List.of(new OrderItemRequest(1L, 5))));

        verify(repository, never()).save(any());
        verifyNoInteractions(events);
    }
}

```

INTERVIEW Testing questions

- **@Mock vs @MockitoBean?** @Mock is plain Mockito for unit tests; @MockitoBean puts a mock into the Spring context for slice/integration tests.
- **@SpringBootTest vs @WebMvcTest vs @DataJpaTest?** Full context vs web layer only vs JPA layer only (with an embedded DB).
- **How do you test a service that calls another service?** Mock the client (here **ProductGateway**) in unit tests; use contract tests or a test double (WireMock) for integration.
- **What does verify do?** Asserts a mock was called (with certain arguments, N times, or never).

15.5 The 14-point integration checklist

#	Test	Expected	Done
1	Register a new user	201 + token	[]
2	Log in and receive a JWT	200 + token + role	[]
3	Reject an invalid login	401 Invalid email or password	[]
4	Create a product through the Gateway (admin)	201	[]
5	List products in Angular	Table rows appear	[]
6	Update a product from Angular	Row changes; DB updated	[]
7	Delete a product	204; row removed	[]
8	Create an order for an existing product	201 + server total	[]
9	Reject an order for a missing product	404 JSON error	[]
10	Verify order data in order_db	orders + order_items rows	[]
11	Verify Kafka notification in notification_db	Row with message	[]
12	Product Service outage	503 then fast failures (breaker OPEN)	[]
13	HTTP error responses	400/401/403/404/503 all return clean JSON or status	[]
14	Eureka registrations	USER, PRODUCT, ORDER, NOTIFICATION, API-GATEWAY all UP	[]

15.6 Postman tips

- Create an **environment** with `baseUrl = http://localhost:8080` and `token`.
- In the login request's **Tests** tab: `pm.environment.set('token', pm.response.json().token);`
- Set collection **Authorization** to `*Bearer Token*` with value `{{token}}` - every request inherits it.
- Save one folder per service; this collection becomes part of your GitHub README.

15.7 Start everything reliably

Create `infrastructure/start-all.ps1`. It starts each service in its own window in the right order, with the secrets set for those windows only.

`infrastructure/start-all.ps1`

BASH

```
$root = "E:\order-management-system"
$env:DB_PASSWORD = "ChangeThis_123"
$env:JWT_SECRET = "paste-your-48-character-secret-here-xxxxxxxxxxxxx"

function Start-Svc($name, $delaySeconds) {
    Start-Process powershell -ArgumentList "-NoExit", "-Command",
        "`$host.UI.RawUI.WindowTitle='$name'; cd '$root\backend\$name'; .\mvnw.cmd spring-boot:run"
    Start-Sleep -Seconds $delaySeconds
}

docker compose -f "$root\infrastructure\docker-compose.yml" up -d
Start-Sleep -Seconds 8

Start-Svc "service-discovery" 25
Start-Svc "product-service" 10
Start-Svc "user-service" 10
Start-Svc "order-service" 10
Start-Svc "notification-service" 10
Start-Svc "api-gateway" 5

Write-Host "Wait ~40 s, then open http://localhost:8761 (Eureka) and start Angular: ng serve"
```

WARNING Do not commit the real passwords in `start-all.ps1`

Either read them from your own user environment variables, or add `infrastructure/start-all.ps1` to `.gitignore` and commit an example file instead.

Order	Start	Why this order
1	MySQL, Kafka (Docker)	Everything else needs them
2	Service Discovery	Services register here at startup
3	Product, User, Order, Notification	Business services
4	API Gateway	Discovers the services' routes
5	Angular (<code>ng serve</code>)	UI calls the Gateway

16 Spring Annotations Cheat-Sheet

Every annotation in the project, grouped, with what interviewers ask

Tip: do not memorise the table. For each annotation, **find where you used it in the project** and say its purpose in one sentence.

16.1 Core / Spring Boot

Annotation	Purpose	Used in project
@SpringBootApplication	= @Configuration + @EnableAutoConfiguration + @ComponentScan; entry point	Every *Application class
@Component	Generic Spring-managed bean	ProductGateway, handlers
@Service	Business-logic bean (a @Component with meaning)	ProductService, OrderService
@Repository	Data-access bean; translates persistence exceptions	Optional on JpaRepository interfaces
@Controller / @RestController	Web controller / controller + @ResponseBody (JSON)	All controllers
@Configuration + @Bean	Declare beans in Java code	SecurityConfig, SampleData, KafkaTopicConfig
@Autowired	Inject a dependency (constructor injection needs no annotation if one constructor)	Implicit via constructors
@Qualifier / @Primary	Choose among several beans of the same type	-
@Value("\${key}")	Inject a property value	JwtService, AdminSeeder
@ConfigurationProperties	Bind a group of properties to a class	Alternative to many @Value
@Profile	Bean only active in a profile (dev/prod)	-
@Scope	Bean scope: singleton (default), prototype, request, session	-
@Lazy	Create bean on first use	-
@PostConstruct / @PreDestroy	Run after construction / before destruction	-
@Conditional...	Conditional bean creation (basis of auto-configuration)	Behind the scenes

16.2 Web / REST

Annotation	Purpose	Example
@RequestMapping	Map URL (+ method) to class/method	@RequestMapping("/products")

Annotation	Purpose	Example
<code>@GetMapping</code> , <code>@PostMapping</code> , <code>@PutMapping</code> , <code>@DeleteMapping</code> , <code>@PatchMapping</code>	Shortcuts per HTTP method	<code>@GetMapping("/{id}")</code>
<code>@PathVariable</code>	Value from the URL path	<code>/products/5 -> Long id</code>
<code>@RequestParam</code>	Query-string value	<code>/products?search=oil</code>
<code>@RequestBody</code>	Deserialise JSON body into an object	<code>@RequestBody ProductRequest r</code>
<code>@RequestHeader</code>	Read a header	<code>@RequestHeader("X-User-Id")</code>
<code>@ResponseStatus</code>	Set the response status code	<code>@ResponseStatus(HttpStatus.CREATED)</code>
<code>@Valid</code> / <code>@Validated</code>	Trigger Bean Validation	On <code>@RequestBody</code> parameter
<code>@RestControllerAdvice</code> + <code>@ExceptionHandler</code>	Global exception handling	<code>GlobalExceptionHandler</code>
<code>@CrossOrigin</code>	CORS on a controller (we use the Gateway instead)	-

16.3 Validation (Jakarta Bean Validation)

Annotation	Checks
<code>@NotNull</code>	Not null
<code>@NotBlank</code>	String not null and not only whitespace
<code>@NotEmpty</code>	Collection/String not null and not empty
<code>@Size(min,max)</code>	Length of a String / size of a collection
<code>@Min</code> , <code>@Max</code> , <code>@DecimalMin</code>	Numeric bounds
<code>@Email</code>	Looks like an email
<code>@Pattern(regexp)</code>	Matches a regular expression
<code>@Positive</code> , <code>@PositiveOrZero</code>	Sign checks

16.4 JPA / Hibernate / Spring Data

Annotation	Purpose	Notes
<code>@Entity</code>	Class mapped to a table	Needs a no-arg constructor and <code>@Id</code>
<code>@Table(name=..., uniqueConstraints=...)</code>	Table name and constraints	Use <code>orders</code> , not <code>order</code>
<code>@MappedSuperclass</code>	Base class whose fields are inherited into child tables	<code>BaseEntity</code> (abstract)
<code>@Id</code>	Primary key	
<code>@GeneratedValue(strategy=...)</code>	Key generation	<code>IDENTITY</code> = MySQL <code>AUTO_INCREMENT</code>
<code>@Column</code>	Column details: <code>nullable</code> , <code>length</code> , <code>precision</code> , <code>scale</code> , <code>updatable</code>	

Annotation	Purpose	Notes
@Enumerated(EnumType.STRING)	Store enum as text	Never use ORDINAL
@OneToMany, @ManyToOne, @OneToOne, @ManyToMany	Relationships	mappedBy marks the non-owning side
@JoinColumn	Foreign-key column on the owning side	order_id
cascade, orphanRemoval	Propagate operations / delete removed children	Order -> items
fetch = LAZY / EAGER	When related data loads	Prefer LAZY
@PrePersist, @PreUpdate	Lifecycle callbacks	Timestamps in BaseEntity
@Transactional	Run method in a DB transaction	Service layer
@Query	Custom JPQL/native query	Fetch join in OrderRepository
@Param, @Modifying	Named parameter / update-delete query	
@EntityGraph	Declarative fetch plan (avoids N+1)	Alternative to JOIN FETCH
@Version	Optimistic locking	Stretch goal for stock

16.5 Spring Cloud, Kafka, Resilience4j, Swagger

Annotation	Module	Purpose
@EnableEurekaServer	Eureka	Turns the app into the registry
@EnableDiscoveryClient	Discovery	Register with Eureka (automatic when the starter is present)
@EnableFeignClients	OpenFeign	Scan and create Feign clients
@FeignClient(name=...)	OpenFeign	Declarative HTTP client interface
@LoadBalanced	LoadBalancer	Client-side balancing for RestTemplate/WebClient.Builder
@KafkaListener	Spring Kafka	Consume messages from topics
@EnableKafka	Spring Kafka	Enable listener detection (auto in Boot)
@CircuitBreaker, @Retry, @Bulkhead, @RateLimiter	Resilience4j	Fault-tolerance on a method
@OpenAPIDefinition, @SecurityScheme	springdoc	API metadata and JWT auth scheme
@Tag, @Operation, @Schema, @ApiResponse, @Parameter	springdoc	Describe controllers, endpoints, DTOs

16.6 Testing

Annotation	Purpose
@SpringBootTest	Load the full application context
@WebMvcTest	Web layer only; use with MockMvc
@DataJpaTest	JPA layer only with an embedded database
@ExtendWith(MockitoExtension.class)	Enable Mockito annotations in plain JUnit 5 tests

Annotation	Purpose
@Mock / @InjectMocks	Create a mock / inject mocks into the class under test
@MockitoBean	Replace a Spring bean with a mock (successor of @MockBean)
@Test, @BeforeEach, @Disabled	JUnit 5 lifecycle and control

16.7 Concepts behind the annotations

Question	Short answer
What is IoC / DI?	The container creates objects and injects their dependencies instead of your code calling new . Result: loose coupling, easy testing.
Constructor vs field vs setter injection	Constructor: immutable, required, testable (preferred). Field: shortest but hides dependencies and hampers tests. Setter: optional dependencies.
@Component vs @Bean	@Component marks your own class for scanning. @Bean is a method in a @Configuration class that creates a bean - needed for third-party classes.
Stereotypes: @Service vs @Repository	Functionally @Component; @Repository also translates persistence exceptions; names communicate the layer.
Bean scopes	singleton (default, one per container), prototype (new each time), request, session, application.
How does auto-configuration work?	@EnableAutoConfiguration loads configuration classes listed in AutoConfiguration.imports; each uses @ConditionalOnClass/@ConditionalOnMissingBean so defaults apply only when you have not defined your own.
What does spring-boot-starter-* do?	A curated dependency bundle with compatible versions (e.g. starter-web = Spring MVC + Jackson + Tomcat).
Bean lifecycle	Instantiate -> inject dependencies -> @PostConstruct -> ready -> @PreDestroy -> destroy.
@Transactional rules	Proxy-based: public methods called from outside the bean. Rolls back on unchecked exceptions by default (rollbackFor for checked). Self-invocation bypasses it.
Propagation REQUIRED vs REQUIRES_NEW	REQUIRED joins the current transaction (default). REQUIRES_NEW suspends it and starts an independent one.
Spring MVC request flow	Request -> DispatcherServlet -> HandlerMapping -> Controller -> (message converter writes JSON) -> response.
Embedded server	Boot ships Tomcat inside the jar; java -jar app.jar runs the web app.
Profiles	application-dev.yml / application-prod.yml, activated with spring.profiles.active.

17 MySQL Interview Queries

A practice database, 45+ queries, indexes, transactions and the theory behind them

Interviewers love SQL because it shows how you think. This chapter gives you a **small practice database** (users, products, orders, employees) so you can run every query yourself in Workbench and **see** the result. Type each query - do not paste - and predict the output first.

17.1 Create the practice database

MySQL Workbench: run once (separate from your service databases)

SQL

```
CREATE DATABASE practice_db;
USE practice_db;

CREATE TABLE users (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE,
  city VARCHAR(50),
  created_at DATE
);
CREATE TABLE products (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(80) NOT NULL,
  category VARCHAR(40),
  price DECIMAL(10,2) NOT NULL,
  stock INT NOT NULL DEFAULT 0
);
CREATE TABLE orders (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT NOT NULL,
  total DECIMAL(12,2),
  status VARCHAR(20),
  order_date DATE,
  FOREIGN KEY (user_id) REFERENCES users(id)
);
CREATE TABLE order_items (
  id INT PRIMARY KEY AUTO_INCREMENT,
  order_id INT NOT NULL,
  product_id INT NOT NULL,
  quantity INT NOT NULL,
  unit_price DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (order_id) REFERENCES orders(id),
  FOREIGN KEY (product_id) REFERENCES products(id)
);
CREATE TABLE employees (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50),
  department VARCHAR(30),
  salary DECIMAL(10,2),
  manager_id INT NULL
);

INSERT INTO users (id,name,email,city,created_at) VALUES
(1,'Afru','afru@x.com','Chennai','2025-01-10'), (2,'Bala','bala@x.com','Madurai','2025-02-14'),
(3,'Chitra','chitra@x.com','Chennai','2025-03-01'), (4,'Dev','dev@x.com','Trichy','2025-03-20'),
(5,'Eswari','esw@x.com','Chennai','2025-04-02');

INSERT INTO products (id,name,category,price,stock) VALUES
(1,'Fuel Injector','Engine',1299.00,20), (2,'Oil Filter','Engine',349.50,100),
(3,'Piston Ring Set','Engine',2150.00,15), (4,'Brake Pad','Brakes',899.00,40),
(5,'Brake Disc','Brakes',1899.00,12), (6,'Wiper Blade','Body',250.00,0);
```

```

INSERT INTO orders (id,user_id,total,status,order_date) VALUES
(1,1,2598.00,'DELIVERED','2025-05-01'), (2,1,349.50,'DELIVERED','2025-05-10'),
(3,2,2150.00,'CANCELLED','2025-05-12'), (4,3,2798.00,'DELIVERED','2025-06-03'),
(5,3,899.00,'SHIPPED','2025-06-20'), (6,2,1899.00,'DELIVERED','2025-06-25');

INSERT INTO order_items (order_id,product_id,quantity,unit_price) VALUES
(1,1,2,1299.00), (2,2,1,349.50), (3,3,1,2150.00), (4,5,1,1899.00),
(4,4,1,899.00), (5,4,1,899.00), (6,5,1,1899.00);

INSERT INTO employees (id,name,department,salary,manager_id) VALUES
(1,'Ravi','IT',90000,NULL), (2,'Meena','IT',75000,1), (3,'Karthik','IT',75000,1),
(4,'Latha','HR',60000,NULL), (5,'Suresh','HR',52000,4),
(6,'Priya','Sales',65000,NULL), (7,'Arun','Sales',70000,6), (8,'Divya','Sales',48000,6);

```

CHECKPOINT Checkpoint

SELECT COUNT(*) FROM users; returns 5, **orders** has 6 rows, **employees** has 8. Users 4 and 5 have **no orders**; product 6 was **never ordered**.

17.2 Basics

1. Filtering, sorting, limiting

```

SELECT name, price
FROM products
WHERE category = 'Engine' AND price BETWEEN 300 AND 2000
ORDER BY price DESC
LIMIT 2;

```

Order of execution (logical): FROM > WHERE > GROUP BY > HAVING > SELECT > ORDER BY > LIMIT.

2. LIKE, IN, IS NULL, DISTINCT

```

SELECT DISTINCT city FROM users;
SELECT * FROM users WHERE name LIKE 'A%';           -- starts with A
SELECT * FROM users WHERE city IN ('Chennai','Trichy');
SELECT * FROM employees WHERE manager_id IS NULL;  -- never '= NULL'

```

17.3 Aggregation: GROUP BY and HAVING

3. Users per city, only cities with more than one user

```

SELECT city, COUNT(*) AS total_users
FROM users
GROUP BY city
HAVING COUNT(*) > 1;

```

Result: Chennai = 3. WHERE filters **rows before** grouping; HAVING filters **groups after** aggregation.

4. Revenue per month (ignore cancelled orders)

```

SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
       SUM(total) AS revenue,
       COUNT(*) AS orders
FROM orders
WHERE status <> 'CANCELLED'
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;

```

5. Revenue per product category

```

SELECT p.category, SUM(oi.quantity * oi.unit_price) AS revenue
FROM order_items oi
JOIN products p ON p.id = oi.product_id

```

```
GROUP BY p.category
ORDER BY revenue DESC;
```

17.4 Joins

Join	Returns	Typical use
INNER JOIN	Only rows that match in both tables	Orders with their user
LEFT JOIN	All rows from the left + matches (NULL if none)	Users with or without orders
RIGHT JOIN	All rows from the right + matches	Rare; swap the tables and use LEFT
FULL OUTER JOIN	Everything from both (MySQL: LEFT UNION RIGHT)	Reconciliation
CROSS JOIN	Every combination (cartesian product)	Generating combinations
Self join	A table joined to itself	Employee -> manager

6. Order details across four tables

```
SELECT u.name AS customer, o.id AS order_id, p.name AS product,
       oi.quantity, oi.unit_price
FROM orders o
JOIN users u      ON u.id = o.user_id
JOIN order_items oi ON oi.order_id = o.id
JOIN products p   ON p.id = oi.product_id
ORDER BY o.id;
```

7. Users who never placed an order (3 ways)

```
-- a) LEFT JOIN + IS NULL
SELECT u.* FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE o.id IS NULL;

-- b) NOT EXISTS (safe with NULLs)
SELECT u.* FROM users u
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.id);

-- c) NOT IN (dangerous: returns nothing if the subquery contains a NULL)
SELECT * FROM users WHERE id NOT IN (SELECT user_id FROM orders);
```

Result: Dev and Eswari. Prefer (a) or (b); explain why (c) can bite you.

8. Products that were never ordered

```
SELECT p.*
FROM products p
LEFT JOIN order_items oi ON oi.product_id = p.id
WHERE oi.id IS NULL;
```

Result: Wiper Blade.

9. Employees and their manager's name (self join)

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON m.id = e.manager_id;
```

10. Employees who earn more than their manager

```
SELECT e.name, e.salary, m.name AS manager, m.salary AS manager_salary
FROM employees e
JOIN employees m ON m.id = e.manager_id
```

```
WHERE e.salary > m.salary;
```

Result: Arun (70000) > Priya (65000).

11. Total spent per user, including users with zero

```
SELECT u.name, COALESCE(SUM(o.total), 0) AS spent
FROM users u
LEFT JOIN orders o ON o.user_id = u.id AND o.status = 'DELIVERED'
GROUP BY u.id, u.name
ORDER BY spent DESC;
```

Put the status filter in the **ON** clause, not in **WHERE**, otherwise the LEFT JOIN behaves like an INNER JOIN and zero-order users disappear.

17.4b Subqueries

12. Orders bigger than the average order

```
SELECT id, total
FROM orders
WHERE total > (SELECT AVG(total) FROM orders);    -- scalar subquery
```

13. Users with at least one delivered order (EXISTS)

```
SELECT u.name
FROM users u
WHERE EXISTS (SELECT 1 FROM orders o
              WHERE o.user_id = u.id AND o.status = 'DELIVERED');
```

EXISTS stops at the first match - often faster than IN for large subqueries.

14. Users who bought both product 4 and product 5

```
SELECT o.user_id
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
WHERE oi.product_id IN (4, 5)
GROUP BY o.user_id
HAVING COUNT(DISTINCT oi.product_id) = 2;
```

Result: user 3 (Chitra).

15. Highest-paid employee(s) in each department

```
SELECT e.name, e.department, e.salary
FROM employees e
WHERE (e.department, e.salary) IN (
    SELECT department, MAX(salary) FROM employees GROUP BY department);
```

17.5 The classic interview questions

16. Second highest salary (3 ways)

```
-- a) subquery
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- b) LIMIT/OFFSET (DISTINCT handles ties)
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1;

-- c) window function
SELECT salary FROM (
```

```
SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees
) t WHERE rnk = 2 LIMIT 1;
```

Result: 75000 (two people earn it - that is why you must think about ties).

17. Nth highest salary (N = 3)

```
SELECT DISTINCT salary
FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
      FROM employees) t
WHERE rnk = 3;
```

Change **3** to any N. Result: 70000.

18. Find duplicate values

```
SELECT salary, COUNT(*) AS cnt
FROM employees
GROUP BY salary
HAVING COUNT(*) > 1;
```

Result: 75000 appears twice. Same pattern finds duplicate emails.

19. Delete duplicate rows, keep the lowest id

```
-- TRY INSIDE A TRANSACTION OR ON A COPY FIRST
DELETE e1
FROM employees e1
JOIN employees e2
  ON e1.name = e2.name AND e1.id > e2.id;
```

Self-join pairs each row with an earlier row that has the same key and deletes the later one.

20. Department with the highest average salary

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
ORDER BY avg_salary DESC
LIMIT 1;
```

21. Employees without a manager / departments with more than 2 people

```
SELECT * FROM employees WHERE manager_id IS NULL;

SELECT department, COUNT(*) FROM employees
GROUP BY department HAVING COUNT(*) > 2;
```

22. Customers who ordered in consecutive months (idea: LAG)

```
SELECT user_id, order_date,
       LAG(order_date) OVER (PARTITION BY user_id ORDER BY order_date) AS previous_order
FROM orders;
```

LAG looks at the previous row inside the same partition. Compare the dates with **DATEDIFF**.

17.6 Window functions

salary	ROW_NUMBER()	RANK()	DENSE_RANK()
90000	1	1	1
75000	2	2	2
75000	3	2	2
70000	4	4	3

salary	ROW_NUMBER()	RANK()	DENSE_RANK()
65000	5	5	4

ROW_NUMBER is always unique. **RANK** gives ties the same rank and **skips** the next numbers. **DENSE_RANK** gives ties the same rank and **does not skip**.

23. Ranking functions

```
SELECT name, department, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_no,
       RANK() OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk
FROM employees;
```

24. Top earner per department (PARTITION BY)

```
WITH ranked AS (
  SELECT name, department, salary,
         ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rn
  FROM employees
)
SELECT name, department, salary FROM ranked WHERE rn = 1;
```

Change **rn = 1** to **rn <= 2** for the top 2 per department - a very common follow-up.

25. Running total of revenue

```
SELECT order_date, total,
       SUM(total) OVER (ORDER BY order_date, id) AS running_total
FROM orders
WHERE status <> 'CANCELLED';
```

26. Each employee's salary vs the department average

```
SELECT name, department, salary,
       AVG(salary) OVER (PARTITION BY department) AS dept_avg,
       salary - AVG(salary) OVER (PARTITION BY department) AS diff
FROM employees;
```

17.7 CASE, UNION, CTE, views, pagination

27. CASE expression

```
SELECT id, total,
       CASE WHEN total >= 2500 THEN 'LARGE'
            WHEN total >= 1000 THEN 'MEDIUM'
            ELSE 'SMALL' END AS size
FROM orders;
```

28. UNION vs UNION ALL

```
SELECT city FROM users
UNION          -- removes duplicates (sorts/dedups: slower)
SELECT 'Chennai';

SELECT city FROM users
UNION ALL      -- keeps duplicates (faster)
SELECT 'Chennai';
```

29. CTE (readable subqueries)

```
WITH user_spend AS (
  SELECT user_id, SUM(total) AS spent
  FROM orders WHERE status = 'DELIVERED' GROUP BY user_id
```

```
)
SELECT u.name, s.spent
FROM user_spend s JOIN users u ON u.id = s.user_id
WHERE s.spent > 2000;
```

30. View

```
CREATE VIEW v_delivered_orders AS
SELECT o.id, u.name AS customer, o.total, o.order_date
FROM orders o JOIN users u ON u.id = o.user_id
WHERE o.status = 'DELIVERED';

SELECT * FROM v_delivered_orders;
```

A view is a saved query - no data is stored. It simplifies access and can hide columns.

31. Pagination

```
-- offset pagination (page 3, 10 per page): simple but slow for deep pages
SELECT * FROM orders ORDER BY id LIMIT 10 OFFSET 20;

-- keyset pagination: fast for large tables
SELECT * FROM orders WHERE id > 20 ORDER BY id LIMIT 10;
```

Spring Data equivalent: `PageRequest.of(page, size)` with `Page<T>`.

17.8 DML and DDL

32. INSERT, UPDATE, DELETE

```
INSERT INTO products (name, category, price, stock) VALUES ('Air Filter', 'Engine', 420.00, 30);

UPDATE products SET stock = stock - 2 WHERE id = 1 AND stock >= 2;

UPDATE products p
JOIN order_items oi ON oi.product_id = p.id
SET p.stock = p.stock - oi.quantity
WHERE oi.order_id = 4;

DELETE FROM users WHERE id = 5;      -- fails if rows reference it (foreign key)
```

33. Upsert and ALTER

```
INSERT INTO users (id, name, email, city)
VALUES (1, 'Afrudeen', 'afu@x.com', 'Chennai')
ON DUPLICATE KEY UPDATE name = VALUES(name);

ALTER TABLE users ADD COLUMN phone VARCHAR(15);
ALTER TABLE users MODIFY COLUMN city VARCHAR(80);
ALTER TABLE users DROP COLUMN phone;
```

	DELETE	TRUNCATE	DROP
Removes	Selected rows (WHERE)	All rows	Table itself (structure + data)
Type	DML	DDL	DDL
Rollback	Yes (in a transaction)	Generally no (implicit commit)	No
Speed	Slower (row by row, logged)	Fast	Fast
Auto-increment	Keeps counter	Resets counter	Gone
Triggers	Fire	Do not fire	-

17.9 Indexes and EXPLAIN

An **index** is a sorted structure (a B+ tree in InnoDB) that lets MySQL find rows without scanning the whole table - like the index of a book. Trade-off: faster reads, slower writes, more disk.

34. Create and check an index

```
CREATE INDEX idx_orders_user_date ON orders (user_id, order_date);

EXPLAIN SELECT * FROM orders WHERE user_id = 3 AND order_date >= '2025-06-01';
-- look at: type (ref/range good, ALL = full scan), key (index used), rows (estimate)

SHOW INDEX FROM orders;
```

Concept	Explanation
Primary key	Unique + not null; in InnoDB it is the clustered index (the table data is stored in PK order)
Secondary index	Separate B+ tree pointing to the primary key
Composite index and leftmost prefix	Index (user_id, order_date) helps queries on user_id or user_id + order_date , not order_date alone
Covering index	The index contains all selected columns, so MySQL never reads the table
Index NOT used when...	A function wraps the column (WHERE YEAR(order_date)=2025), a leading wildcard (LIKE '%oil'), type mismatch, or the column has very low cardinality
Foreign keys	InnoDB automatically indexes the FK column
Where to add indexes	Columns in WHERE, JOIN ... ON, ORDER BY of frequent queries (e.g. orders.user_id)

17.10 Transactions, ACID and isolation

35. A transaction

```
START TRANSACTION;
UPDATE products SET stock = stock - 1 WHERE id = 1 AND stock > 0;
INSERT INTO orders (user_id, total, status, order_date) VALUES (1, 1299.00, 'CREATED', CURDATE());
COMMIT;
-- or ROLLBACK; to undo everything
```

36. Row locking to avoid overselling

```
START TRANSACTION;
SELECT stock FROM products WHERE id = 1 FOR UPDATE; -- other transactions wait
UPDATE products SET stock = stock - 1 WHERE id = 1;
COMMIT;
```

In JPA the equivalent is `@Lock(LockModeType.PESSIMISTIC_WRITE)` or optimistic locking with `@Version`.

ACID	Meaning	Example
Atomicity	All steps succeed or none	Order + items both saved or neither
Consistency	Data moves from one valid state to another (constraints hold)	FK and NOT NULL rules never violated
Isolation	Concurrent transactions do not see each other's partial work	Two buyers of the last item
Durability	Committed data survives a crash	Redo log / flushed to disk

Isolation level	Dirty read	Non-repeatable read	Phantom read
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	No	Possible	Possible
REPEATABLE READ (InnoDB default)	No	No	Mostly prevented (MVCC + next-key locks)
SERIALIZABLE	No	No	No (slowest)

Dirty read = reading uncommitted data. Non-repeatable read = same row read twice, different values. Phantom = same query twice, different set of rows.

17.11 Theory table: what to say out loud

Question	Answer
WHERE vs HAVING	WHERE filters rows before grouping (cannot use aggregates). HAVING filters groups after aggregation.
Primary key vs unique key	PK: one per table, not null, identifies the row. Unique: many allowed, may allow one NULL (MySQL allows multiple NULLs).
Foreign key	A column referencing another table's key to enforce referential integrity (ON DELETE CASCADE / RESTRICT).
COUNT(*) vs COUNT(col)	COUNT(*) counts rows. COUNT(col) counts non-NULL values. COUNT(DISTINCT col) counts unique non-NULLs.
CHAR vs VARCHAR	CHAR is fixed length (padded); VARCHAR variable length.
IN vs EXISTS	IN compares to a list of values; EXISTS checks whether the subquery returns any row and can stop early. NOT IN breaks with NULLs.
Normalisation	1NF atomic values, no repeating groups. 2NF no partial dependency on part of a composite key. 3NF no transitive dependency (non-key depends only on the key). Denormalise deliberately for reads (our order item snapshot).
Stored procedure vs function vs trigger	Procedure: callable block (can modify data). Function: returns a value, usable in SELECT. Trigger: runs automatically on INSERT/UPDATE/DELETE.
InnoDB vs MyISAM	InnoDB: transactions, row-level locks, foreign keys, crash-safe (default). MyISAM: none of those; legacy.
How would you find a slow query?	Slow query log, EXPLAIN, check missing indexes, avoid SELECT *, avoid functions on indexed columns, paginate.
JPA method to SQL	findByIdOrderByCreatedAtDesc(id) -> SELECT ... WHERE user_id=? ORDER BY created_at DESC. existsByEmail -> SELECT 1 ... LIMIT 1. @Query for joins.

INTERVIEW Practice drill

Close this PDF and write from memory: (1) second highest salary, (2) users without orders, (3) top earner per department, (4) duplicate emails, (5) revenue per month. If you can do those five without looking, you are ahead of most candidates.

18 Interview Q&A Bank

60+ questions with short answers - say each one aloud using your own project

Practice rule: answer in **30 to 60 seconds**, give a definition, then a concrete example from **your** project, then one trade-off. That structure impresses more than a long memorised paragraph.

18.1 Spring Boot and Spring

Q. What is Spring Boot and why use it?

A framework on top of Spring that removes boilerplate: auto-configuration, starter dependencies, an embedded server and production features (Actuator). You write business code and run `java -jar`.

Q. What happens when `SpringApplication.run()` executes?

It creates the application context, runs component scan and auto-configuration, creates beans, starts the embedded Tomcat and publishes startup events (`CommandLineRunners` run last).

Q. `@RestController` vs `@Controller`?

`@RestController` = `@Controller` + `@ResponseBody`: return values are written as JSON instead of resolving a view.

Q. How do you handle exceptions globally?

`@RestControllerAdvice` with `@ExceptionHandler` methods. In my project an abstract `BaseException` carries its HTTP status, so one handler covers all business errors.

Q. How does validation work?

Annotate DTO fields with Jakarta constraints (`@NotBlank`, `@Min`) and add `@Valid` to the `@RequestBody`. A failure throws `MethodArgumentNotValidException`, which my handler turns into a 400 with field errors.

Q. What is Spring Boot Actuator?

Production endpoints such as `/actuator/health` and `/actuator/circuitbreakers`. I exposed health and circuit breaker state.

Q. What is the difference between `application.properties` and `.yml`?

Same configuration, different syntax. YAML is hierarchical and shorter for nested keys. Profiles use `application-{profile}`.

Q. How do you keep secrets out of code?

Environment variables via `${DB_PASSWORD}` and `${JWT_SECRET}`; in production a secret manager (Vault, cloud secrets).

Q. What is a REST API and what makes it RESTful?

Resources identified by URLs, standard HTTP verbs, stateless requests, representations such as JSON, correct status codes.

Q. Which HTTP status codes did you use?

200, 201 (created), 204 (deleted), 400 (validation/business), 401, 403, 404, 503 (dependency down).

Q. PUT vs POST vs PATCH?

POST creates (not idempotent). PUT replaces a resource (idempotent). PATCH partially updates.

Q. What is idempotency and why does it matter?

Repeating a request has the same effect as doing it once. It makes retries safe - GET, PUT, DELETE are idempotent; POST is not unless you add an idempotency key.

Q. What is DTO and why not return entities?

A DTO is the API contract. Returning entities leaks columns, couples the API to the schema, and risks lazy-loading errors and infinite recursion.

18.2 Microservices and architecture

Q. Monolith vs microservices?

Monolith: one deployable, simple, strong consistency. Microservices: independently deployable services around business capabilities; better scaling and team autonomy but more operational complexity and eventual consistency.

Q. How do your services communicate?

Synchronous REST through OpenFeign for the price lookup; asynchronous events through Kafka for notifications.

Q. What does an API Gateway do?

Single entry point: routing, authentication, CORS, rate limiting, load balancing. In my project it validates the JWT and forwards `X-User-Id`.

Q. Why Eureka? What are heartbeats?

Dynamic registry: instances register and send heartbeats every ~30 s; missing heartbeats lead to eviction. Callers use logical names, not addresses.

Q. What is client-side load balancing?

The caller picks an instance from the registry list (round-robin) - `lb://PRODUCT-SERVICE`. A server-side load balancer (nginx) does it in front of the service instead.

Q. Why a database per service?

Autonomy and loose coupling: schema changes do not break others. Cost: no cross-service joins or ACID transactions.

Q. How do you handle a transaction across services?

Saga pattern: a sequence of local transactions with compensating actions (choreography via events, or an orchestrator). Avoid distributed 2PC.

Q. What is the Transactional Outbox pattern?

Write the event to an outbox table in the same DB transaction as the business change; a relay publishes it to Kafka. Prevents lost or phantom events.

Q. What is eventual consistency?

Different services may be briefly out of sync but converge. The notification appears moments after the order.

Q. What is a circuit breaker?

A guard around remote calls: after too many failures it opens and fails fast, then half-opens to test recovery. Prevents cascading failures.

Q. What is a cascading failure?

One slow service exhausts threads in its callers, which then fail for their callers. Timeouts, circuit breakers and bulkheads stop the chain.

Q. How do you secure microservices?

JWT at the edge (Gateway), role checks, services not publicly exposed, ideally token validation in each service too, HTTPS, secrets management.

Q. How would you scale this system?

Run several instances per service (Eureka + load balancing), more Kafka partitions and consumers, caching (Redis), database read replicas and indexing.

Q. What is service discovery vs configuration server?

Discovery finds *where* services are. Config server (Spring Cloud Config) centralises *configuration*.

Q. How do you version APIs?

URL prefix (`/v1/orders`), header, or media type; keep old versions until clients migrate.

Q. Observability?

Logs (correlation id), metrics (Micrometer + Prometheus/Grafana), distributed tracing (OpenTelemetry/Zipkin). Mentioned as a next step.

18.3 Java, OOP and JPA

Q. Abstract class vs interface? Where did you use each?

Abstract class: shared state and partial implementation for related classes (`BaseEntity`, `BaseException`, `NotificationHandler`). Interface: a contract many classes can implement (`JpaRepository`, `ProductClient`). A class extends one abstract class but implements many interfaces.

Q. Explain the Template Method pattern.

A base class defines the algorithm skeleton in a `final` method and leaves steps abstract. My `NotificationHandler.handle()` validates, builds a message (abstract), saves and runs a hook.

Q. `final`, `finally`, `finalize`?

`final`: constant / non-overridable / non-inheritable. `finally`: block that always runs after try/catch. `finalize`: deprecated GC hook - do not use.

Q. Checked vs unchecked exceptions?

Checked are verified at compile time (`IOException`); unchecked extend `RuntimeException`. Spring rolls back transactions on unchecked by default; my business exceptions are unchecked.

Q. `==` vs `equals()`; why override `hashCode()`?

`==` compares references; `equals` compares logical equality. Equal objects must have equal hash codes or `HashMap/HashSet` break.

Q. How does HashMap work?

Array of buckets; `hashCode` picks the bucket; collisions are chained (tree after 8 entries). Average $O(1)$ get/put.

Q. String immutability?

Strings cannot change after creation: safe to share and cache, thread-safe, good as map keys. Use `StringBuilder` for heavy concatenation.

Q. What are records?

Concise immutable data carriers. I used them for DTOs and the Kafka event: fields, constructor, accessors, `equals`, `hashCode`, `toString` generated.

Q. Streams and Optional?

Streams process collections declaratively (`map/filter/collect`). `Optional` models a maybe-absent value and avoids null checks (`orElseThrow` in my services).

Q. What is the N+1 problem and the fix?

Loading N parents triggers N extra queries for children. Fix with `JOIN FETCH`, `@EntityGraph` or batch fetching.

Q. LAZY vs EAGER; what is LazyInitializationException?

LAZY loads on access; accessing it after the session/transaction ended throws. Fix by fetching inside the transaction or via DTO projection.

Q. JPA vs Hibernate vs Spring Data JPA?

JPA is the specification; Hibernate implements it; Spring Data JPA generates repositories on top.

Q. Optimistic vs pessimistic locking?

Optimistic: `@Version` column; conflict detected at commit and retried. Pessimistic: DB lock (`FOR UPDATE`). Use optimistic when conflicts are rare.

Q. What are SOLID principles?

Single responsibility, Open/closed, Liskov substitution, Interface segregation, Dependency inversion. Layered services with constructor injection demonstrate S and D.

18.4 Angular, TypeScript, CSS

Q. What is Angular and how is it structured?

A TypeScript framework for SPAs: components, services with DI, router, forms, HttpClient. My app: `core` (services, guard, interceptor) and `features` (pages).

Q. What is a standalone component?

A component that declares its own dependencies in `imports` - no `NgModule` required. The default in modern Angular.

Q. Explain dependency injection in Angular.

A component asks for a service with `inject(ProductApi)`; Angular provides a shared instance (`providedIn: 'root'`) - like Spring `@Service` beans.

Q. How do you call a REST API?

`HttpClient` in a service returns an Observable; the component subscribes (or uses signals) and displays data. Errors handled with `catchError` or the `error` callback.

Q. What is an HTTP interceptor?

Middleware for every request/response. Mine adds `Authorization: Bearer` and logs the user out on 401.

Q. Reactive forms vs template-driven?

Reactive: form model in code (`FormBuilder`, validators) - testable, dynamic (`FormArray` for order lines).
Template-driven: `ngModel`, simpler but less control.

Q. switchMap vs mergeMap?

`switchMap` cancels the previous inner observable (search box). `mergeMap` runs all in parallel.

Q. How do you avoid memory leaks with RxJS?

`takeUntilDestroyed()`, the `async` pipe, or signals; the HTTP client completes by itself.

Q. What are signals?

Reactive primitives holding a value; reading `count()` tracks dependencies so Angular updates only what changed.

Q. How do route guards work?

A function returns `true` or a `UrlTree` to redirect. `authGuard` checks the token; `adminGuard` checks the role.

Q. How do you pass data between components?

Parent to child: `input()`; child to parent: `output()`; unrelated components: a shared service with signals/subjects; routes: params.

Q. What is lazy loading?

Loading route code only when first visited (`loadComponent/loadChildren`) to reduce initial bundle size.

Q. What is CORS and how did you solve it?

Browser same-origin policy: the page at :4200 calling :8080 needs permission. Configured in the Gateway (`globalcors`) for origin `http://localhost:4200` and the `Authorization` header.

Q. What is TypeScript's advantage?

Compile-time type checking, interfaces for API models, better refactoring and tooling; catches errors before runtime.

Q. Flexbox vs Grid? Box model?

Flexbox one-dimensional; Grid two-dimensional. Box model: content, padding, border, margin; `border-box` includes padding and border in the width.

18.5 Security, Kafka and project-specific

Q. Explain JWT.

A signed token with header, payload (claims) and signature. The server verifies the signature and expiry without storing a session - stateless authentication.

Q. Why hash passwords with BCrypt and not encrypt?

Hashing is one-way, so a leaked database does not reveal passwords; BCrypt adds a random salt and a tunable cost to slow brute force.

Q. How did you implement role-based access?

The role is a claim in the JWT; the Gateway filter allows only ADMIN to change products; Angular hides admin buttons and `adminGuard` blocks the routes (UX only - the backend enforces).

Q. Why not trust `userId` in the request body?

A client could send someone else's id. The Gateway derives it from the verified token and passes it as a header.

Q. What happens if Kafka is down when an order is created?

The order is saved but the publish fails (logged). With the outbox pattern the event would be retried; my MVP accepts this risk and documents it.

Q. How do you guarantee a notification is not processed twice?

Make the consumer idempotent: unique constraint on (orderId, type) or check before insert; use consumer groups and commit offsets after processing.

Q. What was the hardest bug and how did you debug it?

Pick a real one (CORS, secret mismatch, Eureka timing, Kafka JSON type). Describe symptom, hypothesis, tool (logs, Network tab, Eureka, Actuator), fix and lesson.

Q. What would you improve with more time?

Refresh tokens + httpOnly cookies, outbox pattern, Flyway migrations, stock reservation (Saga), Testcontainers integration tests, Docker Compose for all services, centralised config, tracing and metrics.

Q. Walk me through your architecture.

Use Figure 2.1: UI -> Gateway (JWT) -> services -> one DB each; Eureka for discovery; Feign for sync; Kafka for async; Resilience4j on the Product call.

19 Finish Line

README, GitHub, your 2-minute pitch, a demo script, troubleshooting and what to build next

19.1 README template (copy to README.md)

README.md

TEXT

```
# Order Management System

Full-stack microservices project: Spring Boot, Spring Cloud, MySQL, Kafka, JWT, Angular.

## Architecture

Angular -> API Gateway (JWT) -> User / Product / Order / Notification services.
Eureka for discovery, OpenFeign for sync calls, Kafka for events, Resilience4j for fault tolerance.

## Services and ports
| Service | Port | Database |
|---|---|---|
| service-discovery | 8761 | - |
| api-gateway | 8080 | - |
| user-service | 8081 | user_db |
| product-service | 8082 | product_db |
| order-service | 8083 | order_db |
| notification-service | 8084 | notification_db |
| Angular UI | 4200 | - |

## Prerequisites
Java 21, Node 24 + Angular CLI, MySQL 8, Docker (Kafka), Git.

## Setup
1. Create databases and the `order_app` user (see docs/database.sql).
2. Set `DB_PASSWORD` and `JWT_SECRET` environment variables.
3. `docker compose -f infrastructure/docker-compose.yml up -d`
4. Start services in order: discovery, product, user, order, notification, gateway.
5. `cd frontend/order-management-ui && npm install && ng serve`

## Default admin (local only)
admin@example.com / Admin@12345

## API docs
Swagger UI per service: http://localhost:808x/swagger-ui/index.html
Aggregated at the Gateway: http://localhost:8080/swagger-ui.html

## Key design decisions
Database per service, server-side totals, JWT validated at the gateway,
circuit breaker around product lookup, event-driven notifications.

## Known limitations / next steps
Outbox pattern, refresh tokens, stock reservation (Saga), Flyway, Testcontainers.
```

19.2 Draw the architecture diagram

1. Open **draw.io** (diagrams.net) in the browser - free, no install.
2. Redraw Figure 2.1: Angular, Gateway, five services, four databases, Kafka, Eureka. Use one colour per layer.
3. Label the arrows: **HTTP + JWT**, **Feign (sync)**, **order-created (Kafka)**, **register**.
4. Export as PNG to **docs/architecture.png** and reference it from the README.

19.3 Git: commit after every checkpoint

PowerShell

BASH

```
git add .
git commit -m "feat(product): CRUD with validation and Swagger"
git commit -m "feat(gateway): routes, CORS and JWT filter"
git commit -m "feat(order): order creation with Feign and server-side total"
git commit -m "feat(kafka): order-created event and notification handler"
git commit -m "feat(resilience): circuit breaker and 503 fallback"
git commit -m "feat(ui): login, products, orders, notifications"
git commit -m "docs: README and architecture diagram"

git branch -M main
git remote add origin https://github.com/<your-user>/order-management-system.git
git push -u origin main
```

WARNING Before you push

Run `git status`. Make sure no `.env`, no passwords and no `target/` or `node_modules/` folders are staged.

19.4 Your 2-minute project pitch

INTERVIEW Memorise the structure, not the words

"I built an **Order Management System** with Spring Boot microservices and an Angular front end. A customer logs in, browses products, places an order, and later receives a notification.

The **API Gateway** is the single entry point; it validates a **JWT** and forwards the user identity. Services register with **Eureka**. Each service has its **own MySQL database**.

When an order is placed, **Order Service** calls **Product Service** with **OpenFeign** to get the real price and stock, calculates the total on the server, saves the order, and publishes an **order-created event to Kafka**. **Notification Service** consumes it asynchronously.

I protected the Product call with a **Resilience4j circuit breaker**: if Product Service is down, orders fail fast with a clear 503 instead of hanging.

On the code side I used **abstract classes** for entities, exceptions and a Template Method notification handler, **Swagger** for API docs, and **JUnit/Mockito** tests.

Known limitations are that stock is not reserved and events are not published through an outbox - those are my next improvements."

19.5 Five-minute live demo script

Min	Show	Say
0:00	Eureka dashboard with all services UP	"Every service registers here; the gateway finds them by name."
0:45	Swagger UI of the Gateway; login request	"Login returns a JWT; the token carries role and user id."
1:30	Angular: log in as admin, add a product	"The interceptor adds the Bearer token; the gateway checks the ADMIN role."
2:30	Log in as user, place an order	"The total is computed by the backend from Product Service's price."
3:15	Workbench: <code>orders</code> , <code>order_items</code> , <code>notifications</code>	"Each service owns its database; the notification came via Kafka."

Min	Show	Say
4:00	Stop Product Service; place an order	"Circuit breaker opens: instant 503 with a clear message."
4:45	Restart; place an order again	"Half-open test succeeds and the breaker closes."

19.6 Master troubleshooting table

Symptom	Cause	Fix
Port 8082 was already in use	Old instance still running	Stop it in IntelliJ, or <code>netstat -ano findstr :8082</code> then <code>taskkill /PID <id> /F</code>
Access denied for user 'order_app'	Wrong password / user not granted	Re-run the GRANT statements; check <code>DB_PASSWORD</code>
Could not resolve placeholder 'DB_PASSWORD'	Environment variable not set for this run	Add it in the IntelliJ run configuration, or restart IntelliJ after <code>setx</code>
Unknown database 'product_db'	Databases not created	Run Chapter 3 SQL
You have an error in your SQL syntax ... order	Table named <code>order</code>	<code>@Table(name = "orders")</code>
Service missing in Eureka	Eureka not running, wrong <code>defaultZone</code> , or just slow	Start Eureka first; wait 30 s; check the yml
Gateway 503 / Unable to find instance	Target service not registered yet	Check Eureka; use uppercase service id in <code>lb://</code>
Gateway returns 404 for routes	Wrong property prefix for your Spring Cloud version	See the warning in Chapter 6
Feign: No instances available for product-service	Product Service not registered, or Eureka client missing in Order Service	Check dependency and <code>eureka.client.service-url</code>
Kafka Connection refused / Broker may not be available	Docker container not running	<code>docker compose up -d; docker ps</code>
Consumer error: Cannot convert from [String] to [OrderCreatedEvent]	Missing JsonSerializer / default type settings	Copy the consumer YAML from Chapter 9 exactly
WeakKeyException	JWT secret too short	Use 32+ characters (48 recommended)
401 on all requests	Different <code>JWT_SECRET</code> in User Service and Gateway	Use the same value in both
CORS error in browser console	Origin/headers not allowed, or double CORS config	Configure only in the Gateway
Angular NG0303 / unknown element	Material module not imported in the component	Add it to the component <code>imports</code>
ng blocked by PowerShell	Execution policy	<code>ng.cmd</code> or <code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code>
LazyInitializationException	Accessing a lazy collection outside a transaction	Map to DTO inside the <code>@Transactional</code> service method
BUILD FAILURE on dependency	Version mismatch between Boot and Cloud	Use the matching Spring Cloud release train

19.7 After the weekend: what to build next

Stretch goal	What you learn	Difficulty
Dockerise every service + docker-compose for MySQL, Kafka, services	Containers, networking, DevOps basics	Medium
Stock decrease with Saga/compensation	Distributed transactions	Hard
Transactional outbox + idempotent consumer	Reliable messaging	Hard
Refresh tokens + httpOnly cookie	Real-world token security	Medium
Flyway migrations instead of ddl-auto	Database versioning	Easy
Pagination + sorting (Pageable) and Material paginator	Spring Data and UI integration	Easy
Redis cache for product reads	Caching, @Cacheable	Medium
Testcontainers integration tests	Realistic testing with real MySQL/Kafka	Medium
Spring Cloud Config + Actuator + Micrometer/Prometheus/Grafana	Config and observability	Medium
CI pipeline with GitHub Actions (build + test)	CI/CD	Easy

19.8 Final Sunday-night checklist

Can you...
☐ Redraw the architecture from memory and narrate an order request
☐ Explain Eureka vs Gateway vs Feign vs Kafka in your own words
☐ Explain CLOSED / OPEN / HALF-OPEN with your project
☐ Walk a JWT from login to localStorage to interceptor to Gateway
☐ Say where abstract classes and Swagger are used and why
☐ Write the 5 SQL drill queries from memory
☐ Explain 10 annotations and when each is used
☐ Describe two limitations of your design and how you would fix them
☐ Demo the app end to end in under 5 minutes

TIP A final word

A project you **built, broke, fixed and explained** is worth far more in an interview than one you copied. When you hit an error, that is not lost time - it is the material for your best interview stories. Good luck, Afrudeen.